Local-Timed Negotiations: A Model Intertwining Concurrency and Time

By

Adwitee Roy

A synopsis of thesis submitted in partial fulfilment of the requirements

for the degree of Doctor of Philosophy

to

Chennai Mathematical Institute

September 2024

Plot-H1, SIPCOT IT Park,

Siruseri, Kelambakkam,

Tamilnadu - 603103

India

CHENNAI
MATHEMATICAL
INSTITUTE

Adwitee Roy

Plot-H1, SIPCOT IT Park
Padur Post, Siruseri
Tamil Nadu, India-603103
E-mail:adwitee@cmi.ac.in

DECLARATION

I declare that the thesis entitled **"Local-Timed Negotiations: A Model Intertwining Concurrency and Time"** submitted by me for the degree of **Doctor of Philosophy in Computer Science** is the record of academic work carried out by me under the guidance of Professor B Srivathsan and this work has not formed the basis for the award of any degree, diploma, associateship, fellowship or other titles in this University or any other University or Institution of Higher Learning.

Adwitee Roy
Chennai Mathematical Institute
September 2024.

CHENNAI
MATHEMATICAL
INSTITUTE

B Srivathsan

Plot-H1, SIPCOT IT Park
Padur Post, Siruseri
Tamil Nadu, India-603103
E-mail:sri@cmi.ac.in

CERTIFICATE

I certify that the thesis entitled **"Local-Timed Negotiations: A Model Intertwining Concurrency and Time"** submitted for the degree of **Doctor of Philosophy in Computer Science** by "Adwitee Roy" is the record of research work carried out by him under my guidance and supervision, and that this work has not formed the basis for the award of any degree, diploma, associateship, fellowship or other titles in this University or any other University or Institution of Higher Learning. I further certify that the new results presented in this thesis represent his independent work in a very substantial measure.

Chennai Mathematical
Institute
Date: September 2024.

B Srivathsan
Supervisor.

ACKNOWLEDGEMENT

Contents

1 Introduction **1**

1.1 Motivation . . . 2

1.1.1 Concurrency, time, and coordinated interaction . . . 2

1.1.2 Why explicit multiparty interaction matters . . . 3

1.1.3 Verification and the reachability problem . . . 4

1.1.4 Why local time changes the question . . . 5

1.2 From timed automata and negotiations to local-timed negotiations . 6

1.2.1 Timed automata and the global-time paradigm . . . 6

1.2.2 Negotiations and explicit multiparty synchronization . . . 7

1.2.3 A running example: an ATM workflow . . . 8

1.2.4 Why combining the two models is not routine . . . 10

1.2.5 A compact comparison of the three viewpoints . . . 10

1.3 Local-timed negotiations at a glance . . . 11

1.3.1 Local reference time and local clocks . . . 11

1.3.2 Time-synchronizing and non-synchronizing interactions . . . 11

1.3.3 A timed reading of the ATM example . . . 12

1.3.4 Why local time is attractive . . . 12

1.3.5 Why local time is difficult . . . 13

1.4 Contributions and main results . . . 14

1.4.1 The central negative result . . . 14

1.4.2 Decidable extremes of the synchronization spectrum . . . 14

1.4.3 Decidability via bounded drift . . . 15

1.4.4 Decidability via communication structure . . . 15

1.4.5 A summary of the landscape . . . 16

1.4.6 What is conceptually new in the thesis . . . 16

1.5 Organization of the thesis . . . 17

1.6 Related work . . . 18

1.6.1 Timed automata and local-time semantics . . . 18

1.6.2 Timed concurrency models beyond timed automata . . . 18

1.6.3 Negotiations and workflow-style concurrency . . . 19

1.6.4 Drift-boundedness and structural restrictions . . . 19

1.6.5 Position of the thesis . . . 20

1.7 What to carry forward . . . 20
1.8 Coming next . . . 20

I Adding local time constraints to negotiations 21

2 Preliminaries 22
2.1 Negotiations . . . 22
2.1.1 Syntax and operational semantics . . . 23
2.1.2 Graphical conventions . . . 25
2.1.3 A running example: ATM PIN reset . . . 25
2.1.4 A richer example: pollination and climate mismatch . . . 27
2.1.5 Relation to Standard Negotiations with Transformers . . . 30
2.1.6 Reachability and known complexity . . . 31
2.1.7 Relation to standard negotiations and to Petri nets . . . 32
2.2 Timed Automata . . . 33
2.2.1 Syntax and Semantics . . . 33
2.2.2 Region Abstraction and Decidability [] . . . 34
2.3 Networks of timed automata and the local-time contrast . . . 38
2.3.1 Global-time semantics for networks . . . 38
2.3.2 Why this is not yet the right model . . . 38
2.4 From these ingredients to LTNs . . . 39
2.5 Terminology summary . . . 39
2.6 What to carry forward . . . 41
2.7 Coming next . . . 41

3 Local-Timed Negotiations 42
3.1 Motivation: Why Local Time? . . . 42
3.2 Related work . . . 44
3.2.1 Comparison with Timed Negotiations [**?**] . . . 44
3.3 Introducing Local-Timed Negotiations: An Informal Example . . . 44
3.3.1 Scenario: ATM Transaction with Timeouts . . . 45
3.3.2 Modeling with Local Clocks . . . 45
3.3.3 Preview: Synchronization . . . 47
3.4 Formal Definition . . . 47
3.5 Semantics . . . 49
3.5.1 Markings, Valuations, and Configurations . . . 49
3.5.2 Transitions: Delays and Actions . . . 50
3.5.3 Location Reachability . . . 51
3.5.4 Design choices and modelling rationale . . . 51
3.6 Expressiveness . . . 52
3.6.1 Causal Enforcement Through Synchronization . . . 53
3.6.2 Non-regular untimed languages . . . 54

3.7 Undecidability of reachability . . . 57
3.7.1 Two-counter machines . . . 57
3.7.2 Overview of the reduction . . . 58
3.7.3 Instruction gadgets . . . 59
3.8 Summary and perspective . . . 65
3.9 Coming next . . . 66

II Decidable subclasses for reachability in LTNs 70

4 Synchronization Extremes: Free and Always-Synchronizing LTNs 72
4.1 Synchronization-Free LTNs . . . 73
4.1.1 Definition and Behavior . . . 73
4.1.2 Product-region abstraction . . . 75
4.1.3 The product-region automaton . . . 79
4.1.4 Product-Region Calculations on an Example . . . 79
4.2 Always-synchronizing LTNs . . . 85
4.2.1 Definition and Intuitive Behavior . . . 85
4.2.2 Monotone runs . . . 85
4.2.3 Reduction to timed automata . . . 90
4.3 Summary and perspective . . . 92
4.3.1 Comparing the Two Fragments . . . 93
4.3.2 Why Both Are PSPACE (Not Lower, Not Higher) . . . 93
4.3.3 The Design Space . . . 94
4.4 Coming next . . . 94

5 Drift-Bounded and Acyclic Local-Timed Negotiations 95
5.1 Introduction: From Synchronization to Drift . . . 95
5.2 Drift: Definition, Intuition, and Examples . . . 97
5.2.1 Geometric Intuition . . . 98
5.2.2 Motivating Examples . . . 98
5.2.3 Drift-Bounded Runs and LTNs . . . 100
5.3 The k-Drift Bounded Fragment . . . 101
5.3.1 Region Abstraction Under Bounded Drift . . . 101
5.3.2 The k-Region Automaton . . . 103
5.3.3 Decidability and Complexity . . . 103
5.4 Deciding Drift Boundedness . . . 104
5.4.1 Universal k-Drift Boundedness (Fixed k) . . . 104
5.4.2 Universal Drift Boundedness (Existence of k) . . . 105
5.5 Acyclic Local-Timed Negotiations . . . 105
5.5.1 Acyclicity Implies Drift Boundedness . . . 105
5.5.2 NP-Completeness of Reachability . . . 106
5.6 Summary and Perspective . . . 108

5.7 Coming Next: Connected Communication 110

6 Connectedly Communicating LTN 111

6.1 Introduction 111

6.1.1 The Connectedly-Communicating Fragment 111

6.2 Understanding Connectedly Communicating LTNs Through Examples 113

6.3 Connectedly communicating LTNs 118

6.3.1 Marking Graphs and Communication Graphs 118

6.4 Main Theorem 120

6.4.1 Proof Roadmap 121

6.4.2 Trace Representation of a Run 123

6.4.3 Tightening a Run 128

6.4.4 Tight Runs of Connectedly-Communicating LTNs are Drift-Bounded 139

6.4.5 Extensions and Variations 145

6.5 Conclusion 148

6.5.1 Summary of Contributions 149

6.5.2 Future Directions 149

6.5.3 Closing Remarks 150

7 Conclusion and Future Directions 151

7.1 Conclusion 151

7.2 Broader Directions and Outlook 153

7.2.1 Counting and Enumeration Problems 153

7.2.2 Quantitative Extensions: Costs, Weights, and Counters . . . 154

7.2.3 Structural Parameters and Well-Structuredness 154

7.2.4 Beyond Reachability: Broader Verification and Synthesis . . 155

7.2.5 Connections to Other Infinite-State Models 156

7.2.6 Toward Practical Modeling and Tool Support 156

Chapter 1

Introduction

This chapter introduces the central verification problem of this thesis: *reachability for local-timed negotiations*. It highlights the unique challenges posed by combining local time with multiparty interaction and provides a roadmap of the thesis, including (un)decidability results.

This chapter is motivational and structural. It tells the reader why the thesis matters and what its main results are. Chapter 2 then provides the formal machinery used throughout the rest of the dissertation.

Concurrent systems rarely consist of a single thread of control evolving against a single clock. They are built from interacting components that proceed partly independently, occasionally coordinate, and often have quantitative timing requirements that matter for correctness. This thesis studies such systems through the model of *local-timed negotiations* (LTNs), which combine explicit multiparty interaction with local real-time behavior. The central verification problem considered throughout is *reachability*: given an initial configuration and a target control point, does there exist an execution that reaches the target while respecting the timing constraints of the model?

The thesis asks a structural question about this verification problem: *what happens when concurrency is governed by explicit multiparty interaction, but time is local rather than global?* The answer is not a simple transfer of known timed-automata results. Once time is allowed to evolve independently for different agents and synchronization is made selective rather than implicit, reachability becomes markedly more expressive. In full generality, that expressiveness leads to undecidability. At the same time, the thesis shows that this negative result is not the end of the story. By imposing natural semantic or structural restrictions, one can recover decidability and obtain a precise landscape of tractable fragments.

This introduction is organized around that landscape. Section 1.1 motivates the need for a model that treats both timing and multiparty interaction as first-class. Section 1.2 places LTNs relative to timed automata and negotiations, but only at a high level; the formal machinery is deferred to Chapter 2. Section 1.3

presents the central modeling ideas behind LTNs and explains why local time fundamentally changes the verification picture. Section 1.4 summarizes the main results of the thesis and the overall decidability map. Sections 1.5 and 1.6 close the chapter with the organization of the thesis and a brief discussion of related work.

1.1 Motivation

1.1.1 Concurrency, time, and coordinated interaction

In the world we live in, concurrency is ubiquitous. Imagine that we are walking inside a forest. Different birds, animals and insects are talking around us. Suddenly we see a beautiful bird peeping from a branch of a sunlit mossy tree. Our brain here is processing multiple sensory information, motor control, and cognition concurrently. Images, sounds, movement, forming memories, making a decision whether to capture this moment in our camera or not— all are processed concurrently by different parts of our brain. At the same time, our heart rate, breathing, and reflexes are being controlled by other parts. These processes evolve largely independently, yet remain sufficiently coordinated to produce coherent behaviour. Such parallel activity illustrates a fundamental property of natural systems: concurrency.

Natural systems process many forms of information in parallel, and engineered systems increasingly do the same. A mobile robot combines sensing, localization, planning, and actuation while interacting continuously with its environment. A communication protocol manages multiple sessions, timers, retransmissions, and acknowledgements concurrently. A medical device monitors physiological signals while coordinating diagnosis and control. Even when such systems are designed modularly, their correctness is rarely determined by the behavior of one component in isolation. It depends on how several components evolve together.

This observation has two consequences for formal modeling. First, a useful model of such systems must capture *independent evolution*: components often make progress without waiting for one another. Second, it must capture *coordination*: components sometimes need to interact in a structured way, and the form of that interaction matters. In many settings, that coordination is not merely pairwise. Several components may have to participate simultaneously in a joint interaction, commit to a common outcome, and then continue according to that joint decision.

Time adds a second layer of difficulty. In many systems, correctness is not purely qualitative. It is not enough that an event eventually happens; it must happen within a deadline, after a minimum delay, or in a precise order relative to other time-constrained activities. A collision-avoidance controller must combine sensor readings fast enough for braking decisions to be effective. A pacemaker

must coordinate sensing and stimulation within tight time windows. A distributed protocol may tolerate independent local progress for some time, but later require the participants to synchronize before continuing. In such settings, time is not an annotation on top of behavior; it is part of the behavioral specification itself.

These two features—concurrency and quantitative time—have long been studied in formal verification through a variety of complementary models. Some provide especially natural descriptions of interaction structure, while others offer a direct treatment of quantitative timing constraints. Local-timed negotiations are motivated by bringing these perspectives together in a setting that makes both interaction patterns and local timing constraints explicit. The present thesis is motivated by the need for a model in which *interaction structure*, *local progress*, and *timing* can all be represented explicitly.

1.1.2 Why explicit multiparty interaction matters

An important aspect of concurrent systems is that coordination often takes place through interactions involving several agents at once. Classical state-based models often reduce multiparty interactions to combinations of smaller steps: shared variables, binary communications, or transitions whose effects are spread across an underlying state space. Such encodings are powerful, but they can hide the real coordination structure of a system. In verification, that hidden structure matters. It affects how independence is recognized, how state-space reductions are justified, and how one reasons about causality.

Negotiation-based models address this issue by taking *atomic multiparty interaction* as primitive. A negotiation node specifies which agents participate, which outcomes are possible, and how those outcomes determine the subsequent local readiness of the participants. This style of modeling has two advantages for the present thesis. First, it makes concurrency explicit: two interactions with disjoint participant sets are structurally independent. Second, it makes synchronization explicit: if several agents have to act together, that requirement is visible in the model rather than emerging indirectly from a lower-level encoding.

Once time enters the picture, this distinction becomes even more important. In globally timed models, two actions that are structurally independent may nevertheless become coupled through the ambient passage of time. In interaction-centric models, by contrast, one can ask more finely *when* time should matter jointly and *when* it should remain local. This is one of the guiding ideas behind LTNs.

A natural local-time intuition: flowering and pollination

A useful way to think about local time is through temporal mismatch in natural systems. Climate change may shift the local flowering schedule of plants without producing the same shift in the emergence schedule of their pollinators. The resulting problem is not simply that "something happens late"; different participants evolve according to partly separate local temporal rhythms, and successful coordination depends on whether those rhythms align when interaction becomes possible.
This thesis returns to that idea through a stylized pollination example. In Chapter 2, the example is presented formally as a negotiation in order to illustrate branching, delayed consequences of choice, and the possible growth of the marking graph. At the motivational level, however, the point is simpler: local time matters even in naturally interaction-driven systems, and mismatches of local timing can be just as important as the interaction structure itself. That intuition carries directly into LTNs, where timing constraints may be local to different agents but their consequences become visible at interactions.

1.1.3 Verification and the reachability problem

The practical motivation for studying concurrent real-time systems is clear: timing or coordination failures in such systems can have serious consequences, from protocol violations and service outages to safety-critical malfunction. The theoretical motivation is equally compelling. These systems lie at the intersection of several classical themes in formal methods—infinite-state verification, concurrency, partial-order reasoning, and real-time analysis—and they raise natural questions about the boundary between expressive power and algorithmic tractability.

Among verification problems, the most basic and most revealing one is *reachability*. Given an initial configuration and a target configuration, is there an execution that reaches the target? Reachability already subsumes several familiar questions:

- **Safety**: can a bad state be reached?
- **Deadlock detection**: can the system reach a configuration from which no further progress is possible?
- **Goal attainment**: can a desired control point, outcome, or marking be reached?

For this reason, reachability serves as a canonical benchmark for the algorithmic behavior of a model. If reachability is already undecidable, richer verification tasks inherit that hardness. If reachability is decidable, the proof techniques often reveal which aspects of the model are fundamentally responsible for tractability.

For concurrent systems, reachability is hard even without time. The source of this hardness is the familiar *state-space explosion*: independent actions can interleave in many ways, causing the number of reachable states to grow rapidly. Partial-order reduction techniques mitigate this problem by exploiting commutativity of independent actions, thereby avoiding redundant exploration of behavior that differs only by interleaving.

Timed systems complicate this picture. Clocks can create dependencies between actions that would otherwise be independent, and global-time semantics may force components to advance together even when there is no direct interaction between them. As a result, the availability of partial-order reasoning depends heavily on how time is modeled. One of the motivations of this thesis is precisely to study a framework where structural independence is explicit and where synchronization of time is a modeling choice rather than a built-in semantic assumption.

1.1.4 Why local time changes the question

A natural way to add time to a concurrent model is to assume a single global notion of time. This is the classical approach in timed automata and in networks of timed automata: all clocks advance uniformly, and the system evolves against a shared temporal background. This choice is elegant and powerful, and it has led to a mature verification theory. But it is not the only meaningful way to model time.

In many distributed or loosely coupled systems, what matters operationally is not a single global time, but the local progression of time for each participant. Components may measure elapsed time relative to their own actions, their own deadlines, or their own waiting periods. They may proceed independently for long stretches and synchronize only at designated coordination points. In such a setting, treating time as inherently global can obscure the very patterns of independence one hopes to analyze.

Local-timed negotiations adopt the opposite viewpoint. Each agent evolves according to its own local reference time. Timing constraints are expressed through local clocks, and some interactions may additionally require participating agents to synchronize their local reference times. The result is a model in which both structural interaction and temporal coordination are explicit. This makes it possible to ask a more refined verification question: *how much timing freedom can a concurrent system retain before reachability becomes undecidable?* The remainder of the chapter positions this question relative to existing models and summarizes the answer developed in the thesis.

1.2 From timed automata and negotiations to local-timed negotiations

Local-timed negotiations are best understood as arising from two complementary traditions. From timed automata they inherit quantitative timing constraints, real-valued clocks, guards, and resets. From negotiations they inherit an interaction-centric view of concurrency in which atomic multiparty interactions are primitive. The central contribution of the model is not merely to combine these ingredients syntactically, but to combine them under a *local-time semantics* that preserves independence wherever synchronization is not explicitly required.

1.2.1 Timed automata and the global-time paradigm

Timed automata, introduced by Alur and Dill [**?**], are one of the foundational models for real-time verification. A timed automaton is a finite-state control graph equipped with real-valued clocks. Transitions can test clocks against constants, reset selected clocks to zero, and thus express timing constraints such as deadlines, waiting periods, and response windows. The reachability problem asks whether a target state can be reached from the initial state by alternating time delays and discrete transitions.

The decisive feature of timed automata for the present discussion is that *all clocks evolve with the same global time*. If a delay of length δ is taken, every clock increases by δ. Although the underlying state space is infinite and uncountable, one can pass to a finite abstraction via *regions*; in practice, more efficient symbolic representations based on *zones* are then used to explore reachable valuations algorithmically [**?**, **?**].

The model is particularly successful when a system really is governed by a common global time, or when such a global-time view is a faithful abstraction. But from the point of view of true concurrency, the semantics has a cost: independent components are still coupled by the ambient flow of time. In a network of timed automata, two local components may be structurally unrelated and still advance together because delay transitions are global. This does not make the model less useful, but it does mean that the semantics of time already imposes a form of synchronization before any explicit synchronization action occurs.

For this thesis, timed automata serve both as inspiration and as a contrast. They provide the clock-based language in which local timing constraints will be expressed. At the same time, their standard semantics highlights the modeling decision that LTNs reject: the assumption that temporal progress is global by default.

Figure 1.1 shows an example timed automaton $\mathcal{A}_1$ with two clocks x and y, initial state q_0, accepting state q_2, and alphabet $\{a, b\}$.

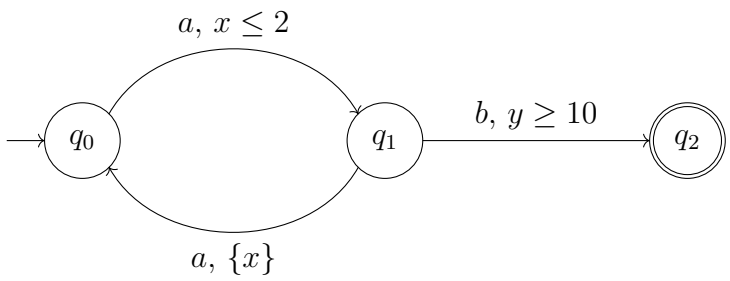


FIGURE 1.1: A timed automaton $\mathcal{A}_1$. Both clocks x and y start at 0 in state q_0. The transition on a from q_0 to q_1 has the guard $x \leq 2$, forcing the system to spend at most 2 time units before taking it. The transition on a from q_1 to q_0 has no guard and resets x. The guard on b from q_1 to q_2 requires that at least 10 time units have elapsed since the start, via the constraint $y \geq 10$.

Figure 1.2 depicts part of the (infinite) transition system of $\mathcal{A}_1$, showing some possible configurations and transitions. Starting from the initial configuration $(q_0, \langle 0, 0\rangle)$, the system may let time elapse in q_0 for any duration between 0 and 2 units before taking the transition on a to q_1. For instance, after 0.8 time units, the clock valuation becomes $\langle 0.8, 0.8\rangle$. Once in q_1, the automaton can again delay, and may take the loop on a (resetting x) or eventually take the transition on b when $y \geq 10$. Different choices of delay durations lead to uncountably many distinct runs.

1.2.2 Negotiations and explicit multiparty synchronization

Negotiations, introduced by Esparza and Desel [**?**], provide a different starting point. They model concurrent behavior as a workflow of *atomic negotiations*. Each negotiation node has a nonempty set of participating agents, called its domain, and a finite set of possible outcomes. When a node occurs, all agents in its domain participate jointly, agree on one of the available outcomes, and update their future readiness accordingly.

This style of modeling makes two aspects of concurrency transparent. First, it makes *synchronization* explicit: if a node involves agents p, q, and r, then the interaction among them is atomic and visible at the top level of the model. Second, it makes *independence* explicit: nodes with disjoint domains can proceed independently.

Operationally, the state of a negotiation is given by a *marking*, which records, for each agent, the set of nodes that the agent is currently ready to participate in. A node is enabled if all its participating agents are ready for it. When an enabled node occurs with a chosen outcome, the readiness sets of the participating agents are updated according to the outcome, while the readiness of non-participants remains unchanged. The reachability problem then asks whether a target marking, outcome, or control point can be reached from the initial marking.

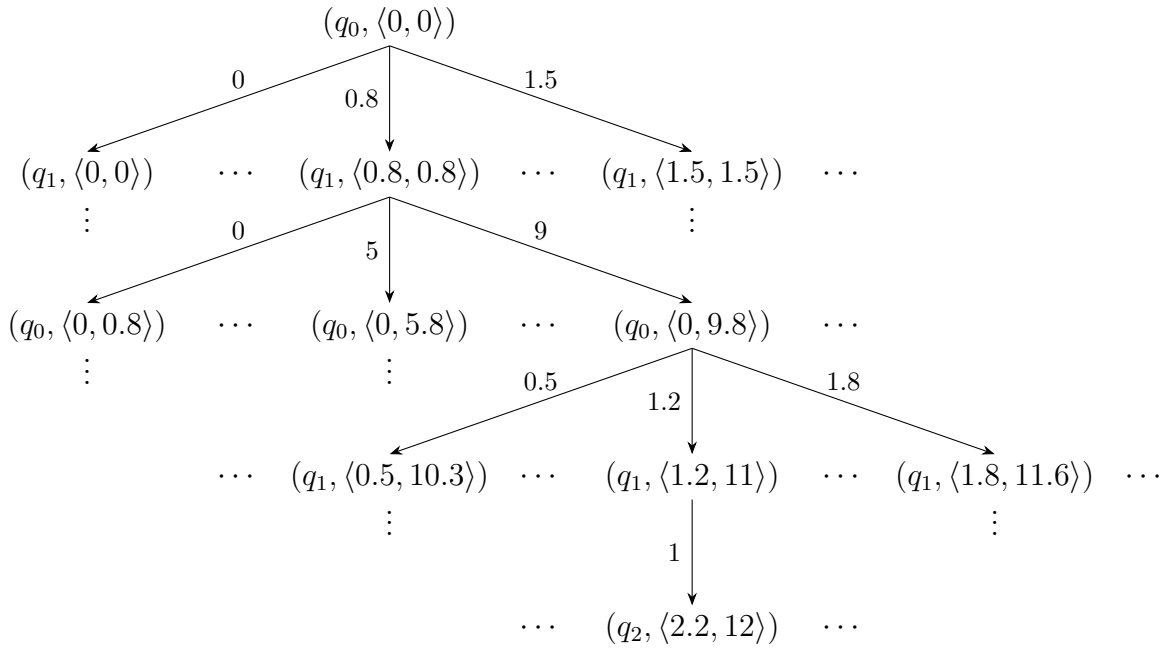


FIGURE 1.2: Part of the transition system of the timed automaton $\mathcal{A}_1$, showing several possible runs.

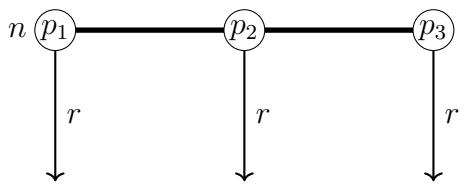


FIGURE 1.3: A simple node n with three processes $\{p_1, p_2, p_3\}$ in its domain, and the set of outcomes is $\{r\}$

Negotiations provide a convenient starting point for the present thesis. They can be used to describe workflows, distributed protocols, and structured coordination patterns, and they are naturally related to workflow Petri nets and business-process models [**?**, **?**]. For our purposes, the key point is that they represent *multiparty* interaction directly. This gives a clear structural layer to which local quantitative timing constraints can later be added.

1.2.3 A running example: an ATM workflow

To illustrate the graphical notation for negotiations, Figure 1.3 shows a single node n with domain $\{p_1, p_2, p_3\}$ and one outcome r. The thick horizontal bar represents the atomic interaction, the white circles indicate the participating processes, and the outgoing arcs labelled r describe how readiness is updated after that outcome occurs.

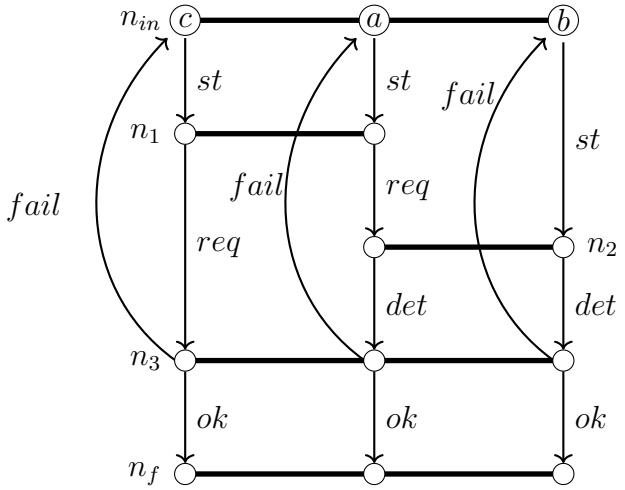


FIGURE 1.4: A distributed negotiations modelling a transaction between a customer, an ATM and a bank

1.2.3.1 A simple example: ATM workflow

Before presenting a more elaborate scenario, we introduce negotiations through a simple example. Consider a transaction in an ATM machine (a), where a customer (c) wants to withdraw money from her bank (b). Here, a, c, and b are the *processes* in the system. Their direct interactions are represented by thick horizontal lines (nodes). After each interaction, the participating agents decide on an outcome, represented by arrows. Initially, all agents are in the node n_{in} node. They choose the outcome st to start the transaction. Agents a and c go to node n_1 and b goes to n_2.The customer gives her card details and requests for money withdrawal in the ATM at node n_1 by the outcome req. At n_2, the ATM conveys this request to the bank and sends the details through the outcome det. All three processes talk to each other at n_3, where the bank checks if there is sufficient money in the account and let the customer know via the ATM. They select ok if the transaction is approved and go to n_f otherwise they go back to n_{in} by choosing the outcome $fail$.

This example is also useful conceptually because it foreshadows the timed extension. In an untimed negotiation, the ATM scenario captures *who* interacts with whom and *in what order* outcomes are possible. What it does not capture is that the customer may wait only so long before giving up, or that the bank may require an OTP to be validated within a limited time window. Introducing such constraints while keeping time local to the relevant participants is precisely the role of LTNs.

1.2.4 Why combining the two models is not routine

At first sight, one might expect that adding clocks to negotiations is straightforward: attach guards and resets to outcomes, let clocks evolve, and inherit the verification theory from timed automata. The thesis shows that this expectation is false. The key reason is semantic.

If one were to add clocks to negotiations while keeping a global-time semantics, the resulting model would collapse toward a timed-automata-like product construction. All agents would still advance under the same delay, and multiparty interaction would be timed against a shared global clock. This produces a meaningful model, but not the one studied here.

LTNs instead assign a local reference time to each agent and allow those reference times to evolve independently. Some interactions may then require the participating agents to synchronize their reference times, while others do not. Once this selective synchronization is permitted, a new phenomenon appears: relative timing differences between agents can accumulate, persist, and later be compared or eliminated through synchronizing interactions. This is exactly the phenomenon that drives the negative results of the thesis.

Thus LTNs are not simply "timed negotiations." They are negotiations enriched with a *local-time semantics* under which synchronization itself becomes part of the expressive power of the model.

1.2.5 A compact comparison of the three viewpoints

The contrast among timed automata, negotiations, and LTNs can be summarized informally as follows.

Model	**What is primitive?**	**How does time evolve?**	**What is explicit?**
Timed automata	local control transitions with guards and resets	globally, through a shared delay semantics	timing constraints; less explicit concurrency structure
Negotiations	atomic multiparty interaction and outcomes	not modeled quantitatively	synchronization and independence in the control structure
Local-timed negotiations	atomic multiparty interaction plus local timing constraints	locally, per agent; synchronization optional at selected nodes	both interaction structure and temporal coordination

This table captures the conceptual niche of LTNs. They are designed to expose exactly the interaction between explicit concurrency and local time that is hidden

in one direction by untimed negotiation models and in the other by global-time real-time models.

1.3 Local-timed negotiations at a glance

This section presents the central ideas of LTNs informally. The full syntax and semantics are given in Chapter 3; here the goal is to give a clear conceptual picture of what the model adds and why the reachability problem becomes interesting.

1.3.1 Local reference time and local clocks

Each agent in an LTN has its own *local reference time*. Intuitively, this is the agent's own notion of elapsed time as the system executes. In addition, each agent may carry ordinary local clocks, which evolve with that agent's reference time and can be tested or reset at negotiation outcomes. In this way, LTNs preserve the familiar clock-based language of timed automata while changing the ambient temporal semantics.

Two kinds of timing information are therefore present:

- **Local clocks**, which are used in guards and can be reset, and
- **Reference times**, which record the agent's accumulated local time and are used to define synchronization.

The essential design choice is that reference times are not assumed to evolve in lockstep. Between interactions, one agent may accumulate more local time than another. This means that the system retains genuine temporal asynchrony even while maintaining a structured interaction graph.

1.3.2 Time-synchronizing and non-synchronizing interactions

A negotiation node in an LTN still represents an atomic multiparty interaction, but it may now come in one of two flavors.

A *non-synchronizing* interaction behaves like an ordinary timed local step embedded in a negotiation node: the participating agents must satisfy the relevant local clock guards, and the outcome may reset some clocks, but their reference times need not be equal. The interaction respects their local temporal histories without forcing alignment.

A *time-synchronizing* interaction adds an equality constraint on the participating agents' reference times. Such an interaction can occur only when the participants have aligned their local times, and once it occurs, the alignment becomes an explicit part of the execution. This is the mechanism that allows LTNs

to model both loosely coupled and tightly coordinated behavior inside the same framework.

Seen this way, LTNs do not simply attach clocks to a concurrent model. They distinguish between two notions that are conflated in standard global-time semantics:

1. *time passing*, and
2. *agents being temporally aligned.*

In LTNs, time may pass without alignment, and alignment may be required only at chosen interactions.

1.3.3 A timed reading of the ATM example

Return to the ATM scenario. Suppose that after the customer initiates the transaction, the ATM must receive the customer's response within a bounded local time window, while the bank requires that an OTP validation complete within another local deadline. Under a global-time semantics, such requirements would be phrased against a single shared time axis. Under LTNs, they can instead be expressed against the local clocks of the relevant agents.

For instance, the customer may carry a clock that is reset when the request is issued and later checked when the OTP is entered. The bank may carry a different clock, reset when the OTP is sent and checked when validation occurs. These constraints are local: they refer to the temporal obligations of the relevant agents. If a later interaction additionally requires the ATM and the bank to be temporally aligned, that is modeled through a time-synchronizing negotiation node. Thus the model distinguishes between *having a local deadline* and *having to synchronize local time with another participant.*

This distinction is the source of both the expressive strength and the verification complexity of LTNs.

1.3.4 Why local time is attractive

The local-time semantics of LTNs is not only mathematically interesting; it is also conceptually attractive for several modeling reasons.

Explicit independence. If two negotiation nodes involve disjoint sets of agents and neither node synchronizes time, then their temporal progress is genuinely independent. There is no hidden global-time coupling forcing them to advance in lockstep.

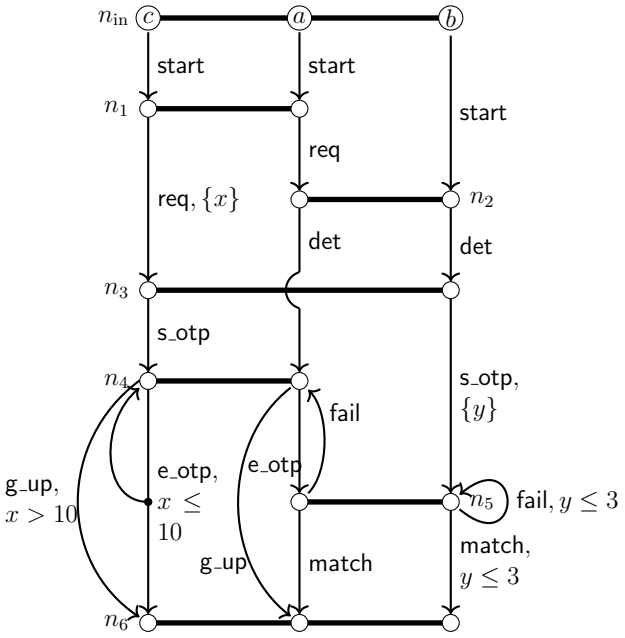


FIGURE 1.5: ATM transaction as a local-timed negotiation. Customer c has local clock x (reset at req, checked at e_otp and g_up). Bank b has local clock y (reset at s_otp, checked at match and fail). Annotations like "$x \leq 10$" are guards; "$\{x\}$" indicates clock reset.

Closer to distributed intuition. In distributed systems, different components often measure time locally. Even when a physical global time exists, algorithms are typically written against local timers, local waiting periods, and local deadlines. LTNs reflect this modeling stance directly.

Selective coordination. Real systems rarely sit at one of two extremes—fully asynchronous or fully synchronous. They are often partly one and partly the other. LTNs support this by allowing time-synchronization to be attached only to selected interactions.

Potential for partial-order reasoning. Because the model preserves structural independence rather than erasing it under a global-time semantics, it invites connections with partial-order methods. Although such algorithmic developments lie mostly beyond the scope of this thesis, the model is designed in a way that keeps this avenue open.

1.3.5 Why local time is difficult

The same feature that makes LTNs expressive also makes verification difficult. Once local reference times can diverge and later be required to align, the system can use relative timing differences as a form of stored information. Informally:

- a non-synchronizing phase allows agents to drift apart in local time,
- a later time-synchronizing interaction can compare or constrain that drift,
- and the alternation of these two behaviors can simulate unbounded computation.

This is the conceptual core of the undecidability result proved later in the thesis. It also explains the structure of the positive results: each decidable fragment works by disabling, bounding, or structurally controlling this interplay between *drift* and *synchronization*.

The contribution of the thesis is therefore not merely to define LTNs, but to identify the precise conditions under which their reachability problem becomes tractable.

1.4 Contributions and main results

This thesis provides a systematic study of the decidability and complexity of reachability for local-timed negotiations. Its main results can be read as a map of what happens when one varies the strength of synchronization and the extent to which local times can diverge.

1.4.1 The central negative result

The first main result is that general LTNs are too expressive for decidable reachability. When time-synchronizing and non-synchronizing interactions can be mixed arbitrarily, the model can encode unbounded computation. The proof, developed in Chapter 3, gives a reduction from two-counter machines. At a high level, counter values are represented by differences between local reference times, while selective synchronization is used to compare or transfer these differences.

This shows that the difficulty of LTNs is not due merely to the presence of clocks. Rather, it is the combination of three features that matters:

1. local evolution of time,
2. persistence of relative timing differences, and
3. selective synchronization that can later constrain those differences.

This result provides the negative boundary against which all later decidability results are measured.

1.4.2 Decidable extremes of the synchronization spectrum

The thesis then identifies two structurally extreme fragments in which reachability becomes decidable and, in fact, PSPACE-complete.

Synchronization-free LTNs. In this fragment, no interaction synchronizes local reference times. Agents still carry local clocks and can still be constrained by local guards, but there is no mechanism for comparing their accumulated temporal drift. The reachability analysis can therefore be organized via a product-style region abstraction across agents, leading to PSPACE-completeness.

Always-synchronizing LTNs. At the opposite extreme, every interaction synchronizes local reference times. In this setting, the local-time semantics collapses toward a global-time behavior: although the model remains interaction-centric, every reachable target admits a monotone witness run that can be interpreted through a timed-automata-style analysis. Reachability is again PSPACE-complete.

These two results are important conceptually. They show that undecidability does not come from local time alone, nor from synchronization alone. It arises from the *combination* of independent drift and selective synchronization.

1.4.3 Decidability via bounded drift

Between the two synchronization extremes lies a broader semantic class: *drift-bounded LTNs*. Informally, an LTN is drift-bounded if every discrete behavior admits some timed realization in which the difference between local reference times never grows beyond a fixed constant.

This is a natural intermediate notion. It allows selective synchronization and local-time evolution to coexist, but forbids them from generating unbounded temporal divergence. The thesis shows that this condition is sufficient to recover a finite abstraction and hence decidability of reachability. For fixed drift bound k, the resulting problem is PSPACE-complete. Chapter 5 also studies the meta-problem of deciding drift boundedness itself and analyzes the special case of acyclic LTNs, where acyclicity implies bounded drift and yields an NP-complete reachability problem.

Thus drift-boundedness plays two roles in the thesis: it provides both a semantic explanation of why some fragments are decidable and a bridge between the undecidable general model and the structurally restricted classes that follow.

1.4.4 Decidability via communication structure

The final major contribution is a structural fragment called *connectedly-communicating LTNs*. Here the restriction is not stated directly in terms of time, but in terms of how synchronization is distributed across loops in the marking graph. Roughly speaking, every cyclic behavior must contain enough synchronizing communication to connect the participating agents.

The intuition is that if all agents in a loop are repeatedly tied together through synchronization paths, then no one agent can drift arbitrarily far away without that drift eventually being absorbed by the rest of the system. The thesis formalizes this idea through communication graphs on loops and proves that connectedly-communicating LTNs are drift-bounded. Decidability of reachability then follows from the drift-bounded framework.

This result is particularly important because it shows that decidability can be recovered not only by bounding time semantically, but also by imposing a natural structural condition on how synchronization is arranged in the control graph.

1.4.5 A summary of the landscape

Taken together, the main results of the thesis can be summarized as follows.

Fragment	**Reachability**	**Main reason**
General LTNs with mixed synchronization	Undecidable	unbounded drift can be created and later tested
Synchronization-free LTNs	PSPACE-complete	drift exists but cannot be compared via synchronization
Always-synchronizing LTNs	PSPACE-complete	drift cannot persist across interactions
k-drift-bounded LTNs	PSPACE-complete (fixed k)	relative timing differences stay finitely representable
Acyclic LTNs	NP-complete	acyclicity prevents unbounded accumulation of drift
Connectedly-communicating LTNs	Decidable	communication structure forces bounded drift

This table is more than a list of results. It expresses the central explanatory thesis of the dissertation: *reachability in LTNs becomes hard precisely when relative timing information can accumulate without bound and later be interrogated by selective synchronization.* Each decidable fragment removes or controls one side of that mechanism.

1.4.6 What is conceptually new in the thesis

Beyond the individual complexity results, the thesis contributes a perspective on timed concurrency built around three axes:

1. whether time is global or local,
2. whether synchronization is absent, selective, or pervasive, and
3. whether relative timing information can persist across interactions.

LTNs inhabit the part of this design space where time is local, synchronization is explicit, and relative timing differences may persist. The results of the thesis then show how restrictions along these axes delineate the boundary between decidability and undecidability.

This perspective is one of the main conceptual contributions of the work. It makes clear that the interesting verification question is not simply "what is the complexity of a new timed model?" but rather "which combinations of concurrency structure and temporal coordination preserve algorithmic tractability?"

1.5 Organization of the thesis

The thesis is divided into two parts.

Part I: introducing the model and establishing the negative boundary. Chapter 2 collects the technical background needed later. It recalls negotiations, timed automata, and networks of timed automata, together with the notation and semantic conventions used throughout the dissertation. The chapter is intentionally foundational: it provides the formal toolkit against which LTNs are introduced.

Chapter 3 then defines local-timed negotiations formally. It presents the local-time semantics, explains the role of local clocks and time-synchronizing nodes, and studies the expressive phenomena that arise from this semantics. The main technical result of the chapter is the undecidability of reachability for general LTNs.

Part II: decidable subclasses. Chapter **??** begins the positive side of the story by studying the two synchronization extremes. It shows that both synchronization-free LTNs and always-synchronizing LTNs have PSPACE-complete reachability, but for fundamentally different reasons.

Chapter 5 moves beyond these extremes by introducing drift-bounded LTNs. It develops the region-style abstraction used for bounded drift, proves decidability and complexity results for that fragment, and analyzes acyclic LTNs as a structurally simpler special case.

Chapter **??** studies connectedly-communicating LTNs. It introduces communication graphs on loops in the marking graph, develops the tightening and propagation arguments needed to control temporal divergence, and proves decidability of reachability for this structurally defined fragment.

Conclusion and outlook. Chapter 7 summarizes the overall picture that emerges from the thesis and discusses broader directions, including quantitative extensions, structural parameters, and possible connections to other infinite-state models.

How to read the thesis. A reader primarily interested in the main results can read this introduction, then Chapter 3 for the model and the undecidability result, and finally Chapters **??**–**??** for the decidable fragments. A reader interested in the technical foundations behind the constructions and comparisons should read Chapter 2 first.

1.6 Related work

The thesis sits at the intersection of several research strands: local-time semantics for real-time systems, timed models of concurrency, negotiation-based models, and structurally restricted distributed systems. This section briefly positions LTNs relative to those lines of work.

1.6.1 Timed automata and local-time semantics

Timed automata [**?**] and their symbolic verification theory form one of the main foundations of the present work. The use of clocks, guards, resets, and region-style reasoning in LTNs is directly inspired by this tradition. Symbolic representations based on zones and difference-bound matrices [**?**] have also strongly influenced how real-time reachability is usually approached, even though the present thesis is more concerned with decidability and complexity than with implementation.

A more specific point of contact is the *local-time semantics* for networks of timed automata [**?**]. In that setting, local progress and partial-order structure become more visible than under the standard global-time semantics. Subsequent work has used this viewpoint for zone-based reachability [**?**, **?**] and for Büchi-style analyses [**?**]. LTNs differ from these approaches in an important way: the local-time viewpoint is not an alternative semantics layered on top of another model, but is built into the very definition of the system. Moreover, synchronization in LTNs is explicit at the level of negotiation nodes, rather than emerging solely from shared actions.

1.6.2 Timed concurrency models beyond timed automata

A broad range of formalisms combine concurrency and quantitative time, including time Petri nets [**?**], timed-arc Petri nets [**?**], and time-constrained message sequence charts [**?**]. These models vary in how they represent synchronization, causality, and the passage of time. Some support strong structural reasoning; others favor symbolic timing analysis. LTNs belong to this general family of timed-concurrency

models, but differ in taking *multiparty interaction* as primitive and in separating local temporal evolution from synchronization.

Partial-order methods for timed systems have also been studied extensively, for instance through unfoldings and related abstractions [**?**, **?**]. These works share a guiding concern with the present thesis: to retain as much independence structure as possible while still reasoning effectively about time. LTNs contribute to this agenda from the angle of interaction-centric modeling.

1.6.3 Negotiations and workflow-style concurrency

Negotiations themselves were introduced as a concurrency model centered on atomic multiparty interactions [**?**]. Their theory has been developed in connection with soundness, summarization, and the analysis of workflow-style behavior, and they have close links to workflow Petri nets and business-process models [**?**, **?**]. This connection is part of what makes negotiations a natural base model for timed extensions: they already support structured distributed coordination with explicit synchronization and visible independence.

To the best of our knowledge, local-timed negotiations as studied in this thesis were first proposed in [**?**]. That work identified two initial decidable fragments corresponding to the synchronization extremes. The present thesis extends and systematizes this line of research by developing the general undecidability result, introducing drift-boundedness as a semantic restriction, and establishing connected communication as a structural route to decidability.

1.6.4 Drift-boundedness and structural restrictions

The notion of bounded drift has previously appeared for networks of timed automata [**?**, **?**], where it is used to justify simulation arguments and finite symbolic exploration under local-time semantics. The idea transfers naturally to LTNs, but the surrounding theory is different: in LTNs, drift must be understood against a marking-based interaction semantics rather than against a product of component automata. One of the contributions of this thesis is precisely to adapt bounded-drift reasoning to this interaction-centric setting.

The connectedly-communicating fragment studied later in the thesis is also related in spirit to earlier decidability results based on communication structure. Hierarchical message sequence charts [**?**] and work on distributed controller synthesis [**?**] show, in different technical settings, that suitable connectivity conditions on communication can restore decidability where unrestricted distributed behavior is too expressive. The present thesis brings a comparable idea into the setting of local-timed negotiations.

1.6.5 Position of the thesis

In summary, LTNs can be viewed as a meeting point of three themes:

1. the clock-based quantitative language of timed automata,
2. the interaction-centric concurrency structure of negotiations, and
3. the local-time and communication-sensitive viewpoints developed in parts of the timed-concurrency literature.

What is new in the thesis is not any one of these ingredients in isolation, but the systematic study of how they interact at the level of reachability. The results show that the resulting model is expressive enough to cross the undecidability boundary, but also structured enough to support several robust decidable fragments. That combination of a sharp negative result with a principled positive landscape is the main contribution of the work.

1.7 What to carry forward

The main points to carry forward from this introduction are the following.

1. **The thesis is about one verification boundary.** It studies how local time and selective synchronization affect reachability.
2. **LTNs are not just timed negotiations with clocks attached.** Their semantics changes the timed verification problem by making time local and synchronization explicit.
3. **The landscape has both a sharp negative result and a structured positive side.** General LTNs are undecidable, but several natural restrictions recover decidability.
4. **The next chapter changes role completely.** This chapter explains why the thesis matters; Chapter 2 supplies the formal toolkit used in the later technical chapters.

1.8 Coming next

Chapter 2 now turns from thesis-level motivation to formal groundwork. It recalls negotiations, timed automata, and networks of timed automata in the precise form needed later. In particular, it develops the clock-based machinery and the region viewpoint that LTNs will inherit and then challenge. After that setup, Chapter 3 introduces local-timed negotiations formally and proves the first major result of the thesis: reachability is undecidable in full generality.

Part I

Adding local time constraints to negotiations

Chapter 2

Preliminaries

Chapter 1 gave an informal overview of this thesis: it identified the verification problem, introduced local-timed negotiations informally, and stated the main decidability landscape. This chapter plays a different role. It fixes the formal background used in the rest of the thesis.

Why this chapter matters

- It introduces the two formal ingredients from which local-timed negotiations are built: *negotiations* and *timed automata*.
- It records the exact semantic notions that later proofs rely on: markings, guards, resets, regions, and reachability.
- It highlights one key contrast that drives the thesis: classical timed automata rely on *global time*, whereas LTNs will use *local time*.

No new results are proved in this chapter. Its purpose is to provide machinery, notation, and a shared reference point for later chapters.

Organization. Section 2.1 recalls negotiations as the control-flow model reused later. Section 2.2 recalls timed automata, now in full formal detail, because clocks, guards, resets, and regions are part of the later machinery. Section 2.3 reviews networks of timed automata only to isolate the global-time assumption from which LTNs depart. Section 2.4 states explicitly what Chapter 3 will keep and what it will change. Section 2.5 provides a compact terminology summary for quick reference in later chapters.

2.1 Negotiations

Negotiations provide the concurrency model underlying this thesis. Their defining feature is that *multiparty interaction* is primitive: an atomic step is not a local

transition of one component, but a joint interaction among all agents participating in a node. This makes both synchronization and independence visible in the model itself.

In the original formulation of Esparza and Desel [**?**], outcomes may additionally update internal states of the participating agents through transformers. For the reachability questions studied in this thesis, those internal states are not essential. We therefore work with a control-flow presentation that records only which nodes each agent is ready to engage in next.

2.1.1 Syntax and operational semantics

Let P be a finite set of agents (or processes), and let Σ be a finite set of outcome labels.

Definition 2.1 (Negotiation)**.** A *negotiation* is a tuple

$$\mathcal{N} = (N, \mathrm{dom}, \mathrm{out}, \delta)$$

where:

- N is a finite set of nodes (also called *atoms*),
- $\mathrm{dom} : N \to 2^P \setminus \{\emptyset\}$ assigns to each node a nonempty set of participating agents, called its *domain*,
- $\mathrm{out} : N \to 2^\Sigma \setminus \{\emptyset\}$ assigns to each node a nonempty finite set of *available outcomes*,
- for each agent $p \in P$, writing $N_p := \{n \in N : p \in \mathrm{dom}(n)\}$, we have a transition function
$$\delta_p : \{(n, a) \in N_p \times \Sigma : a \in \mathrm{out}(n)\} \to 2^{N_p},$$
where $\delta_p(n, a)$ is the set of nodes that agent p becomes ready for after outcome a occurs at node n.

We assume a distinguished initial node $n_{\mathrm{in}} \in N$ with $\mathrm{dom}(n_{\mathrm{in}}) = P$.

Thus, each node carries its own outcome set. The functions δ_p are total on the valid node–outcome pairs for agent p; there is no need to make them partial once $\mathrm{out}(n)$ is recorded explicitly.

A node n with domain $\{p, q, r\}$ should be read as a point where agents p, q, r must interact jointly. When the node occurs, they collectively choose one of the outcomes in $\mathrm{out}(n)$. The control effect of that outcome is described agent-by-agent by the functions δ_p.

Definition 2.2 (Marking)**.** A *marking* is a function

$$X : P \to 2^N$$

with $X(p) \subseteq N_p$ for each $p \in P$. The intended meaning is that $X(p)$ is the set of nodes in which agent p is currently ready to participate.

The *initial marking* X_0 is given by $X_0(p) = \{n_{\text{in}}\}$ for every $p \in P$. We also use the *final marking*

$$X_f(p) = \emptyset \qquad \text{for all } p \in P,$$

which represents termination. A node is enabled exactly when all of its participating agents are ready for it.

Definition 2.3 (Enabled node and occurrence)**.** A node n is *enabled* at marking X if $n \in X(p)$ for every $p \in \text{dom}(n)$. If n is enabled and the agents choose some outcome $a \in \text{out}(n)$, the resulting marking X' is defined by

$$X'(p) = \begin{cases} \delta_p(n, a) & \text{if } p \in \text{dom}(n), \\ X(p) & \text{if } p \notin \text{dom}(n). \end{cases}$$

We write

$$X \xrightarrow{(n,a)} X'$$

and call this a *small step*.

A few points are worth emphasizing.

- Readiness is *replaced*, not accumulated. When an agent participates in an occurring node, its previous readiness set is discarded and replaced by the one prescribed by the chosen outcome.
- Synchronization is explicit. If two agents must move together, it is because they both belong to the domain of a node.
- Independence is also explicit. If two enabled nodes have disjoint domains, they may occur in either order.

Definition 2.4 (Run, deadlock, livelock)**.** A *run* of $\mathcal{N}$ is a finite or infinite sequence

$$X_0 \xrightarrow{(n_1,a_1)} X_1 \xrightarrow{(n_2,a_2)} X_2 \to \cdots .$$

A finite run ending in X_f is also called a *large step*. A marking that enables no node is a *deadlock*, unless it is the final marking X_f. A marking that can reoccur along an infinite run while the final marking remains unreachable is a *livelock*.

Definition 2.5 (Reachability graph)**.** The *reachability graph* of $\mathcal{N}$ has as vertices all markings reachable from X_0. There is an edge

$$X \xrightarrow{(n,a)} X'$$

whenever X' is obtained from X by one occurrence of (n, a).

This marking graph is the basic finite-state object associated with an untimed negotiation. In general it can be exponentially large in the size of the negotiation, and that blow-up is already visible before time is introduced.

2.1.2 Graphical conventions

Negotiations admit a convenient graphical notation.

- Each node is drawn as a thick black bar.
- For every participating agent, a white circle (a *port*) is drawn on the bar.
- Outcome-labelled arrows leaving the ports describe the control flow of each participant after that outcome is chosen.
- If one outcome makes an agent ready for several possible next nodes, the edge branches as a *hyperarc*.
- A marking is visualized by placing a token on the appropriate outgoing arc or branching point for each agent.

This representation is useful because it displays both causality and independence directly. Shared ports indicate enforced synchronization; disjoint ports indicate potential concurrency.

2.1.3 A running example: ATM PIN reset

We use a small ATM protocol as a running example throughout the preliminaries and later chapters. It is simple enough to read quickly, but rich enough to exhibit branching, retries, and multiparty interaction.

Scenario. A customer c wants to reset their ATM PIN through an ATM machine a and a bank b. The protocol proceeds as follows.

1. All three agents first synchronize to start the transaction.
2. The customer and the ATM interact to submit the request.
3. The ATM and the bank interact to forward the details.

4. The bank and the customer interact so that an OTP is sent.
5. The customer re-enters the OTP through the ATM.
6. The ATM and the bank validate the OTP; if the validation fails, the protocol returns to the OTP-entry stage.
7. At a later attempt, the customer may instead give up; this makes the customer and the ATM ready for the completion node, while the bank continues according to its own current readiness.

A fully explicit instance of Definition 2.1. Let

$$P = \{c, a, b\}, \qquad N = \{n_{\text{in}}, n_1, n_2, n_3, n_4, n_5, n_6\},$$

and let the global label alphabet be

$$\Sigma = \{\mathsf{start}, \mathsf{req}, \mathsf{det}, \mathsf{s_otp}, \mathsf{e_otp}, \mathsf{match}, \mathsf{fail}, \mathsf{g_up}, \mathsf{end}\}.$$

The node domains are:

$$\text{dom}(n_{\text{in}}) = \{c, a, b\}, \qquad \text{dom}(n_1) = \{c, a\}, \qquad \text{dom}(n_2) = \{a, b\},$$

$$\text{dom}(n_3) = \{c, b\}, \qquad \text{dom}(n_4) = \{c, a\}, \qquad \text{dom}(n_5) = \{a, b\}, \qquad \text{dom}(n_6) = \{c, a, b\}.$$

The node-wise outcome sets are:

$$\text{out}(n_{\text{in}}) = \{\mathsf{start}\}, \quad \text{out}(n_1) = \{\mathsf{req}\}, \quad \text{out}(n_2) = \{\mathsf{det}\}, \quad \text{out}(n_3) = \{\mathsf{s_otp}\},$$

$$\text{out}(n_4) = \{\mathsf{e_otp}, \mathsf{g_up}\}, \quad \text{out}(n_5) = \{\mathsf{match}, \mathsf{fail}\}, \quad \text{out}(n_6) = \{\mathsf{end}\}.$$

Writing $N_c = \{n_{\text{in}}, n_1, n_3, n_4, n_6\}$, $N_a = \{n_{\text{in}}, n_1, n_2, n_4, n_5, n_6\}$, and $N_b = \{n_{\text{in}}, n_2, n_3, n_5, n_6\}$, the actually defined transition values are:

Customer c.

$$\delta_c(n_{\text{in}}, \mathsf{start}) = \{n_1\}, \qquad \delta_c(n_1, \mathsf{req}) = \{n_3\}, \qquad \delta_c(n_3, \mathsf{s_otp}) = \{n_4\},$$

$$\delta_c(n_4, \mathsf{e_otp}) = \{n_4, n_6\}, \qquad \delta_c(n_4, \mathsf{g_up}) = \{n_6\}, \qquad \delta_c(n_6, \mathsf{end}) = \emptyset.$$

ATM a.

$$\delta_a(n_{\text{in}}, \mathsf{start}) = \{n_1\}, \qquad \delta_a(n_1, \mathsf{req}) = \{n_2\}, \qquad \delta_a(n_2, \mathsf{det}) = \{n_4\},$$

$$\delta_a(n_4, \mathsf{e_otp}) = \{n_5\}, \qquad \delta_a(n_4, \mathsf{g_up}) = \{n_6\}, \qquad \delta_a(n_5, \mathsf{match}) = \{n_6\},$$

$$\delta_a(n_5, \mathsf{fail}) = \{n_4\}, \qquad \delta_a(n_6, \mathsf{end}) = \emptyset.$$

Bank b.

$$\delta_b(n_{\text{in}}, \mathsf{start}) = \{n_2\}, \qquad \delta_b(n_2, \mathsf{det}) = \{n_3\}, \qquad \delta_b(n_3, \mathsf{s_otp}) = \{n_5\},$$

$$\delta_b(n_5, \mathsf{match}) = \{n_6\}, \qquad \delta_b(n_5, \mathsf{fail}) = \{n_5\}, \qquad \delta_b(n_6, \mathsf{end}) = \emptyset.$$

No other values of δ_c, δ_a, or δ_b are in the domain of the corresponding function, because outcomes are only defined at nodes whose outcome sets contain them.

How the control flow works. From the initial marking X_0, outcome start sends c and a to n_1 and sends b to n_2. After the request step at n_1, the ATM moves to n_2, enabling the ATM–bank interaction that forwards the request to the bank. The OTP is then sent at n_3, the customer re-enters it at n_4, and validation occurs at n_5. Outcome fail returns the ATM to n_4 and keeps the bank at n_5, thereby creating the retry pattern. Outcome g_up at n_4 makes the customer and the ATM ready for n_6; whether n_6 then becomes enabled depends on where the bank currently is in the protocol.

Why this example matters later. This example will reappear in Chapter 3 once clocks are added. It is particularly convenient there because the untimed control structure is already familiar, so one can isolate what is genuinely new about the local-time semantics.

Figure 3.1 shows the graphical representation of this negotiation.

2.1.4 A richer example: pollination and climate mismatch

The ATM protocol is useful as a running workflow example, but it is mostly a structured coordination pattern with one main retry loop. For branching, hyperarcs, delayed resolution of choice, and possible blow-up of the marking graph, it is helpful to keep a second, more visibly nondeterministic example in view.

Consider a stylized negotiation inspired by the mismatch between flowering time and pollinator emergence. Let the agents be

$$P = \{t, e, b\},$$

where t stands for *trees*, e for the *environment*, and b for *bees*. The point of the example is not ecological realism, but a compact interaction pattern in which several local choices jointly determine what can happen later.

Informal reading. The trees, the environment, and the bees first evolve separately. Trees may move through a seasonal-change step, the environment may choose between a "good climate" and a "climate change" branch, and bees may move through an emergence step. Later, trees and the environment interact to

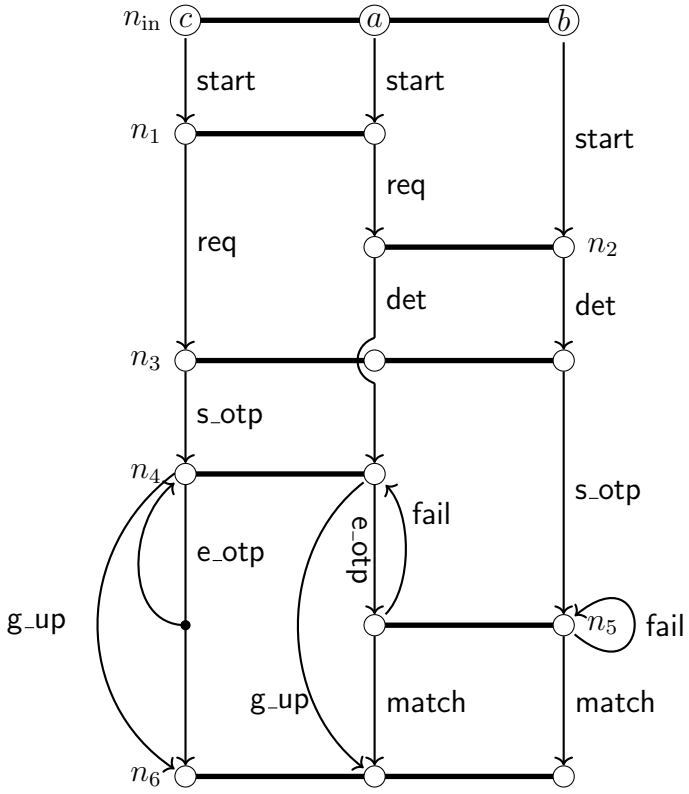


FIGURE 2.1: ATM transaction negotiation. Customer (c), ATM (a), and bank (b) coordinate through atomic interactions. Note the hyperarc at n_4 for customer c after e_otp: non-deterministic control flow allows readiness at both n_4 (retry) and n_6 (completion).

determine whether flowering is normal or early. Finally, the system returns to the beginning of a new cycle. The example is useful because the consequences of earlier choices are not resolved immediately: they constrain which later multiparty interactions can become enabled.

Key observations from this example. This example illustrates several features of negotiations:

1. **Multiparty interactions are atomic:** An interaction involves all its participating agents simultaneously and cannot be decomposed into pairwise steps. Node n_6 requires all three processes to participate simultaneously.

2. **Synchronization is explicit, not emergent:** Agents synchronize because the model says so, not because of shared state.

3. **Independence is visible in the structure:** Disjoint sets of agents can proceed independently, and this is evident in the graph itself. Nodes n_1, n_2 and n_3 can occur in either order, or even conceptually "at the same time" since they involve disjoint sets of processes.

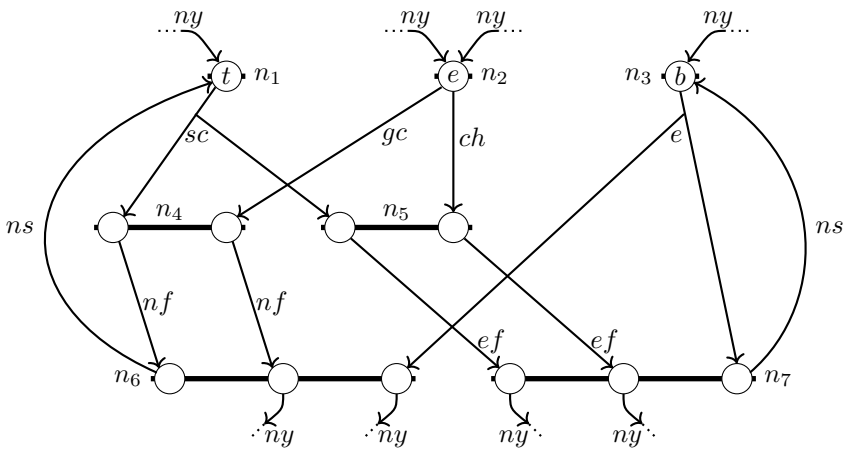


FIGURE 2.2: A stylized negotiation inspired by flowering and pollinator emergence. The example is used structurally, not biologically: it illustrates hyperarcs, branching, delayed consequences of choice, and cyclic control flow.

4. **Non-determinism:** After n_1, t is ready to engage in both n_4 and n_5 and the choice is resolved only through interaction.

5. **Control flow is global, not agent-local:** Choices made at one node propagate to future possibilities for all involved agents.

6. **Information transfer:** The example highlights that synchronization itself acts as a mechanism for information flow between agents – by participating in a joint interaction, agents implicitly exchange information that influences future behavior, even though no explicit shared state or data is modeled.

7. **Succicntness:** The underlying reachability graph could be exponential on the number of nodes.

Figure 2.2 gives the negotiation graph. A sample run and an excerpt of the corresponding reachability graph are deferred to Appendix **??** so that the main text can keep the focus on the formal machinery.

In general, occurrence sequences can be arbitrarily long, and the number of reachable markings can be very large. For negotiations with multiple hyperarcs, each with branching degree n, the number of distinct branching patterns can grow as n^k, where k is the number of hyperarcs, leading to an exponential blow-up in the marking graph. It is known that the reachability problem for negotiations is PSPACE-complete in general, and already coNP-hard and in DP for acyclic negotiations (see, e.g., [**?**]).

In this example, nodes n_1, n_2, and n_3 are single-agent nodes for the trees, the environment, and the bees, respectively. The two flowering nodes n_4 and n_5 involve trees and environment jointly. The outcome chosen at the environment node determines whether normal flowering or early flowering can later occur. Thus

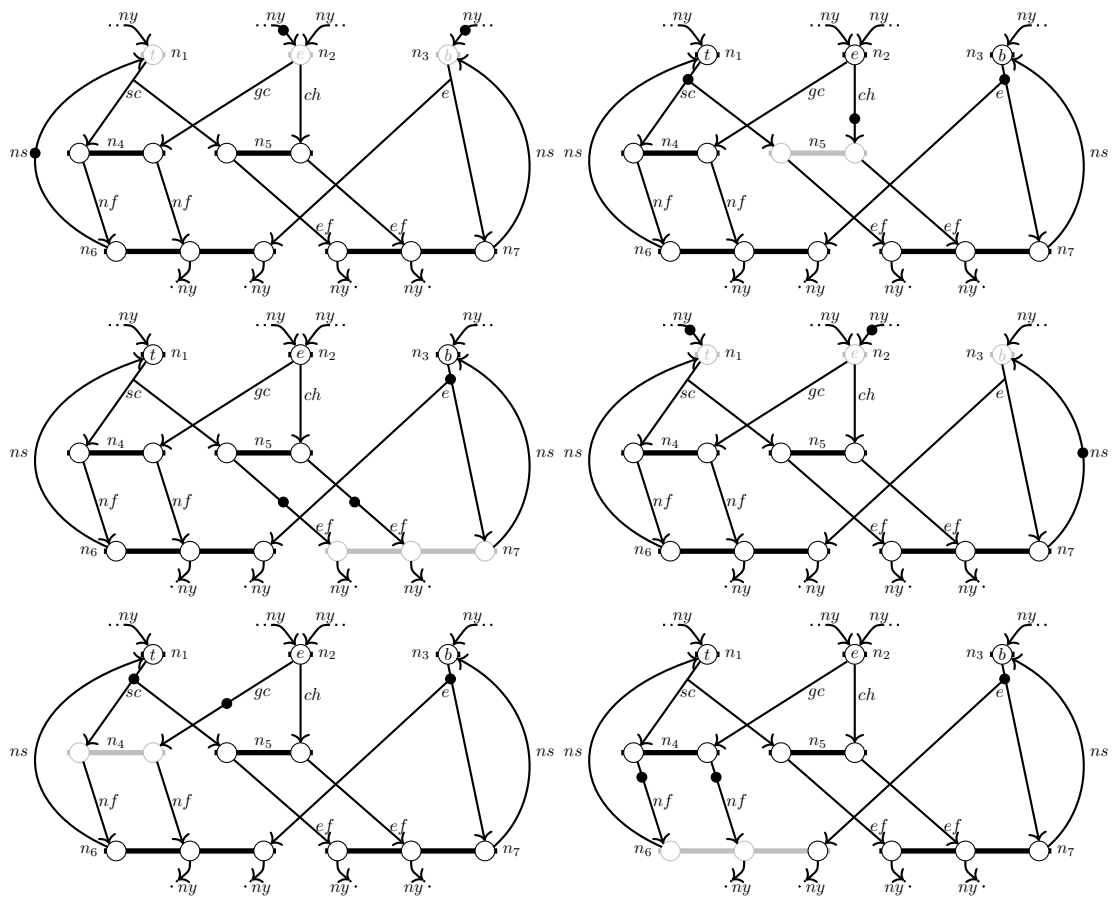


FIGURE 2.3: A possible run of the negotiation in Figure **??**. Tokens represent the current readiness of each process. Gray nodes denote enabled nodes. The run illustrates how choices at n_1, n_2, and n_3 propagate to subsequent flowering and synchronization behaviour.

an early local choice propagates forward and is resolved only when a later joint interaction becomes enabled.

Figure 2.3 shows part of a run of the negotiation in Figure **??**. Gray nodes denote enabled nodes under the current marking.

2.1.5 Relation to Standard Negotiations with Transformers

The negotiation model presented in Section **??** abstracts away internal agent states and transformers. For completeness and to connect with the original literature [**?**], we briefly recall the standard definition and establish equivalence for control-flow properties. For the purposes of this thesis, the internal state evolution carried by transformers plays no role in enabling or disabling interactions.

Standard definition (informal). In the original model, each agent p has an internal state space Q_p with initial states Q_{0p}. An atom n involves parties P_n and has outcomes R_n. Each outcome r is associated with a *transformer* τ_r: a left-total relation $\tau_r \subseteq Q_A \times Q_A$ (where $Q_A = \prod_{p \in P} Q_p$) that is a P_n-transformer—it only affects states of agents in P_n.

When outcome (n, r) occurs, agents in P_n change states according to τ_r, while agents outside P_n remain unchanged. Crucially, *enablement of an atom depends only on readiness*, not on the specific internal states of agents. The choice of outcome is non-deterministic and not restricted by internal states. Transformers determine only how internal states evolve *after* an outcome occurs.

Why the simplification suffices. Since this thesis focuses on reachability and control-flow properties, internal states are orthogonal: they do not influence which atoms are enabled or which outcomes can occur. All information relevant to reachability is captured by tracking which nodes each agent is ready for—exactly what markings record in our simplified definition.

Equivalence. The simplified definition of negotiations used in this thesis is equivalent to the standard definition with internal states and transformers, as far as reachability is concerned. Formally, from a standard negotiation one obtains the simplified model by projecting away internal states and retaining only the next-atom relation, which fully determines enablement and control flow. This simplification is semantic rather than approximate: it preserves the reachability graph exactly. Conversely, any simplified negotiation can be viewed as a standard one by equipping each agent with a trivial one-state internal domain and identity transformers. In both cases, the induced reachability graphs coincide. In both constructions, there is a bijection between reachable markings preserving enabled atoms and outcome occurrences. Occurrence sequences and reachability graphs correspond exactly. Properties expressible in terms of control flow (e.g., reachability, deadlock) are preserved.

2.1.6 Reachability and known complexity

The central decision problem for negotiations is reachability of a marking from the initial marking. This already captures safety questions, deadlock detection, and goal attainment. For unrestricted negotiations, reachability is decidable but can be computationally expensive; in particular, the size of the reachability graph can grow exponentially. Several analysis problems for negotiations are PSPACE-hard in general. For important restricted classes, including acyclic or sound fragments, the complexity may drop substantially [**?**, **?**, **?**].

For the purposes of this thesis, the main point is not the exact untimed complexity landscape, but the structural character of the model: negotiations provide

an explicit control-flow semantics in which multiparty interaction is primitive. That is the concurrency ingredient that LTNs retain.

2.1.7 Relation to standard negotiations and to Petri nets

Relation to Petri nets. A more precise connection with Petri nets is developed by Desel and Esparza [**?**]. They show that every negotiation can be translated into an associated *1-safe labelled Petri net*. The translation is behaviour-preserving in a strong sense: the label reachability graph of the negotiation is isomorphic to the label reachability graph of the associated labelled Petri net, and likewise the concurrent-step reachability graph is preserved. Thus, both sequential behaviour and the concurrency structure of the negotiation are faithfully reflected in the net.

The translation can be sketched as follows. For each agent a and each set of atoms X that may be simultaneously ready for a, one introduces a place $[a, X]$, interpreted as saying that agent a is currently ready to engage in exactly the atoms of X. For each outcome (n, r), one introduces a family of Petri-net transitions labelled by (n, r), one for each possible combination of readiness places that enables the occurrence of n. Such a transition consumes one readiness token for each participating agent and produces the new readiness tokens prescribed by the negotiation's transition function. As a consequence of this construction, the associated net is 1-safe.

This translation also clarifies where the main structural difference lies. In the general case, the associated Petri net may be exponentially larger than the negotiation, because proper hyperarcs may give rise to many combinations of input places. For *deterministic* negotiations, however, the situation is better: the associated Petri net has only linear size, it is a *free-choice* net, and soundness is preserved in the workflow-net setting via place equivalence. Conversely, every sound free-choice workflow net is place equivalent to a net associated with a sound deterministic negotiation. These results explain why negotiations may be viewed as a structurally disciplined interaction-centric subclass of workflow-style Petri-net behaviour.

For this thesis, the main consequence is conceptual. Negotiations are close enough to Petri nets to inherit a robust concurrency-theoretic interpretation, yet explicit enough to let us work directly with participants, outcomes, and readiness. This is exactly the level of abstraction at which we later add local quantitative time.

Negotiations have been applied to the analysis of business processes, often modelled by workflow Petri nets [**?**]. Such nets underlie popular graphical notations like BPMN, EPC, and UML Activity Diagrams [**?**]. Deterministic negotiations are closely related to free-choice workflow nets [**?**], which are expressive enough for many industrial processes; empirical studies show that a large fraction of real-world models fall into the free-choice class.

In the remainder of the thesis, we extend negotiations with quantitative timing information, in the form of local clocks and time constraints, and study reachability in this enriched model.

2.2 Timed Automata

Timed automata [**?**] extend finite automata with real-valued clock variables that evolve continuously with time. They provide the standard framework for modeling and verifying real-time systems, and form the semantic foundation for the timing aspects of local-timed negotiations.

We briefly recall their definition, operational semantics, and the region-based decidability result for reachability, which will serve as a conceptual template for later constructions. This material is standard; we include it to establish notation and to highlight which techniques transfer (or fail to transfer) to local-timed negotiations.

2.2.1 Syntax and Semantics

We work in dense time over the non-negative reals $\mathbb{R}_{\geq 0}$. All clocks evolve synchronously with global time in standard timed automata.

Clock constraints. Let X be a finite set of *clocks*. A *clock valuation* is a function $v : X \to \mathbb{R}_{\geq 0}$. For $\delta \in \mathbb{R}_{\geq 0}$, we write $v + \delta$ for the valuation $(v + \delta)(x) = v(x) + \delta$ (uniform time elapse). For $Y \subseteq X$, we write $[Y]v$ for the valuation that resets clocks in Y to zero: $([Y]v)(x) = 0$ if $x \in Y$, and $([Y]v)(x) = v(x)$ otherwise.

The set of *clock constraints* over X, denoted $\mathcal{C}(X)$, is defined by the grammar:

$$g ::= x \bowtie c \mid g \wedge g$$

where $x \in X$, $c \in \mathbb{N}$, and $\bowtie \in \{<, \leq, =, \geq, >\}$. A valuation v *satisfies* constraint g, written $v \models g$, if g evaluates to true when each clock x is replaced by $v(x)$.

Definition 2.6 (Timed automaton)**.** A timed automaton (TA) is a tuple $\mathcal{A} = (Q, \Sigma, X, T, q_0, \mathrm{Acc})$ where:

- Q is a finite set of *states* (or *locations*),
- Σ is a finite *alphabet* of actions,
- X is a finite set of *clocks*,
- $q_0 \in Q$ is the *initial state*,
- $\mathrm{Acc} \subseteq Q$ is the set of *accepting states*,

- $T \subseteq Q \times \Sigma \times \mathcal{C}(X) \times 2^X \times Q$ is a finite set of *transitions*. A transition $(q, a, g, R, q') \in T$ has source state q, action label a, guard g, reset set R, and target state q'.

Semantics. The behavior of $\mathcal{A}$ is given by an infinite-state transition system whose *configurations* are pairs (q, v) consisting of a state $q \in Q$ and a valuation $v : X \to \mathbb{R}_{\geq 0}$. The *initial configuration* is $(q_0, \mathbf{0})$ where $\mathbf{0}$ denotes the valuation assigning zero to all clocks.

There are two kinds of transitions:

- **Delay transitions:** $(q, v) \xrightarrow{\delta} (q, v+\delta)$ for any $\delta \in \mathbb{R}_{\geq 0}$. Time elapses uniformly for all clocks.
- **Action transitions:** $(q, v) \xrightarrow{a} (q', v')$ if there exists a transition $(q, a, g, R, q') \in T$ such that $v \models g$ and $v' = [R]v$. The guard must be satisfied, and clocks in R are reset.

A *run* is a finite or infinite alternating sequence of delay and action transitions starting from the initial configuration:

$$(q_0, \mathbf{0}) \xrightarrow{\delta_0} (q_0, v_0) \xrightarrow{a_0} (q_1, v_1) \xrightarrow{\delta_1} (q_1, v_1') \xrightarrow{a_1} (q_2, v_2) \xrightarrow{\delta_2} \cdots$$

We often write $(q, v) \xrightarrow{\delta, a} (q', v')$ to denote a delay δ followed immediately by action a. A run is *accepting* if it reaches some configuration (q_f, v) with $q_f \in \mathrm{Acc}$.

A small example. A useful mental picture is the following two-clock automaton. One clock x measures how long the system has waited since the last a-transition, while another clock y measures the total elapsed time since the start. A transition labelled a may only be taken while $x \leq 2$, and taking it resets x. A later transition labelled b becomes available only when $y \geq 10$. This tiny example already exhibits the central tension of timed automata: the system has finitely many control locations but infinitely many possible valuations.

Definition 2.7 (Reachability problem for TA)**.** Given a timed automaton $\mathcal{A}$ and a target state $q_{\text{target}} \in Q$, the reachability problem asks: does there exist an accepting run reaching some configuration (q_{target}, v)?

2.2.2 Region Abstraction and Decidability []

For this thesis, regions serve as the main conceptual tool for proving decidability and establishing complexity bounds, in particular as a reference point when extending negotiations with local time. We therefore briefly recall the region abstraction for timed automata, focusing on the properties needed later.

Since the transition system of a timed automaton is infinite, the standard approach is to construct a finite abstraction by grouping clock valuations that are indistinguishable with respect to future behaviour. Concretely, two valuations are grouped together if they enable the same transitions and allow the same sequences of discrete states to be reached, possibly with different delays. This yields a finite partition of the valuation space into regions, and a corresponding finite region graph obtained by combining regions with control states. For reachability, the exact delays are irrelevant; only the existence of a run following a given sequence of transitions matters.

Let X be a finite set of clocks. Let $M : X \xrightarrow{N} \cup\{-\infty\}$ be a bound function that associates a constant $M_x \in N$ to every clock x.

Definition 2.8 (Region equivalence []). Two valuations $v, v' \in \mathbb{R}_{\geq 0}$ are region equivalent w.r.t. M, denoted $v \sim_M v'$ iff for every $x, y \in X$:

- $v(x) > M_x$ iff $v'(x) > M_x$;
- if $v(x) \leq M_x$, then $\lceil v(x) \rceil = \lceil v'(x) \rceil$;
- if $v(x) \leq M_x$, then $\{v(x)\} = 0$ iff $\{v'(x)\} = 0$;
- if $v(x) \leq M_x$ and $v(y) \leq M_y$ then $\{v(x)\} \leq \{v(y)\}$ iff $\{v'(x)\} \leq \{v'(y)\}$.

Given a timed automaton A, a bound function is obtained by choosing for a clock x, the maximum constant appearing in a guard involving x. Then, the first three conditions in the above definition ensure that the two valuations satisfy the same guards. The last one enforces that for every $\delta \in \mathbb{R}_{\geq 0}$ there is $\delta' \in \mathbb{R}_{\geq 0}$, such that valuations $v + \delta$ and $v' + \delta'$ satisfy the same guards.

Definition 2.9 (Regions []). Let $M : X \to \mathbb{N} \cup \{-\infty\}$ be a bound function. A region is an equivalence class of $\sim_M$. We write $[v]^M$ for the region of v, and $\mathcal{R}_M$ for the set of all regions with respect to M.

Figure **??** shows the division into regions when there are two clocks x and y. We also give below a constructive definition of regions which would be useful to estimate the number of regions.

Definition 2.10 (Region: constructive definition []). A region with respect to bound function M is a set of valuations specified as follows:

- for each clock $x \in X$, one constraint from the set: $\{x = c \mid c = 0, \ldots, M_x\} \cup \{c - 1 < x < c \mid c = 1, \ldots, M_x\} \cup \{x > M_x\}$
- for each pair of clocks x, y having interval constraints: $c - 1 < x < c$ and $d - 1 < y < d$, it is specified if $\{x\}$ is less than, equal to or greater than $\{y\}$.

If r is a region then we will write $r \models g$ to mean that every valuation in r satisfies the guard g. It is straightforward to see that if a valuation $v \in r$ satisfies the guard g, then every valuation $v' \in r$ satisfies g. We will now show the other property with respect to time-elapse mentioned above.

Lemma 2.11. *Let v, v' be valuations such that $v' \sim v$. Then, for all $\delta \in \mathbb{R}_{\geq 0}$ there exists a $\delta' \in \mathbb{R}_{\geq 0}$ such that $v' + \delta' \sim_M v + \delta$.*

Proof. Let $v' \sim v$ and fix $\delta \in \mathbb{R}_{\geq 0}$. We construct δ'. Put $\lceil \delta' \rceil$ to be $\lceil \delta \rceil$. Clearly, we have $v' + \lceil \delta' \rceil \sim_M \lceil \delta \rceil$: that is, valuations $v' + \lceil \delta' \rceil$ and $v + \lceil \delta \rceil$ have the same integral parts and the same ordering of fractional parts (modulo M). Let $x_1 \lessdot_1 x_2 \lessdot_2 \cdots \lessdot_{k-1} x_k$ be the ordering of fractional parts of clocks less than M in both the valuations. Here $\lessdot$ denotes either $<$ or $=$.

From $v + \lceil \delta \rceil$, elapsing a fractional amount $\{\delta\}$ might move some of the clocks up to the next integer. Let $x_j, x_{j+1}, \ldots, x_k$ be the clocks that have their integral values increased from $v + \lceil \delta \rceil$ due to the fractional elapse $\{\delta\}$. Thanks to the denseness of the real line, one can choose $\{\delta'\}$ between the fractional values of clocks x_{j-1} and x_j in $v' + \lceil \delta' \rceil$ so that $v' + \delta'$ has the same integers as $v + \delta$ and the same ordering of fractional parts (modulo M). □

Given a bound function M, the number of regions in $\mathcal{R}_M$ is finite. Once this finite partition of the valuations is obtained, we proceed to define a finite graph built from these regions, that captures the behaviour of the timed automaton. For an automaton A, to define its region graph, we consider a bound function M_A that is obtained from the automaton's definition.

Definition 2.12 (Maximal bounds)**.** Given an automaton A, the maximal bounds function $M_A : X \to \mathbb{N} \cup \{-\infty\}$ associates to each clock x the biggest constant appearing in a guard of the automaton that involves x. If there is no guard involving x, then $M_A(x)$ is assigned $-\infty$.

We define the region graph of an automaton A using the $\sim_{M_A}$ relation. Note that in class, we used the terminology region automaton. As we are not interested in the language accepted by this automaton, and instead we care if there is a path to the accepting state, we will stick to calling this a region graph.

Definition 2.13 (Region graph []**).** Nodes of the region graph denoted by $RG(A)$ are of the form (q, r) for q a state of A and $r \in \mathcal{R}_{M_A}$ a region. There is a transition $(q, r) \xrightarrow{t} (q', r')$ if there are $v \in r, \delta \in \mathbb{R}_{\geq 0}$ and $v' \in r'$ with $(q, v) \xrightarrow{\delta, t} (q', v')$. The initial node of the region graph is $(q_0, [0]_{\sim_{M_A}})$ where $[0]_{\sim_{M_A}}$ represents the region to which the initial valuation 0 belongs to. A node (q, r) is said to be an accepting node if $q \in Acc$.

A key property of region graphs is pre-stability, which ensures that transitions are representative of all valuations in a region.

Lemma 2.14 (Pre-stability of regions)**.** *Let A be a timed automaton and let $RG(A)$ be its region graph. Transitions in $RG(A)$ are pre-stable: for every transition $(q, r) \xrightarrow{t} (q', r')$ in $RG(A)$ and every valuation $v \in r$, there exist $\delta \in \mathbb{R}_{\geq 0}$ and a valuation $v' \in r'$ such that $(q, v) \xrightarrow{\delta, t} (q', v')$ in A.*

Proof. Fix a transition $(q, r) \xrightarrow{t} (q', r')$ in $RG(A)$. By definition of the region graph, there exist $v_0 \in r$, $\delta_0 \in \mathbb{R}_{\geq 0}$ and $v_0' \in r'$ such that $(q, v_0) \xrightarrow{\delta_0, t} (q', v_0')$ is a valid step in A. Let $v \in r$ be arbitrary. By the time-elapse property of region equivalence (Lemma **??** in your text), there exists $\delta \in \mathbb{R}_{\geq 0}$ such that $v + \delta$ lies in the same region as $v_0 + \delta_0$. Since valuations in the same region satisfy the same guards, $v + \delta$ satisfies the guard of t. Let $v' := [R](v + \delta)$ be the valuation obtained by applying the reset set R of transition t. Region equivalence is preserved under resets, hence $v' \in r'$. Therefore $(q, v) \xrightarrow{\delta, t} (q', v')$ is a valid step of A with $v' \in r'$. □

The following lemma is a direct consequence of the pre-stability property.

Lemma 2.15. *Every path in $RG(A)$ is an abstraction of a run of A, and conversely, every run of A is an instantiation of a path in $RG(A)$.*

The above lemma shows that the region graph is sound and complete for state reachability.

Theorem 2.16 ([])**.** *Automaton A has an accepting run iff there is a path in the region graph $RG(A)$ starting from its initial node to an accepting node.*

While this theorem gives an algorithm for solving our problem, it turns out that this method is very impractical. The number of regions obtained using a bound function M is $\mathcal{O}(|X|! \cdot 2^{|X|} \cdot \prod_{x \in X}(2M_x + 2))$ [] and constructing all of them, or even searching through them on-the-fly, has proved to be very costly. Understanding why region abstractions work in the global-time setting is crucial to appreciating the techniques that are required in local-timed negotiations. The crucial assumption underlying regions is the existence of a single global time along which all clocks evolve uniformly.

What to retain from timed automata. Only three ingredients from the timed-automata story are essential for the rest of the thesis:

1. clocks, guards, and resets as a language for quantitative time,

2. reachability as the central decision problem,

3. the region idea as the canonical finite-abstraction technique under global time.

LTNs will keep the first two and challenge the third.

2.3 Networks of timed automata and the local-time contrast

A network of timed automata models a system made of several timed components evolving concurrently and synchronizing on shared actions. This is the closest classical reference point for LTNs, and it is useful precisely because it shows what LTNs do *not* assume.

2.3.1 Global-time semantics for networks

Consider timed automata $\mathcal{A}_1, \ldots, \mathcal{A}_k$ with pairwise disjoint location sets and pairwise disjoint clock sets. A global state of the network consists of one control location per component together with one global valuation over the union of all clocks.

The standard semantics again has two kinds of transitions.

- **Global delay.** A delay δ advances *all* clocks of *all* components by the same amount.
- **Synchronized action.** An action label a may be taken jointly by all components that contain an a-labelled outgoing transition whose guard is satisfied; participating clocks are reset accordingly, while nonparticipating components remain in place.

The reachability problem remains decidable and PSPACE-complete. Conceptually, nothing fundamentally new happens compared with a single timed automaton: the clocks are still governed by one ambient time axis, and the region argument extends to the product system.

2.3.2 Why this is not yet the right model

The standard network semantics captures concurrent control flow, but it still imposes a strong global-time discipline. All components age together. Even when two subsystems are structurally independent, the semantics couples them through the shared passage of time.

That is exactly the assumption LTNs later reject. In LTNs,

- time progresses *locally* rather than globally,
- synchronization of time is *explicit* rather than automatic,
- and relative time differences between agents may persist across interactions.

This contrast is worth keeping in mind: the classical product view says "many automata, one time." LTNs will instead allow "many agents, many local times, selective synchronization."

Local-time semantics as a bridge. There is a line of work on local-time semantics for networks of timed automata [**?**, **?**, **?**, **?**]. Those models already suggest that independence becomes more visible when global time is relaxed. LTNs pursue the same conceptual direction, but in an interaction-centric setting where multiparty negotiation structure rather than automaton product structure is primary.

2.4 From these ingredients to LTNs

At this point, the two ingredients of the thesis are in place.

From negotiations we keep:

- multiparty interaction as primitive,
- markings as the control-state notion,
- explicit visibility of dependence and independence.

From timed automata we keep:

- real-valued clocks,
- guards and resets,
- reachability as the central verification question,
- region-style finite abstractions as the standard benchmark for decidability.

What Chapter 3 will change. The semantic change is simple to state: instead of a single global delay, LTNs will use *local delay vectors*. Each agent will carry its own reference time and its own local clocks. Some negotiation nodes will require the participating reference times to align; others will not. This is exactly the point where the standard timed-automata argument ceases to apply directly. Once time becomes local and synchronization becomes selective, clocks can no longer be quotiented solely by the classical global-time region construction.

Thus Chapter 3 does not merely combine two existing models. It alters the temporal semantics in a way that creates a genuinely new reachability problem.

2.5 Terminology summary

For ease of reference, Table 2.1 collects the main terms that will recur throughout the thesis. Its purpose is practical: after Chapter 1 has introduced the story and Chapter 2 has fixed the machinery, this table should let the reader recover the basic vocabulary quickly without searching backward through definitions.

Term	**Meaning in this thesis**
Agent / process	A participating component of the system. In negotiations and LTNs, all control information is tracked agent by agent.
Node / atom	A basic interaction point in a negotiation. Its domain specifies which agents participate jointly.
Domain $\mathrm{dom}(n)$	The set of agents that must participate in node n.
Outcome set $\mathrm{out}(n)$	The finite set of outcome labels available at node n.
Outcome	One of the possible joint results of an enabled negotiation node.
Marking	The control-state notion for negotiations: for each agent, the set of nodes it is currently ready to engage in.
Final marking X_f	The marking with empty readiness sets for all agents; it represents termination.
Enabled node	A node whose every participating agent is currently ready for it.
Small step	One occurrence of an enabled node with a chosen outcome, producing a new marking.
Run / occurrence sequence	A finite or infinite sequence of small steps.
Deadlock	A nonfinal marking enabling no further node.
Livelock	A recurring nonfinal behavior that never reaches the desired final marking.
Clock valuation	An assignment of nonnegative real values to the clocks of a timed automaton.
Guard	A clock constraint that must hold for a transition or outcome to occur.
Reset	An operation that sets selected clocks to 0 when a transition or outcome is taken.
Region	An equivalence class of valuations that are indistinguishable for future reachability under global time.
Region graph	The finite quotient graph obtained by pairing automaton locations with regions.
Global-time semantics	The standard timed-automata semantics in which every clock advances by the same delay.
Network of timed automata	A collection of timed automata synchronized on shared action labels.
Local-time viewpoint	The idea that components may progress against distinct local times and synchronize only when explicitly required.

Table 2.1: Terminology summary for Chapters 2–6.

2.6 What to carry forward

This chapter provides the machinery that the rest of the thesis relies on. The main points to carry forward are the following.

1. **Negotiations provide the control skeleton.** Their semantics is marking-based, interaction-centric, and explicitly multiparty.

2. **Timed automata provide the timing language.** Clocks, guards, resets, and the reachability problem are inherited directly from that tradition.

3. **The classical region argument depends on global time.** It works because all clocks evolve uniformly. This dependence is not incidental; it is exactly what Chapter 3 will challenge.

4. **Networks of timed automata are still globally timed.** They are a useful comparison model, but they do not yet capture local temporal evolution with selective synchronization.

2.7 Coming next

The next chapter is where the thesis proper begins technically. Chapter 1 supplied the motivation and global picture; this chapter supplied the machinery. Chapter 3 now combines those ingredients into one formal model. Each agent will receive its own local reference time and its own local clocks. Negotiation outcomes will carry guards and resets, and selected nodes will additionally enforce time synchronization among their participants. This one semantic shift—from one global delay to local delay vectors—is the reason the rest of the thesis is needed. It makes the model expressive enough that the standard finite-abstraction story no longer applies automatically. Chapter 3 formalizes this model and proves the first major result of the thesis: in full generality, reachability for LTNs is already undecidable.

Chapter 3

Local-Timed Negotiations

The previous chapters developed two complementary viewpoints on concurrency and time: negotiations make causality and independence explicit through interaction structure, while timed automata capture quantitative timing through clocks and guards. This chapter combines those viewpoints without collapsing distributed behaviour onto a single global timeline. We introduce *local-timed negotiations* (LTNs), in which each agent carries its own local time, time elapses independently for different agents, and equality of local times is imposed only at explicitly chosen interaction points.

The chapter has two goals. The first is modelling: LTNs provide a natural formalism for local deadlines, response windows, and selectively synchronized coordination in distributed systems. The second is algorithmic: the combination of local drift and explicit synchronization already yields a model whose location-reachability problem is undecidable. This boundary result motivates the decidable fragments studied in Part II.

3.1 Motivation: Why Local Time?

The inadequacy of global time. Networks of timed automata (Section **??**) assuming *global time*, i.e., all clocks advance uniformly during delay transitions, and synchronization is instantaneous is inadequate to represents distributed reality for three main reasons:

- **Local clocks:** in realistic distributed protocols, agents progress at different rates due to clock drift, computation time, and network delays.
- **Partial synchrony:** real systems combine tightly synchronized interactions with loosely coupled, independent computation.
- **Agent-centric timing:** timing constraints are often relative to an agent's own actions (e.g. completing a workflow within a fixed time after initiation), which is awkward to express using a single global timeline.

The local-time perspective. Local-timed negotiations adopt a fundamentally different view: each agent p has a *reference clock* t_p that measures its local elapsed time. Between interactions, agents progress independently—one agent may experience 5 time units while another experiences only 2. At interactions (negotiation nodes), agents coordinate by selecting outcomes that respect their local clock constraints (guards).

Crucially, LTNs introduce *explicit synchronization* as a modeling choice: selected nodes may require that participating agents' reference clocks are equal when the interaction occurs. This captures scenarios where coordination demands temporal alignment, such as simultaneous authentication or tightly coupled sensor fusion.

What we gain. This synthesis yields several benefits:

- **Explicit concurrency**: The negotiation structure makes independence and causality transparent. Actions involving disjoint agents can occur concurrently without artificial interleaving.
- **Agent-centric timing**: Each agent tracks its own deadlines, timeouts, and response times via local clocks, more naturally modeling distributed systems than shared global clocks.
- **Selective synchronization**: Synchronizing nodes explicitly capture coordination points where temporal alignment matters, while unsynchronized nodes allow agents to drift.
- **Rich expressiveness**: As we'll see, LTNs can model behaviors impossible in purely global-time or purely asynchronous settings, including causal enforcement through timing and non-regular untimed languages.

What we sacrifice. This expressiveness comes at a cost: *unrestricted LTNs have undecidable reachability.* The combination of unbounded clock drift and explicit synchronization creates computational power equivalent to counter machines. The remainder of this thesis identifies restrictions that recover decidability while preserving modeling advantages.

Chapter roadmap. Section 3.3 motivates the model through a running ATM example. Section 3.4 formalizes LTNs, and Section 3.5 gives their operational semantics. Section 3.2 positions the model with respect to related timed extensions of negotiations and to local-time semantics for timed automata. Section 3.6 illustrates two characteristic expressiveness phenomena, and Section 3.7 proves undecidability of location-reachability. Section 3.8 distils the main message of the chapter and prepares the transition to Part II.

3.2 Related work

Negotiations were introduced by Esparza et al. [?, ?] and admit efficient analysis for several subclasses [?, ?]. Timed negotiations, which associate durations with outcomes of atomic negotiations, were studied in [?]; we provide a detailed comparison with LTNs below.

Local-time semantics for networks of timed automata originates in [?] and has recently been used for zone-based verification [?, ?] and Büchi reachability [?]. In contrast to NTA, where shared actions enforce synchronization by design, LTNs start from a local-time view and offer synchronization as a modeling choice.

3.2.1 Comparison with Timed Negotiations [?]

Timed negotiations [?] extend negotiations by associating durations with outcomes of atomic negotiations. Time progress is modelled by assigning durations to individual interactions, which determine when subsequent negotiations become available.

In contrast, LTNs adopt a clock-based semantics inspired by timed automata, where time elapses independently of interactions and can be constrained across multiple negotiations. This difference has several consequences. First, LTNs can naturally express timing constraints that relate distinct interactions, such as deadlines or response-time constraints spanning an entire workflow. Second, LTNs allow agents to accumulate and compare local time through explicit synchronization nodes, whereas timed negotiations lack a mechanism for comparing temporal progress across interactions.

From an algorithmic perspective, these additional features strictly increase expressive power: as shown in Section 3.7, location-reachability is undecidable for LTNs, whereas timed negotiations retain decidability for basic analysis problems. This contrast highlights the trade-off between expressiveness and tractability when extending negotiations with time.

Viewed together, timed negotiations and LTNs represent complementary points in the design space: the former favor tractable timing of individual interactions, while the latter prioritize expressive, agent-centric temporal reasoning across interactions.

3.3 Introducing Local-Timed Negotiations: An Informal Example

Before formal definitions, we introduce LTNs through an extended version of the ATM transaction example from Chapter 2, now enhanced with timing constraints.

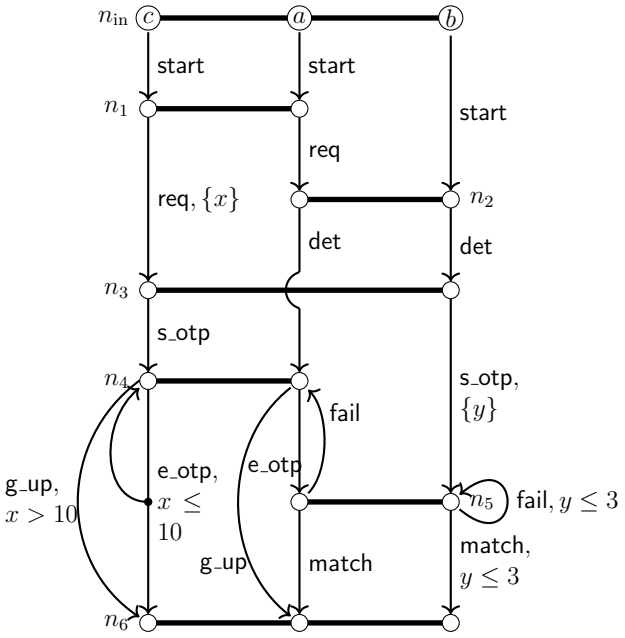


FIGURE 3.1: ATM transaction as a local-timed negotiation. Customer c has local clock x (reset at req, checked at e_otp and g_up). Bank b has local clock y (reset at s_otp, checked at match and fail). Annotations like "$x \leq 10$" are guards; "$\{x\}$" indicates clock reset.

3.3.1 Scenario: ATM Transaction with Timeouts

Recall the basic workflow: a customer (c) resets their ATM PIN using a one-time password (OTP) sent by the bank (b) through the ATM machine (a). We now add realistic timing requirements:

1. **Customer timeout**: The customer wants the entire transaction (from request to completion) to take at most 10 time units. If this deadline is exceeded, the customer gives up.

2. **Bank security policy**: The bank requires that at most 3 time units elapse between sending the OTP and validating it (to prevent replay attacks). Failed validation attempts must also respect this 3-unit window.

3. **Local time progression**: The customer, ATM, and bank may experience different amounts of elapsed time between interactions due to processing delays, network latency, or user hesitation.

3.3.2 Modeling with Local Clocks

Figure 3.1 depicts this scenario as a local-timed negotiation. The basic structure follows the untimed negotiation from Chapter 2, augmented with timing annotations:

Agent local clocks. Each agent maintains local clocks:

- Customer c has clock x and reference clock t_c,
- ATM a has reference clock t_a (no additional local clocks in this example),
- Bank b has clock y and reference clock t_b.

Reference clocks t_c, t_a, t_b measure each agent's total elapsed time. They are never reset and never appear in guards—they serve only to track local time progression. Regular clocks like x and y can be reset and used in guards to express timing constraints.

How timing constraints work. Consider the customer's 10-unit deadline:

1. At outcome req (node n_1), customer c resets clock x to zero. This marks the start of their personal timer.
2. Time elapses locally: c experiences some delay before n_3 (OTP receipt), then more delay before n_4 (OTP entry).
3. At outcome e_otp (from n_4), the guard $x \leq 10$ is checked. This outcome is enabled only if c's local clock x hasn't exceeded 10 units.
4. If $x > 10$, the alternative outcome g_up (give up) becomes the only option, with guard $x > 10$.

Similarly, the bank's 3-unit security window:

1. At outcome s_otp (node n_3), bank b resets clock y to zero, starting the OTP validity timer.
2. Validation at n_5 checks $y \leq 3$ for both match and fail outcomes, ensuring validation attempts occur within the window.

Local time progression. Between interactions, agents' reference clocks can advance by different amounts. For instance:

- Customer c might hesitate for 2 units before entering the OTP (t_c advances by 2),
- Meanwhile, bank b might experience only 0.5 units of processing time (t_b advances by 0.5),
- ATM a might wait 1.8 units for network communication (t_a advances by 1.8).

This reflects the asynchronous reality of distributed systems. Regular clocks like x and y evolve with their agent's reference clock: if t_c advances by 2 units, so does x (unless reset).

Illustrative run. We briefly illustrate the semantics using the ATM example of Figure 3.1. Consider the initial configuration (C_0, v_0), where $C_0(p) = \{n_{in}\}$ for all agents and all clocks are 0.

Suppose the customer c delays by 4 time units, the bank b by 1 unit, and the ATM a by 0 units. This corresponds to the delay vector $\Delta(c) = 4$, $\Delta(b) = 1$, $\Delta(a) = 0$, yielding a valuation v_1 with $v_1(t_c) = 4$, $v_1(t_b) = 1$, and $v_1(t_a) = 0$. All local clocks of an agent advance together with its reference clock.

At this point, the outcome st at the initial node can be executed provided that all guards are satisfied. If the initial node is synchronizing, then in addition we must have $v_1(t_c) = v_1(t_b) = v_1(t_a)$; otherwise the action is blocked, and a different delay vector must be chosen. After executing the action, the marking is updated according to the outcome, and the specified local clocks are reset.

This example illustrates two characteristic features of the semantics: delay choices are made independently by agents, but synchronization may retroactively restrict which delay vectors are admissible, and resets affect only local clocks, never the reference clocks.

Why this matters. The local-time semantics captures realistic timing behavior that global-time models cannot express naturally:

- Each agent enforces its own deadlines independently,
- Timing constraints relate events within an agent's experience (e.g., "10 units after I started"),
- Agents need not progress in lock-step—they coordinate only at interaction points.

3.3.3 Preview: Synchronization

The ATM example uses only local clocks and guards—no explicit synchronization of reference clocks. However, some scenarios require temporal alignment. Imagine the bank wants to ensure the customer and ATM begin validation simultaneously (perhaps for two-factor authentication). We could make node n_5 a *synchronizing node*, requiring $t_a = t_b$ when n_5 occurs.

Sections 3.6.1 and 3.6.2 explore how synchronization enables powerful modeling patterns—and contributes to undecidability.

3.4 Formal Definition

We now formalize local-timed negotiations, building on the control-flow framework from Chapter 2.

Notation. For a finite set S, write $\mathcal{P}(S)$ for its power set (subsets). Let $\mathbb{R}_{\geq 0}$ denote non-negative reals and $\mathbb{N}$ the natural numbers.

Guards. Let X be a finite set of clocks. A *guard* over X is a conjunction of constraints of the form $x \bowtie c$ where $x \in X$, $\bowtie \in \{<, \leq, =, \geq, >\}$, and $c \in \mathbb{N}$. Write $\Phi(X)$ for the set of guards over X. A valuation $v : X \to \mathbb{R}_{\geq 0}$ *satisfies* guard g, written $v \models g$, if all constraints in g evaluate to true under v.

Definition 3.1 (Local-timed negotiation)**.** Let P be a finite set of *agents*, Σ a finite set of *outcomes*, and X a finite set of *local clocks* partitioned as $X = \bigcup_{p \in P} X_p$ where X_p are the local clocks of agent p.

Additionally, each agent p has a distinguished *reference clock* t_p. Reference clocks are never reset and never appear in guards. Let $T := \{t_p \mid p \in P\}$ denote the set of all reference clocks.

A local-timed negotiation (LTN) is a tuple:

$$\mathcal{N} = (N, \mathrm{dom}, \delta, \mathrm{Sync})$$

where:

- N is a finite set of *nodes* (atomic negotiations) with a distinguished *initial node* $n_{\mathrm{in}} \in N$,
- $\mathrm{dom} : N \to \mathcal{P}(P) \setminus \{\emptyset\}$ maps each node to its non-empty set of participating agents (its *domain*), with $\mathrm{dom}(n_{\mathrm{in}}) = P$. For each agent $p \in P$, define $N_p := \{n \in N \mid p \in \mathrm{dom}(n)\}$ as the set of nodes involving p.
- $\delta = \{\delta_p\}_{p \in P}$ is a family of local transition functions, where:

  $$\delta_p : N_p \times \Sigma \to \Phi(X_p) \times \mathcal{P}(N_p) \times \mathcal{P}(X_p).$$

  For a location (n, a) with $p \in \mathrm{dom}(n)$, the value $\delta_p(n, a) = (g_p, M_p, Y_p)$ specifies:

  - $g_p \in \Phi(X_p)$: a *guard* over p's local clocks,
  - $M_p \subseteq N_p$: the set of nodes p becomes ready for after outcome a,
  - $Y_p \subseteq X_p$: the set of p's local clocks to reset when outcome a occurs.

  We call pairs (n, a) *locations*.
- $\mathrm{Sync} \subseteq N$ is the set of *synchronizing nodes*. At synchronizing nodes, participating agents' reference clocks must be equal.

Intuition recap. An LTN extends untimed negotiations (Definition **??**) by adding:

1. **Local clocks** X_p for each agent to track time within their local frame,
2. **Reference clocks** t_p to measure total local elapsed time (implicit, never manipulated directly),
3. **Guards** constraining when outcomes can occur based on local clock values,
4. **Resets** allowing clocks to be zeroed at interaction points,
5. **Synchronizing nodes** where temporal alignment (equality of reference clocks) is required.

3.5 Semantics

The semantics of an LTN describes how the system evolves through delays (time passage) and actions (outcome occurrences). Configurations track both control state (markings) and timing (valuations).

3.5.1 Markings, Valuations, and Configurations

Definition 3.2 (Marking)**.** A marking is a function $C : P \to \mathcal{P}(N)$ where $C(p) \subseteq N_p$ for each agent p. The set $C(p)$ records which nodes agent p is currently ready to participate in.

The initial marking C_0 has $C_0(p) = \{n_{\text{in}}\}$ for all $p \in P$.

Definition 3.3 (Valuation and well-formedness)**.** A valuation is a function $v : X \cup T \to \mathbb{R}_{\geq 0}$ assigning real values to all local clocks and reference clocks. A valuation v is well-formed if for every agent $p \in P$ and every local clock $x \in X_p$:

$$v(x) \leq v(t_p).$$

The intuition for well-formedness is that, since local clocks $x \in X_p$ evolve with reference clock t_p and can only be reset to zero (never increased independently), we always have $v(x) \leq v(t_p)$. This invariant ensures clocks don't "run ahead" of local time.

Definition 3.4 (Configuration)**.** A configuration is a pair (C, v) where C is a marking and v is a well-formed valuation. The initial configuration is (C_0, v_0) where $C_0(p) = \{n_{\text{in}}\}$ for all p and v_0 maps all clocks in $X \cup T$ to zero.

3.5.2 Transitions: Delays and Actions

Local delays. A *local delay vector* is a function $\Delta : P \to \mathbb{R}_{\geq 0}$ assigning each agent a non-negative delay. Given valuation v and local delay Δ, define $v + \Delta$ by:

$$(v + \Delta)(z) = v(z) + \Delta(p) \quad \text{for all } p \in P \text{ and all } z \in X_p \cup \{t_p\}.$$

Thus, all clocks belonging to agent p (both local clocks in X_p and reference clock t_p) advance together by $\Delta(p)$. Different agents may experience different delays.

Clock resets. For a set $Y \subseteq X$ of local clocks, write $v[Y \mapsto 0]$ for the valuation that resets clocks in Y to zero and leaves all other clocks unchanged:

$$v[Y \mapsto 0](z) = \begin{cases} 0 & \text{if } z \in Y, \\ v(z) & \text{otherwise.} \end{cases}$$

Delay steps.

Definition 3.5 (Delay step)**.** A delay step has the form:

$$(C, v) \xrightarrow{\Delta} (C, v + \Delta)$$

where Δ is a local delay vector and $v+\Delta$ is well-formed. The marking is unchanged; only clock values increase.

Action steps.

Definition 3.6 (Action step)**.** Let (n, a) be a location. We write $(C, v) \xrightarrow{(n,a)} (C', v')$ if the following conditions hold:

1. **Enablement:** $n \in C(p)$ for all $p \in \mathrm{dom}(n)$.
   (All participating agents are ready at n.)

2. **Synchronization (if required):** If $n \in \mathrm{Sync}$, then $v(t_p) = v(t_q)$ for all $p, q \in \mathrm{dom}(n)$.
   (At synchronizing nodes, reference clocks must be equal.)

3. **Local guards:** For each $p \in \mathrm{dom}(n)$, let $\delta_p(n, a) = (g_p, M_p, Y_p)$. Then $v \models g_p$ for all $p \in \mathrm{dom}(n)$.
   (All local guards are satisfied.)

4. **Marking update:** Define C' by:

$$C'(p) = \begin{cases} M_p & \text{if } p \in \mathrm{dom}(n) \text{ and } \delta_p(n, a) = (g_p, M_p, Y_p), \\ C(p) & \text{if } p \notin \mathrm{dom}(n). \end{cases}$$

(Participating agents update readiness; others remain unchanged.)

5. **Clock resets:** Let $Y = \bigcup_{p \in \mathrm{dom}(n)} Y_p$ where $\delta_p(n, a) = (g_p, M_p, Y_p)$ for $p \in \mathrm{dom}(n)$. Then $v' = v[Y \mapsto 0]$.
(All clocks in reset sets are zeroed; reference clocks never reset.)

Small steps and runs.

Definition 3.7 (Small step)**.** A small step combines a delay followed immediately by an action:

$$(C, v) \xrightarrow{\Delta} (C, v + \Delta) \xrightarrow{(n,a)} (C', v').$$

Conceptually, agents accumulate delays until the desired location becomes enabled and all guards are satisfied.

Definition 3.8 (Run)**.** A run is a finite or infinite sequence of small steps starting from the initial configuration (C_0, v_0):

$$(C_0, v_0) \xrightarrow{\Delta_0} (C_0, v_0 + \Delta_0) \xrightarrow{(n_0,a_0)} (C_1, v_1) \xrightarrow{\Delta_1} \cdots$$

3.5.3 Location Reachability

Definition 3.9 (Location reachability)**.** A location (n, a) is reachable if there exists a run that executes it. The location-reachability problem asks: given an LTN $\mathcal{N}$ and a location (n, a), does there exist a run executing (n, a)?

This is the central decision problem studied in this thesis. As we'll see, unrestricted LTNs have undecidable location reachability.

Untimed projection. The *untimed outcome language* of $\mathcal{N}$ is the set of words $w = a_0 a_1 \cdots a_k \in \Sigma^*$ for which there exists a run whose executed locations are $(n_0, a_0), (n_1, a_1), \ldots, (n_k, a_k)$. This abstracts away delays, retaining only the sequence of outcomes.

3.5.4 Design choices and modelling rationale

Local-timed negotiations combine ideas from negotiations and timed automata, but the resulting model reflects a number of deliberate design choices. We briefly discuss the most important ones and contrast them with plausible alternatives.

Local clocks and delay vectors. Time elapse in LTNs is described by delay vectors $\Delta \in \mathbb{R}^P_{\geq 0}$, rather than by a single global delay. This reflects the intended local-time semantics: each agent progresses according to its own clock, and different agents may accumulate time at different rates. Using a single delay, as in timed automata, would collapse the model back to a global-time view and eliminate the possibility of drift between agents.

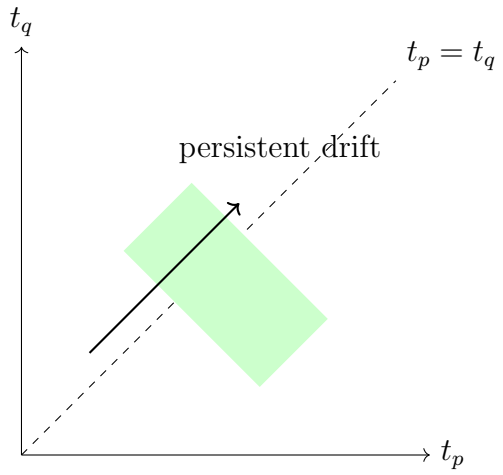


FIGURE 3.2: Independent local time evolution allows relative timing differences between agents to persist across interactions.

Reference clocks. Each agent p is equipped with a reference clock t_p that is never reset and never appears in guards. Reference clocks serve as a semantic device for tracking the accumulated local time of an agent across the execution. By excluding reference clocks from guards, we ensure that agents cannot directly observe or test global time; instead, comparisons between reference clocks are possible only indirectly, via explicit synchronization.

Synchronization as a constraint. In LTNs, synchronization is expressed as the equality constraint $t_p = t_q$ at synchronizing nodes, rather than as an instantaneous time jump or reset. Equality must be achieved by choosing appropriate local delays before the action. This makes synchronization an explicit modelling choice and separates "time passing" from "time comparison" in the semantics.

Node-based synchronization. Synchronization is associated with nodes (atomic negotiations) rather than with outcomes. This aligns with the interaction-centric nature of negotiations: timing constraints are imposed at the level of joint interactions, not at the level of local control-flow choices.

Local guards and resets. Guards and resets are local to each participating agent. This prevents implicit global coordination through shared clocks and ensures that all cross-agent timing dependencies arise either from the negotiation structure itself or from explicit synchronization nodes.

3.6 Expressiveness

Here we understand the model better via two more examples.

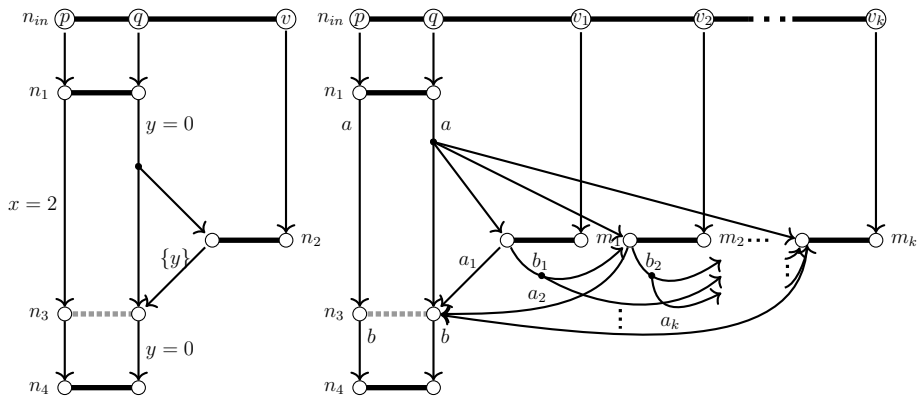


FIGURE 3.3: Vendor enforcement example. Nodes n_1 and n_3 are synchronizing (shown with dotted style). At n_1, outcome a has guards $x = 2$ for p and $y = 0$ for q. At n_3, outcome b requires $y = 0$ for q. Synchronization forces q to visit vendor v at n_2 to accumulate the needed 2 time units. In the figure on the right, the a transition has guards $x = m$ for p and $y = 0$ for q. Each b_i transition has a guard $y = 1$ and a reset of y to ensure exactly 1 unit of time is spent in the nodes m_i before outcomes b_i. The outcomes a_i have a reset of y. The b transition from n_3 has a guard $y = 0$.

3.6.1 Causal Enforcement Through Synchronization

In global-time models, causality arises from control flow and shared clocks. LTNs add a new mechanism: *using synchronization to force intermediate interactions.* An agent can be compelled to perform actions (accumulate local time) to align with a partner's reference clock.

Example 3.9.1 (Forcing intermediate interactions via timing). Consider three agents: p, q, and a vendor v. Agent p wants to ensure that q has visited the vendor between their two meetings. Figure 3.3 shows how this can be enforced using local clocks and synchronization.

Setup. Agents p and q have local clocks x and y respectively. Nodes n_1 and n_3 are synchronizing. At n_1, outcome a has guard $x = 2$ for p and $y = 0$ for q.

What happens. After outcome a at n_1:

- Agent p has advanced its reference clock by exactly 2 units (enforced by guard $x = 2$), so $t_p = 2$.
- Agent q has not advanced at all (guard $y = 0$), so $t_q = 0$.

Now both agents are ready at n_3 (control-flow wise), and n_3 is synchronizing. For outcome b to occur at n_3:

- Synchronization requires $t_p = t_q$, but currently $t_p = 2$ and $t_q = 0$.
- The guard at b requires $y = 0$ for agent q. Since y evolves with t_q, letting time elapse for q would make $y > 0$, violating the guard.

The only solution. Agent q cannot simply wait—doing so would violate the guard $y = 0$. Instead, q must:

1. Visit node n_2 (interaction with vendor v),
2. Spend exactly 2 time units there (advancing t_q by 2),
3. Reset clock y to zero when leaving n_2,
4. Return to n_3 with $t_q = 2$ and $y = 0$, satisfying both synchronization and the guard.

This demonstrates *causal enforcement*: the structure of timing constraints and synchronization forces q to take a specific path through the negotiation. Agent p can compel q to perform intermediate interactions by setting up a timing mismatch that can only be resolved by those interactions.

Generalization. This pattern extends to forcing q to visit m out of k vendors, each requiring 1 time unit. By setting p delay to m at n_1 and requiring q to accumulate exactly m time units before resynchronizing at n_3, we force q to interact with at least m vendors. This shows that LTNs can encode counting and comparison—foreshadowing undecidability.

Contrast with global time. In networks of timed automata with global time, all clocks advance uniformly. One cannot create a situation where agent p is "ahead" of agent q in a way that forces q to perform compensating actions. The local-time semantics is essential for this expressiveness.

3.6.2 Non-regular untimed languages

A classical fact about timed automata is that their untimed languages are regular. LTNs need not share this limitation. The reason is that local drift can encode a counter-like quantity, while synchronization can compare the values accumulated by different agents.

Example 3.9.2 (Generating $\{a^n b^n c \mid n \geq 0\}$). Consider the LTN in Figure 5.2 with two agents p and q. Agent p performs action a repeatedly at node n_1, and agent q performs action b repeatedly at node n_2. Both can then synchronize at node n_3 to perform action c.

Mechanism. Both p and q start with reference clocks at zero. Each iteration of a at n_1 takes exactly 1 time unit (guard $x = 1$, then reset x), advancing t_p by 1. Similarly, each b at n_2 advances t_q by 1. The loops are concurrent—there's no ordering between as and bs.

Node n_3 is synchronizing and requires:

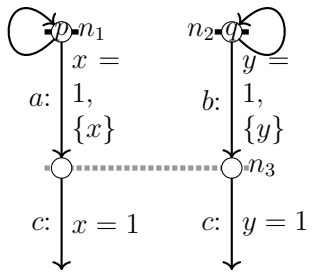


FIGURE 3.4: Non-regular untimed language. Each loop iteration of a and b takes exactly 1 time unit. Node n_3 is synchronizing with guards $x = 1$ and $y = 1$. This forces the number of as to equal the number of bs before c occurs.

- $t_p = t_q$ (synchronization condition),
- $x = 1$ and $y = 1$ (guards at outcome c).

For these conditions to hold simultaneously:

- Both agents must have performed their last loop iteration exactly 1 time unit ago,
- Both agents must have performed the same number of loop iterations (so $t_p = t_q$).

Therefore, if p performs n iterations of a and q performs m iterations of b, action c can occur if and only if $n = m$. The untimed language includes all words of the form $w \cdot c$ where w has equal number of a and b. Note that we can add another identical process with identical guards and which also time-synchronizes at n_3, and then the language would be beyond context-free.

Proposition 3.10. *Let L be the untimed outcome language of the gadget in Figure 5.2. Then*

$$L \cap a^*b^*c = \{a^n b^n c \mid n \geq 1\}.$$

In particular, L is not regular.

Proof. Consider a run whose untimed word belongs to a^*b^*c. As long as the run stays in the a-phase at n_1, only p advances, and it does so by exactly one time unit per a. Once a b is taken, p moves to n_3 and can never execute a again, so the word has the form $a^n b^m c$ with $n, m \geq 1$.

At node n_3, outcome c is synchronizing, so it requires $t_p = t_q$. At that moment, p has accumulated exactly n time units, one for each a, and q has accumulated exactly m time units, one for each b. Therefore c is enabled only if $n = m$.

Conversely, for every $n \geq 1$, the run can execute a exactly n times at n_1, then take the first b from n_1, then execute the b-loop at n_2 another $n - 1$ times, and finally execute c at the synchronizing node n_3. Hence every word $a^n b^n c$ with $n \geq 1$

Feature	**Modelling / expressive effect**
Local delay vectors	Independent progress and persistent drift
Reference clocks	Long-range timing information for each agent
Synchronizing nodes	Equality tests between local times
Local guards and resets	Counter-style timing patterns within a participant

TABLE 3.1: Where the extra expressive power of LTNs comes from.

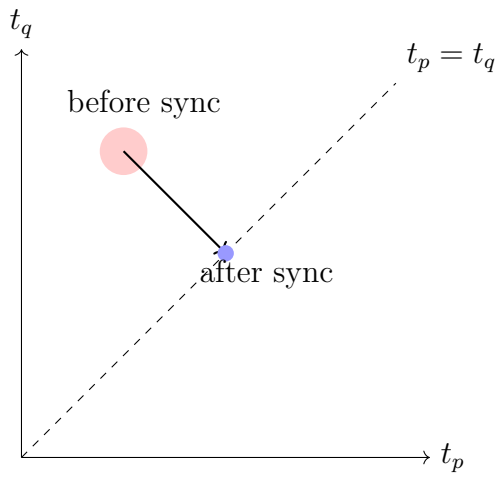


FIGURE 3.5: A time-synchronizing interaction collapses relative timing differences, allowing accumulated information to be compared or reset.

belongs to $L \cap a^*b^*c$. The displayed equality follows. Since a^*b^*c is regular and $\{a^nb^nc \mid n \geq 1\}$ is not, L itself cannot be regular. □

If one prefers the index set $n \geq 0$, it suffices to add a direct c-transition from the initial node. The essential point is unchanged: synchronization compares quantities that were accumulated independently in local time.

Why this matters. This demonstrates that:

1. LTNs can encode *counting* constraints across concurrent actions,
2. Synchronization acts as a *comparison* operator between accumulated counts (via reference clocks),
3. The untimed behavior transcends finite-state structure—language-theoretically, LTNs are more powerful than timed automata.

This expressiveness enables rich modelling but also implies undecidability, as we'll see next.

Counter machine object	LTN encoding
Instruction label ℓ	Control node n_ℓ
Counter value $C_i = c_i$	Drift $v(t_{p_i}) - v(t_{q_i}) = c_i$
Increment of C_i	Let p_i alone elapse one unit of local time
Decrement / zero-test of C_i	Compare q_i with a frozen witness agent r_i
Machine step	One *big step* between encoding configurations

TABLE 3.2: How the reduction encodes a two-counter machine inside an LTN.

3.7 Undecidability of reachability

Proof roadmap. The undecidability proof proceeds by a reduction from the halting problem of 2-counter machines. The key idea is to encode counter values as differences between reference clocks of carefully chosen agents.

Increment and decrement operations are implemented by allowing certain agents to accumulate local time while others are forced to remain idle. Zero-tests rely on synchronizing nodes, which require equality of reference clocks and thus permit the comparison of accumulated drift. Unsynchronized nodes are essential for allowing drift to grow without bound, while synchronized nodes are essential for testing and transferring this information.

The construction alternates systematically between synchronized and unsynchronized phases. This interplay is what enables the simulation of unbounded counters and ultimately yields undecidability of location-reachability.

We now show that location-reachability for LTNs is undecidable. The proof reduces the halting problem of 2-counter machines to reachability in an LTN. The key source of power is the unbounded drift between reference clocks of different agents, together with the ability to impose synchronization at selected nodes.

Theorem 3.11. *Location-reachability is undecidable for local-timed negotiations.*

We prove the theorem by reduction from the halting problem for two-counter machines. Before giving the gadgets, it is useful to keep the following dictionary in mind.

3.7.1 Two-counter machines

A *two-counter machine* M consists of labelled instructions $\ell : I$, where I is one of the following forms for some $i \in \{1, 2\}$:

increment $C_i{+}{+}$, which increments C_i and proceeds to instruction $\ell + 1$;

decrement if $C_i > 0$ then $C_i{-}{-}$, which decrements C_i and proceeds to $\ell + 1$, and is otherwise blocked;

jump-on-zero if $C_i = 0$ goto ℓ', which jumps to ℓ' when $C_i = 0$ and otherwise proceeds to $\ell + 1$.

A configuration of M is a triple (ℓ, c_1, c_2), where ℓ is the current instruction label and $c_1, c_2 \in \mathbb{N}$ are the current counter values. The machine halts if it reaches its designated final instruction.

3.7.2 Overview of the reduction

Given a two-counter machine M, we construct an LTN $\mathcal{N}_M$ with six agents

$$p_1, q_1, r_1, p_2, q_2, r_2.$$

For each $i \in \{1, 2\}$, the agents p_i, q_i, r_i simulate counter C_i. Their local clocks are

$$X_{p_i} = \{x_{p_i}\}, \qquad X_{q_i} = \{x_{q_i}, x'_{q_i}\}, \qquad X_{r_i} = \{x_{r_i}\}.$$

For every machine instruction label ℓ, the negotiation contains a node n_ℓ in which all six agents participate.

A configuration (C, v) of $\mathcal{N}_M$ *encodes* (ℓ, c_1, c_2) if the following conditions hold:

1. $C(\alpha) = \{n_\ell\}$ for every agent α;
2. all local clocks are 0;
3. for each $i \in \{1, 2\}$,
$$v(t_{q_i}) \leq v(t_{r_i}) \leq v(t_{p_i});$$
4. for each $i \in \{1, 2\}$,
$$v(t_{p_i}) - v(t_{q_i}) = c_i.$$

The initial configuration of $\mathcal{N}_M$ encodes the initial machine configuration: all agents are ready at the node for the first instruction, and all local and reference clocks are 0.

A run segment that starts in one encoding configuration, ends in the next encoding configuration, and has no intermediate configuration encoding a machine state will be called a *big step*. The reduction is designed so that big steps simulate exactly machine steps:

- if $(\ell, c_1, c_2) \to (\ell', c'_1, c'_2)$ is a machine transition, then every configuration encoding (ℓ, c_1, c_2) has a big step to some configuration encoding (ℓ', c'_1, c'_2);
- conversely, every big step between encoding configurations corresponds to a valid machine transition.

The proof then concludes by designating a location that is reachable in $\mathcal{N}_M$ iff the machine reaches its halting instruction.

Throughout the gadgets, guards of the form $x = 0$ serve one simple but crucial purpose: they prevent an agent from letting time pass unless the gadget explicitly

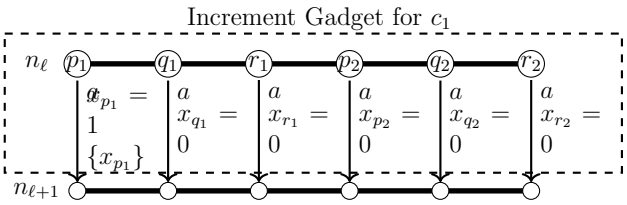


FIGURE 3.6: Gadget for implementing increment instruction on c_1

allows it. Any unintended positive delay would violate such a guard and block the run.

3.7.3 Instruction gadgets

Increment. Assume an instruction $\ell : C_1{+}{+}$ with ℓ not being the final instruction. The case with $C_2{+}{+}$ is symmetric. Every configuration (ℓ, c_1, c_2) on executing this instruction goes to $(\ell+1, c_1+1, c_2)$. Figure 3.6 shows the gadget for the increment instruction. All agents other than p_1 cannot elapse time due to the guard checking local clocks to 0. Agent p_1 elapses exactly one time unit, after which clock x_{p_1} is reset. It is easy to see that the configuration (C', v') that results from (C, v) encodes $(\ell', c_1 + 1, c_2)$. The big step $(C, v) \Rightarrow (C', v')$ is in fact a single transition. Both the properties are easily seen to be satisfied.

Lemma 3.12. *Assume that instruction ℓ of the counter machine is $C_1{+}{+}$. Every successful big step of the gadget in Figure 3.6 from a configuration encoding (ℓ, c_1, c_2) ends in a configuration encoding $(\ell + 1, c_1 + 1, c_2)$. Conversely, every configuration encoding (ℓ, c_1, c_2) has such a big step.*

Proof. Let (C, v) encode (ℓ, c_1, c_2), and write

$$A := v(t_{p_1}), \qquad B := v(t_{q_1}),$$

so that $A - B = c_1$. Since every local clock is 0 in an encoding configuration, the zero guards on the edges for q_1, r_1, p_2, q_2, r_2 force those five agents to take the unique outgoing action with delay 0. The guard $x_{p_1} = 1$ on the edge of p_1 forces p_1 to wait for exactly one time unit, after which x_{p_1} is reset. Thus the gadget has a single possible big step.

At the end of that step, all local clocks are again 0. The reference clocks satisfy

$$t'_{p_1} = A + 1, \qquad t'_{q_1} = B,$$

while $t_{r_1}, t_{p_2}, t_{q_2}, t_{r_2}$ are unchanged. Hence

$$t'_{p_1} - t'_{q_1} = (A + 1) - B = c_1 + 1,$$

and the ordering constraints $t_{r_i} \leq t_{q_i} \leq t_{p_i}$ remain true for both counters. Therefore the target configuration encodes $(\ell + 1, c_1 + 1, c_2)$. The converse direction is the same computation read forwards: from any encoding configuration, the forced delay pattern described above yields the required big step. □

Decrement. Now consider an instruction of the form "if $C_1 > 0$ then $C_1{-}{-}$". Suppose (C, v) encodes (ℓ, c_1, c_2). The counter value c_1 is stored as the drift

$$v(t_{p_1}) - v(t_{q_1}) = c_1.$$

To test whether this difference is positive without destroying the encoding, the gadget uses the auxiliary agent r_1 as a frozen witness.

The gadget works in three phases.

Phase 1: store the current reference time of p_1. Agents p_1 and r_1 pass through a synchronizing node while p_1 is prevented from letting time elapse. After this phase,

$$t_{r_1} = t_{p_1},$$

while q_1 remains unchanged. The current value of the counter is therefore remembered indirectly as the gap between q_1 and the witness r_1.

Phase 2: replay the difference. At an unsynchronized loop, agents p_1 and q_1 each advance by exactly one unit per iteration, while r_1 is forced to stay put. This preserves the drift $t_{p_1} - t_{q_1}$, but moves q_1 steadily toward r_1.

Phase 3: test equality with the witness. The gadget now attempts to execute a synchronizing action between q_1 and r_1. This is possible exactly when q_1 has advanced by

$$v(t_{p_1}) - v(t_{q_1}) = c_1$$

units in Phase 2. If the loop was executed too few or too many times, the final synchronization is impossible and the run blocks.

At this point the auxiliary clock x'_{q_1} records whether a positive amount of time elapsed during Phase 2. If the value is positive, the decrement branch lets q_1 advance by one additional unit, thereby reducing the stored drift from c_1 to $c_1 - 1$ and moving control to $n_{\ell+1}$. If no time elapsed in Phase 2, then the original counter value was 0, and the gadget moves into a blocking branch instead.

Lemma 3.13. *The decrement gadget for C_1 has a successful big step from a configuration encoding (ℓ, c_1, c_2) iff $c_1 > 0$. In that case the target configuration encodes $(\ell + 1, c_1 - 1, c_2)$.*

Proof. Let (C, v) encode (ℓ, c_1, c_2), and abbreviate

$$A := v(t_{p_1}), \qquad B := v(t_{q_1}), \qquad R := v(t_{r_1}),$$

so that $A - B = c_1$ and $R \leq B \leq A$. We reason only about the agents p_1, q_1, r_1 shown in Figure 3.7; the other three agents are wired in parallel through zero-guard edges to the gadget for $\ell + 1$, so they never let time pass and preserve the value of C_2.

Phase 0: the forced start action. Because all local clocks are 0 in an encoding configuration and the outgoing edges of n_ℓ are guarded by $x_{p_1} = 0$, $x_{q_1} = 0$, $x'_{q_1} = 0$, and $x_{r_1} = 0$, the outcome st must be taken with delay 0. After this action, p_1 and r_1 are at node n_1, while q_1 is at node n_2, and all clock values are unchanged.

Phase 1: store the current time of p_1 in the witness r_1. The next relevant step is the synchronizing action at node n_1. The guard $x_{p_1} = 0$ forbids any positive delay for p_1, so the only way to fire this action is to let r_1 delay by exactly $A - R$ and keep p_1 fixed. After the action, p_1 moves to n_2 and r_1 to n_3, with

$$t_{p_1} = A, \qquad t_{q_1} = B, \qquad t_{r_1} = A,$$

and with

$$x_{p_1} = x_{q_1} = x'_{q_1} = x_{r_1} = 0.$$

Thus the current reference time of p_1 is now frozen in the witness r_1.

Phase 2: replay the stored drift. At the unsynchronizing node n_2, the only progress-making action is the loop b. Its guards require $x_{p_1} = 1$ and $x_{q_1} = 1$, and the action resets x_{p_1} and x_{q_1} but not x'_{q_1}. Therefore, after exactly k iterations of b,

$$t_{p_1} = A + k, \qquad t_{q_1} = B + k, \qquad t_{r_1} = A,$$

and

$$x_{p_1} = x_{q_1} = 0, \qquad x'_{q_1} = k, \qquad x_{r_1} = 0.$$

In particular, the drift $t_{p_1} - t_{q_1}$ is still $A - B = c_1$, while x'_{q_1} remembers how much local time has elapsed for q_1 since the start of the gadget.

Phase 3: test equality with the witness. The action a out of n_2 has guards $x_{p_1} = 0$ and $x_{q_1} = 0$, so it must be taken with zero additional delay and moves p_1 to n_4 and q_1 to n_3. The following action c is taken at the synchronizing node n_3 and has guards $x_{q_1} = 0$ and $x_{r_1} = 0$. Since these local guards already hold, the synchronization requirement is the only nontrivial condition:

$$t_{q_1} = t_{r_1} \iff B + k = A \iff k = A - B = c_1.$$

Hence a successful continuation must execute the loop b exactly c_1 times, neither fewer nor more.

After the action c, the three reference clocks satisfy

$$t_{p_1} = A + c_1, \qquad t_{q_1} = A, \qquad t_{r_1} = A,$$

and all local clocks except x'_{q_1} are 0, while

$$x'_{q_1} = c_1.$$

If $c_1 = 0$, then only the branch guarded by $x'_{q_1} = 0$ is enabled. This enters the block gadget. The only outgoing edge there requires $x_{p_1} < 0$, which is impossible over nonnegative clock valuations. Therefore no successful big step can start from an encoding of $(\ell, 0, c_2)$.

Assume now that $c_1 > 0$. Then the branch guarded by $x_{q_1} = 0,\ x'_{q_1} > 0,\ \{x'_{q_1}\}$ is enabled and moves the run to the decrement sub-gadget. From there, the only way back to an encoding configuration is the branch in which p_1 takes its outgoing edge with $x_{p_1} = 0$ (hence lets no time pass) and q_1 takes its outgoing edge with $x_{q_1} = 1$ (hence lets exactly one time unit pass). Both clocks are then reset. The resulting reference-clock values are

$$t'_{p_1} = A + c_1, \qquad t'_{q_1} = A + 1, \qquad t'_{r_1} = A.$$

Therefore

$$t'_{p_1} - t'_{q_1} = c_1 - 1, \qquad t'_{r_1} \le t'_{q_1} \le t'_{p_1},$$

and all local clocks of p_1, q_1, r_1 are once again 0. Since the second counter has been threaded through unchanged, the full target configuration encodes $(\ell+1, c_1-1, c_2)$.

The converse direction has been proved along the way: any successful big step must fire c, hence must use exactly c_1 loop iterations, and the guard on x'_{q_1} then forces the run into the decrement branch precisely when $c_1 > 0$. In that branch the target encoding computed above is the only possible one. □

Jump-on-zero. The jump-on-zero gadget reuses the same three-phase timing test. As before, it first stores the current value of t_{p_1} in the witness agent r_1, then advances p_1 and q_1 together until q_1 meets the witness. The auxiliary clock x'_{q_1} again records whether the replay phase consumed a positive amount of time.

If $c_1 = 0$, then q_1 is already aligned with the witness after Phase 1, so the replay phase needs zero iterations. The auxiliary clock therefore records the zero case, and the gadget branches to the target instruction ℓ' while preserving the encoded drift. If $c_1 > 0$, then the replay phase must take a positive amount of time before the final synchronization becomes possible; in that case the gadget branches to $n_{\ell+1}$, again without changing the stored value of the counter.

Lemma 3.14. *The jump-on-zero gadget for C_1 has a successful big step from a configuration encoding (ℓ, c_1, c_2) to a configuration encoding (ℓ', c_1, c_2) iff $c_1 = 0$, and to a configuration encoding $(\ell + 1, c_1, c_2)$ iff $c_1 > 0$.*

Proof. Let (C, v) encode (ℓ, c_1, c_2), and write

$$A := v(t_{p_1}), \qquad B := v(t_{q_1}), \qquad R := v(t_{r_1}),$$

so that $A - B = c_1$ and $R \leq B \leq A$. The upper part of the jump gadget in Figure 3.8 is identical to the zero-check part of the decrement gadget. Repeating the argument from the proof of Lemma 3.13, we obtain the following facts.

First, the initial st action is forced with delay 0 by the zero guards on all local clocks. Next, the synchronizing node n_1 copies the current reference time of p_1 into the witness r_1 while keeping p_1 fixed, so after Phase 1 we have

$$t_{p_1} = A, \qquad t_{q_1} = B, \qquad t_{r_1} = A$$

and all local clocks are 0.

Then the unsynchronizing loop b at node n_2 may be executed k times. As in the decrement gadget, after k loop iterations we have

$$t_{p_1} = A + k, \qquad t_{q_1} = B + k, \qquad t_{r_1} = A,$$

$$x_{p_1} = x_{q_1} = 0, \qquad x'_{q_1} = k, \qquad x_{r_1} = 0.$$

The subsequent action a is taken with zero additional delay, and the final synchronizing action c at node n_3 is enabled iff

$$B + k = A,$$

that is, iff $k = c_1$. Hence any successful continuation reaches node n_4 with

$$t_{p_1} = A + c_1, \qquad t_{q_1} = A, \qquad t_{r_1} = A,$$

and with

$$x_{p_1} = x_{q_1} = x_{r_1} = 0, \qquad x'_{q_1} = c_1.$$

At this point the gadget branches on the value of x'_{q_1}. If $c_1 = 0$, then only the branch guarded by $x'_{q_1} = 0$ is enabled. This branch resets x'_{q_1} and routes the run to the gadget implementing ℓ'. No additional delay is taken by p_1, q_1, r_1, so the reference-clock values remain

$$t'_{p_1} = A, \qquad t'_{q_1} = A, \qquad t'_{r_1} = A,$$

and all local clocks are again 0. Therefore the target configuration encodes $(\ell', 0, c_2)$, which is the same as (ℓ', c_1, c_2) in this case.

If $c_1 > 0$, then only the branch guarded by $x_{q_1} = 0,\ x'_{q_1} > 0,\ \{x'_{q_1}\}$ is enabled. This branch also adds no further local delay for p_1, q_1, r_1; it merely resets x'_{q_1} and routes the run to the gadget implementing $\ell + 1$. Hence the reference-clock values

remain

$$t'_{p_1} = A + c_1, \qquad t'_{q_1} = A, \qquad t'_{r_1} = A,$$

so that

$$t'_{p_1} - t'_{q_1} = c_1, \qquad t'_{r_1} \leq t'_{q_1} \leq t'_{p_1}.$$

All local clocks are again 0, and the second counter is unchanged because the agents p_2, q_2, r_2 are threaded through the branch with only zero-guard transitions. Thus the target configuration encodes $(\ell + 1, c_1, c_2)$.

Conversely, the branch guards show that the jump target ℓ' is reachable exactly when $x'_{q_1} = 0$, i.e. exactly when $c_1 = 0$, and that the fall-through target $\ell + 1$ is reachable exactly when $x'_{q_1} > 0$, i.e. exactly when $c_1 > 0$. These are the only successful big steps from an encoding configuration. □

The gadgets for counter C_2 are symmetric. By composing the appropriate gadget after each instruction node n_ℓ, the LTN $\mathcal{N}_M$ simulates every step of M and only those steps.

Proof of Theorem 3.11. For every machine instruction ℓ, attach to the node n_ℓ the gadget of the appropriate type: the increment gadget for instructions $C_i{+}{+}$, the decrement gadget for instructions "if $C_i > 0$ then $C_i{-}{-}$", and the jump-on-zero gadget for instructions "if $C_i = 0$ goto ℓ'". For instructions on counter C_2, use the symmetric copies of the gadgets discussed above.

By Lemma 3.12, Lemma 3.13, and Lemma 3.14, the resulting negotiation $\mathcal{N}_M$ has the following property: from every encoding configuration of a machine state (ℓ, c_1, c_2), the successful big steps are exactly the encodings of the machine successors of (ℓ, c_1, c_2). In other words, machine steps and big steps coincide on the encoding invariant.

Let ℓ_f be the halting instruction of M. Add to the corresponding node n_{ℓ_f} a fresh outcome halt whose local guard for each agent is the appropriate zero guard on its local clock(s), whose reset set is empty, and whose successor marking is an arbitrary sink marking. In every encoding configuration of the halting instruction all local clocks are already 0, so halt is enabled immediately. Conversely, the location $(n_{\ell_f}, \mathsf{halt})$ cannot be executed from any encoding of a non-halting instruction, because its source node is n_{ℓ_f}.

We now prove the correctness of the reduction. If

$$(\ell_0, c_1^0, c_2^0) \to (\ell_1, c_1^1, c_2^1) \to \cdots \to (\ell_m, c_1^m, c_2^m)$$

is a run of the machine, then repeated application of the three gadget lemmas yields a sequence of big steps

$$(C_0, v_0) \Rightarrow (C_1, v_1) \Rightarrow \cdots \Rightarrow (C_m, v_m)$$

such that (C_j, v_j) encodes (ℓ_j, c_1^j, c_2^j) for every j. If $\ell_m = \ell_f$, the additional outcome halt is enabled at (C_m, v_m), so the location $(n_{\ell_f}, \mathsf{halt})$ is reachable in $\mathcal{N}_M$.

Conversely, suppose $(n_{\ell_f}, \mathsf{halt})$ is reachable in $\mathcal{N}_M$. By construction, the only incoming edges to an instruction node n_ℓ are the final exits of instruction gadgets, and Lemma 3.12, Lemma 3.13, and Lemma 3.14 show that each such exit re-establishes the encoding invariant. Hence any configuration from which halt can be executed at n_{ℓ_f} is necessarily an encoding configuration of the halting instruction. Removing the final halt action, the remaining part of the run decomposes into big steps between encoding configurations, and the converse directions of the three gadget lemmas turn those big steps into a valid run of the counter machine that reaches ℓ_f.

Therefore M halts iff $\mathcal{N}_M$ reaches the location $(n_{\ell_f}, \mathsf{halt})$. Since the halting problem for two-counter machines is undecidable, location reachability for LTNs is undecidable as well. □

The reduction also makes precise why the model crosses the decidability boundary: local drift provides an unbounded numerical store, and synchronizing nodes act as equality tests on that store. Either ingredient alone is much less dangerous; together they simulate counters.

3.8 Summary and perspective

This chapter introduced *local-timed negotiations* (LTNs) as a real-time extension of negotiations in which each agent carries its own set of local clocks, and time elapses *locally* (via delay vectors). The semantics combines three ingredients:

- the negotiation-style control flow captured by markings (who is ready to participate in which node),
- local clock constraints (guards and resets attached to outcomes), and
- *optional* explicit synchronization at selected nodes, expressed as the constraint that all participating agents have equal reference clocks.

We illustrated how LTNs model timing constraints between interactions using the ATM example (Figure 3.1). We then highlighted two expressiveness phenomena that are characteristic of the local-time setting: (i) synchronization can be used as a *causal enforcement* mechanism, forcing agents to perform intermediate interactions (Section **??**), and (ii) the untimed outcome language of an LTN can be non-regular (Section 3.6.2), in sharp contrast to timed automata where the untimed language is always regular.

The main algorithmic message of the chapter is negative: we proved that *location-reachability is undecidable* for LTNs (Theorem 3.11) by a reduction from

halting of 2-counter machines. Conceptually, the reduction pinpoints the source of power: reference clocks can drift arbitrarily, and synchronizing nodes allow the system to *compare* and *transfer* information encoded in those drifts. In particular, the proof relies on a careful interplay between synchronized nodes (where equality of reference clocks must be met) and unsynchronized nodes (where agents can advance independently and thus accumulate drift).

Perspective. LTNs sit between two well-studied extremes. On one hand, they inherit the interaction-centric, partial-order flavour of negotiations; on the other hand, they adopt a clock-based view of time as in timed automata, but without a global time line. This combination is expressive enough to model realistic timing patterns (local deadlines, response windows, and time-bounded protocols), while also making plain that *unrestricted* local-time drift together with *explicit* synchronization yields a model beyond decidability for basic reachability.

This motivates the rest of the thesis: to identify robust restrictions that preserve the modelling advantages of LTNs but regain algorithmic tractability. Two orthogonal axes stand out naturally from this chapter:

- restricting (or eliminating) explicit synchronization, so that drift cannot be tested directly; and
- bounding the amount of drift that can accumulate (globally, per-agent, or per-cycle), so that comparisons do not simulate unbounded counters.

Understanding where decidability reappears along these axes also clarifies which expressiveness features (e.g. enforcing intermediate interactions, or generating non-regular untimed languages) survive in the restricted models, and which do not.

What causes undecidability? The reduction pinpoints two features:

1. **Unbounded drift**: Reference clocks can diverge arbitrarily, encoding unbounded integer values (counter contents).
2. **Explicit synchronization**: Synchronizing nodes allow the system to test equality of reference clocks, enabling comparisons and conditional behavior based on encoded values.

Removing or restricting either feature may recover decidability—this is the focus of Part II.

3.9 Coming next

This chapter concludes Part I, where we introduced local-timed negotiations and established that, in their full generality, even basic location-reachability is undecidable (Theorem 3.11). The remainder of the thesis (Part II) is devoted to

recovering decidability by identifying restrictions that preserve the modelling advantages of LTNs while taming the two sources of algorithmic hardness exposed by the reduction: *unrestricted drift* between local reference clocks and the ability to *test* that drift via explicit synchronization.

Part II develops several decidable fragments along complementary axes:

- **Extreme synchronization regimes:** we study the two endpoints where synchronization is either *forbidden* (synchronization-free LTNs) or *mandatory* at every interaction (always-synchronized LTNs), and analyze how these extremes change both expressiveness and the reachability problem.
- **Quantitative restrictions on drift:** we introduce drift-bounded variants that limit how far apart agents' reference clocks may deviate, yielding finite (or well-structured) symbolic state spaces.
- **Structural restrictions:** we consider communication/topological constraints on the underlying negotiation graph—for instance, connected communication patterns—and show how they can re-establish decidability even when clocks remain local.
- **Few-process settings:** we isolate the two-process case as a particularly informative boundary, where local time still matters but the concurrency structure becomes constrained enough to admit specialized algorithms.

Immediate next chapter. We begin Part II by focusing on the first axis above, namely the two extreme synchronization regimes: *synchronization-free* LTNs and *always-synchronized* LTNs.

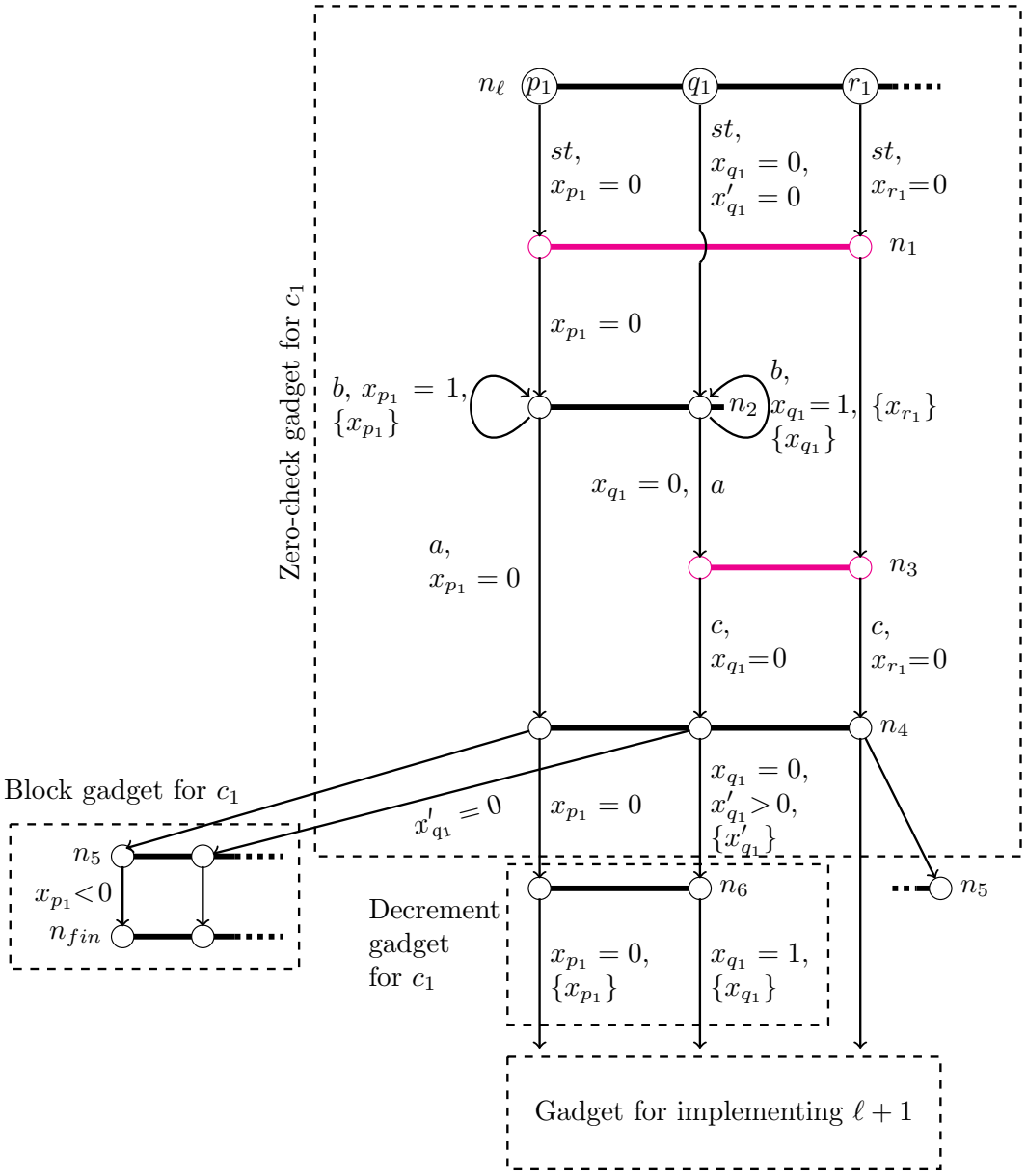


FIGURE 3.7: Decrement gadget for counter C_1. The highlighted horizontal bars are the synchronizing nodes n_1 and n_3; the middle node n_2 is deliberately unsynchronizing. The gadget first copies the current reference time of p_1 into the witness agent r_1, then advances p_1 and q_1 together until q_1 catches up with that frozen witness. The branch taken afterwards depends on whether a positive amount of time was needed.

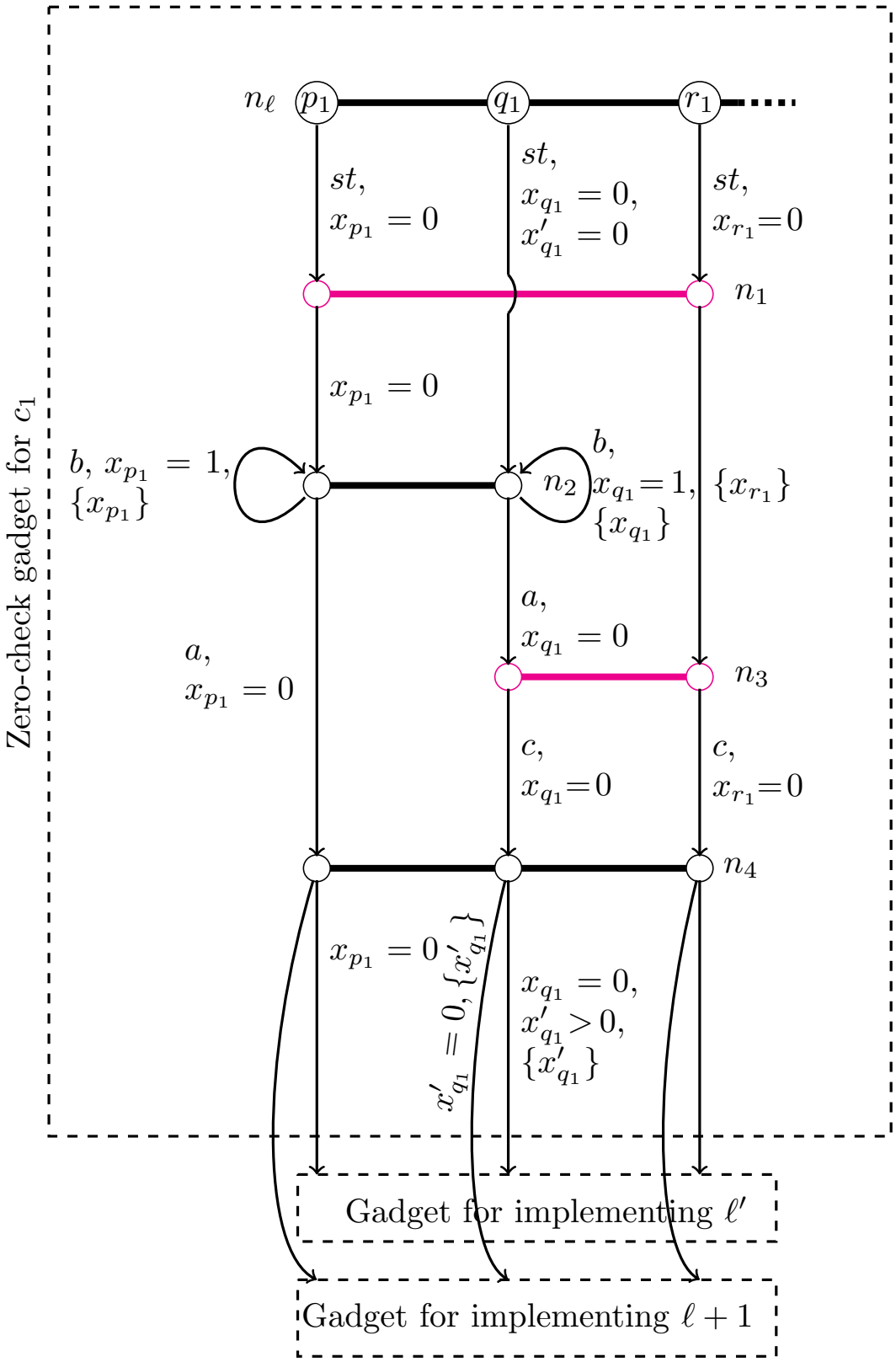


FIGURE 3.8: Jump-on-zero gadget for counter C_1. It shares the same timing test as the decrement gadget, but routes the zero case to instruction ℓ' and the positive case to instruction $\ell + 1$.

Part II

Decidable subclasses for reachability in LTNs

Part I established local-timed negotiations as a model for concurrent real-time systems and showed that this expressiveness comes at a fundamental cost: location reachability is undecidable in the presence of unrestricted local-time drift combined with selective synchronization (Chapter 3). The undecidability proof isolated two sources of algorithmic hardness—unbounded divergence of reference clocks and the ability to compare accumulated drift via synchronizing nodes.

In this part, we turn to the complementary question: under which restrictions can reachability be recovered as a decidable problem, without collapsing the model back to global time or eliminating concurrency altogether? We study several fragments of local-timed negotiations obtained by constraining synchronization patterns, temporal drift, or communication structure, and develop tailored proof techniques for each. Chapter 4 begins this investigation by analyzing the two extreme synchronization regimes, which serve as conceptual and technical anchor points for the decidable landscape explored in the chapters that follow.

Chapter 4

Synchronization Extremes: Free and Always-Synchronizing LTNs

Chapter 3 showed that location reachability is undecidable for general local-timed negotiations. The proof of Theorem 3.11 identified two features whose interaction is essential for this result:

1. *Unbounded clock drift*: the reference clocks t_p and t_q of different agents may diverge arbitrarily, allowing the encoding of unbounded integer values.

2. *Explicit synchronization*: synchronizing nodes require equality of reference clocks, enabling the system to compare and condition behavior on accumulated drift.

It is precisely the combination of these two mechanisms—arbitrary accumulation of drift and the ability to test it—that yields the expressive power of counter machines. Part II explores how decidability can be recovered by restricting one or both of these sources of power.

This chapter focuses on the first axis of restriction, namely synchronization. Rather than allowing arbitrary mixtures of synchronizing and non-synchronizing nodes, we study the two extreme regimes:

- **Synchronization-free LTNs** (Sync $= \emptyset$), where no node imposes equality of reference clocks and agents evolve completely independently.

- **Always-synchronizing LTNs** (Sync $= N$), where every interaction requires temporal alignment of participating agents.

In both cases, the critical interaction between unbounded drift and synchronization is eliminated, albeit in opposite ways. As a consequence, location reachability becomes decidable.

Conceptual role of the extremes. In these two fragments, we recover decidability by removing the interplay between drift and synchronization. Understanding why these extremes are decidable clarifies which features cause undecidability in the general model. Despite of having the same worst-case complexity, the two fragments require fundamentally different proof techniques. Synchronization-free LTNs admit a componentwise abstraction that exploits the complete independence of agents, whereas always-synchronizing LTNs collapse to a global-time semantics and can be reduced to classical timed automata.

Main results. The central results of this chapter are the following.

Theorem 4.1 (Chapter results)**.** *Location reachability is PSPACE-complete for each of the following classes:*

1. *synchronization-free local-timed negotiations (Section 4.1);*
2. *always-synchronizing local-timed negotiations (Section* **??***).*

In both cases, the complexity matches that of reachability for timed automata. Thus, while these fragments do not exploit the negotiation structure to reduce complexity, they also do not incur additional algorithmic cost beyond what is inherent to quantitative timing.

Organization. Section 4.1 studies synchronization-free LTNs via a product-style region abstraction. Section **??** analyzes always-synchronizing LTNs by reducing them to a global-time model. Section **??** synthesizes the lessons from both extremes, and Section **??** positions them within the broader landscape of decidable fragments studied in subsequent chapters.

4.1 Synchronization-Free LTNs

4.1.1 Definition and Behavior

Definition 4.2 (Synchronization-free LTN)**.** An LTN $\mathcal{N} = (N, \mathrm{dom}, \delta, \mathrm{Sync})$ is synchronization-free if $\mathrm{Sync} = \emptyset$.

In synchronization-free LTNs, no action step requires equality of reference clocks. When location (n, a) is executed, participating agents must satisfy their local guards (constraints on clocks in X_p), but their reference clocks t_p need not be equal. Agents accumulate local time independently between interactions, coordinating only through control flow (markings) and timing constraints (guards), never through explicit temporal alignment.

Expressiveness implications. Recall the examples from Chapter 3:

- **Vendor example (Figure 3.3)**: If n_3 is not synchronizing, agent q need not visit the vendor to "catch up" to p's reference clock. The causal enforcement mechanism disappears—q can proceed directly from n_1 to n_3 as soon as control flow permits and guards are satisfied.

- $a^n b^n c^n$ **example (Figure 5.2)**: If n_3 is not synchronizing, the language changes dramatically. Without synchronization forcing $t_p = t_q$, there's no mechanism to compare how many a's versus b's occurred. The untimed language becomes simply $\{w \cdot c \mid w \in \{a, b\}^*,\ w \text{ contains} \geq 1\, a \text{ and} \geq 1\, b\}$ (any interleaving with at least one of each), which is regular.

Since Sync $= \emptyset$, no action step tests $t_p = t_q$, and reference clocks t_p never appear in any guard and are never reset (by definition, guards use only clocks in X_p), only advance monotonically. Therefore, for reachability purposes, the absolute values of reference clocks are irrelevant—only the values of local clocks in X matter.

The following example illustrates that although each agent's timing constraints are local, the combination of guards across agents at a location can create implicit coordination requirements. Consider Figure 4.2:

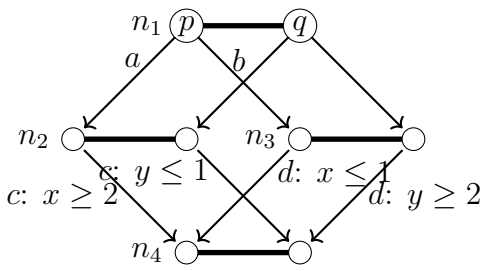


FIGURE 4.1: Synchronization-free LTN illustrating implicit coordination. At n_1, agents p and q non-deterministically choose outcome a or b. For outcome c at n_4 to be feasible, both must have chosen a (route through n_2), since c requires $x \geq 2$ from p and $y \leq 1$ from q—constraints incompatible with the b-route guards.

At n_1, outcome a or b is chosen non-deterministically. To reach location (n_4, c), both agents must route through n_2 (both choosing a), since c requires $x \geq 2$ from p and $y \leq 1$ from q. Routing through n_3 (via outcome b) imposes $x \leq 1$ for p and $y \geq 2$ for q. These constraints make (n_4, c) unreachable if either agent chose b. Thus, local timing decisions at n_1 have global reachability consequences, even without explicit synchronization. The agents must implicitly 'agree' on a path to make later locations feasible.

4.1.2 Product-region abstraction

Geometric intuition. In synchronization-free LTNs, each agent evolves according to its own local clocks without ever comparing time with other agents. Consequently, the global time state of the system can be seen as a tuple of independent local time states—one per agent.

For each agent p, the standard region abstraction partitions $\mathbb{R}_{\geq 0}^{X_p}$ into finitely many regions. The global abstraction is then obtained by taking the Cartesian product of these per-agent partitions. Two global valuations are equivalent if and only if they are equivalent for each agent separately.

Intuitively, synchronization-free LTNs behave like a collection of independent timed automata whose discrete transitions must occur together at negotiation nodes, but whose timing evolution is otherwise decoupled.

Let

$$M := \max\{\, c \mid c \text{ appears in some atomic constraint } x \bowtie c \text{ of a guard in } \mathcal{N} \,\}.$$

In this fragment, reference clocks $T = \{t_p \mid p \in P\}$ do not appear in guards and are never tested for synchronization. Thus, for reachability, it suffices to abstract only the local clocks X.

We define a region equivalence *per agent* and take their product.

Definition 4.3 (Per-agent region equivalence)**.** Fix an agent $p \in P$. For valuations $v, v' : X_p \to \mathbb{R}_{\geq 0}$, write $v \sim_M^p v'$ if for all clocks $x, y \in X_p$:

1. **Integer parts agree (or both exceed M):**
   Either $\lfloor v(x) \rfloor = \lfloor v'(x) \rfloor$, or both $\lfloor v(x) \rfloor > M$ and $\lfloor v'(x) \rfloor > M$.

2. **Zero fractional parts agree:**
   If $v(x) \leq M$, then $\mathrm{frac}(v(x)) = 0$ iff $\mathrm{frac}(v'(x)) = 0$.

3. **Fractional orderings agree (for clocks $\leq M$):**
   If $v(x) \leq M$ and $v(y) \leq M$, then $\mathrm{frac}(v(x)) \leq \mathrm{frac}(v(y))$ iff $\mathrm{frac}(v'(x)) \leq \mathrm{frac}(v'(y))$.

This is precisely the standard timed-automata region equivalence [**?**] applied to clocks X_p of agent p.

Definition 4.4 (Product-region equivalence)**.** For valuations $v, v' : X \to \mathbb{R}_{\geq 0}$ (over all local clocks), write $v \sim_{\mathrm{pr}} v'$ if:

$$v|_{X_p} \sim_M^p v'|_{X_p} \quad \text{for every agent } p \in P.$$

We call $\sim_{\mathrm{pr}}$ the *product-region equivalence.* Let $[v]_{\mathrm{pr}}$ denote the equivalence class of v.

Lemma 4.5 (Finite index)**.** *The product-region equivalence $\sim_{\mathrm{pr}}$ has finite index. In particular, the number of product-regions is bounded by $\mathcal{O}(|X|!\cdot 2^{|X|}\cdot(2M+2)^{|X|})$.*

Proof of Lemma 4.5. Fix an agent $p \in P$ and let $k_p := |X_p|$. We first bound the number of $\sim_M^p$-classes over valuations of X_p, and then take the product over all agents.

A $\sim_M^p$-class is determined by the following finite data:

(A) Truncated integer parts. For each $x \in X_p$, define its *truncated integer part*

$$\iota(x) \;:=\; \begin{cases} \lfloor v(x) \rfloor & \text{if } \lfloor v(x) \rfloor \le M, \\ M+1 & \text{if } \lfloor v(x) \rfloor > M. \end{cases}$$

Thus $\iota(x) \in \{0, 1, \ldots, M, M+1\}$, giving at most $(M+2)^{k_p}$ possibilities.

(B) Zero/nonzero fractional flags for clocks not above M. For each $x \in X_p$ with $\iota(x) \le M$ we record the Boolean

$$z(x) \;:=\; \big(\{v(x)\} = 0\big).$$

For clocks with $\iota(x) = M+1$ (i.e. $v(x) > M$) this flag is irrelevant in the definition of $\sim_M^p$, so we may still count it pessimistically. This yields at most 2^{k_p} possibilities.

(C) Ordering of fractional parts among clocks not above M. Let $S := \{x \in X_p \mid \iota(x) \le M\}$. The third bullet in Definition 4.4 requires that for all $x, y \in S$,

$$\{v(x)\} \le \{v(y)\} \quad \text{iff} \quad \{v'(x)\} \le \{v'(y)\}.$$

Equivalently, it fixes the total preorder of the multiset $\big(\{v(x)\}\big)_{x\in S}$. Any total preorder on $|S| \le k_p$ elements can be obtained by taking a total order and allowing ties; hence the number of such preorders is bounded by $k_p! \cdot 2^{k_p}$ (choose an order, then choose where to merge adjacent items). Therefore, the number of possibilities for (C) is at most $k_p! \cdot 2^{k_p}$.

Combining (A), (B), (C), the number of $\sim_M^p$-classes is bounded by

$$(M+2)^{k_p} \cdot 2^{k_p} \cdot (k_p! \cdot 2^{k_p}) \;=\; k_p! \cdot 2^{2k_p} \cdot (M+2)^{k_p}.$$

Now $\sim_{\mathrm{pr}}$ is the product of the per-agent relations, so the number of product-regions is at most the product over agents:

$$\prod_{p\in P}\Big(k_p! \cdot 2^{2k_p} \cdot (M+2)^{k_p}\Big) \;=\; \Big(\prod_p k_p!\Big) \cdot 2^{2\sum_p k_p} \cdot (M+2)^{\sum_p k_p}.$$

Since $\sum_p k_p = |X|$ and $\prod_p k_p! \le |X|!$, we obtain

$$\#(\text{product-regions}) \;\le\; |X|! \cdot 2^{2|X|} \cdot (M+2)^{|X|}.$$

Finally, using $M + 2 \leq 2M + 2$ for $M \geq 0$ and absorbing the extra $2^{|X|}$ into the $\mathcal{O}$-notation, we get the stated bound

$$\#(\text{product-regions}) \;=\; \mathcal{O}\big(|X|! \cdot 2^{|X|} \cdot (2M+2)^{|X|}\big).$$

Hence $\sim_{\text{pr}}$ has finite index. □

Lemma 4.6 (Compatibility with local delays)**.** *Let $v \sim_{\text{pr}} v'$. For every local delay vector $\Delta \in \mathbb{R}^P_{\geq 0}$ there exists a local delay vector Δ' such that $v + \Delta \sim_{\text{pr}} v' + \Delta'$.*

Proof. Assume $v \sim_{\text{pr}} v'$ and fix a local delay vector $\Delta \in \mathbb{R}^P_{\geq 0}$. We construct a local delay vector $\Delta' \in \mathbb{R}^P_{\geq 0}$ such that $v + \Delta \sim_{\text{pr}} v' + \Delta'$.

Fix an agent $p \in P$ and write $v_p := v|_{X_p}$ and $v'_p := v'|_{X_p}$. By definition of $\sim_{\text{pr}}$, we have $v_p \sim^p_M v'_p$.

Now consider the standard timed-automata region equivalence on the clock set X_p with maximal constant M; our relation $\sim^p_M$ is exactly this equivalence (restricted to clocks of agent p). By the time-elapse compatibility of regions proved in the preliminaries (Lemma **??**), from $v_p \sim^p_M v'_p$ and $d := \Delta(p) \geq 0$ we obtain a delay $d' \geq 0$ such that

$$v_p + d \;\sim^p_M\; v'_p + d'.$$

Define $\Delta'(p) := d'$.

Doing this independently for every agent p yields a vector $\Delta' \in \mathbb{R}^P_{\geq 0}$ satisfying, for each p,

$$(v + \Delta)|_{X_p} \;=\; v_p + \Delta(p) \;\sim^p_M\; v'_p + \Delta'(p) \;=\; (v' + \Delta')|_{X_p}.$$

Since $\sim_{\text{pr}}$ is defined componentwise over agents, this implies $v + \Delta \sim_{\text{pr}} v' + \Delta'$, as required. □

Lemma 4.7 (Compatibility with guards and resets)**.** *Let $v \sim_{\text{pr}} v'$. Let g be any guard whose constants are at most M. Then:*

1. *$v \models g$ iff $v' \models g$;*

2. *for every set of local clocks $Y \subseteq X$, we have $v[Y] \sim_{\text{pr}} v'[Y]$.*

Proof. Assume $v \sim_{\text{pr}} v'$, and let g be any guard whose constants are at most M. By the syntax used in the chapter, g is a Boolean combination of atomic constraints of the form $x \bowtie c$ where $c \in \{0, 1, \ldots, M\}$ and $\bowtie \in \{<, \leq, =, \geq, >\}$, and x is a local clock.

(1) Preservation of guard satisfaction. Since g is a Boolean combination, it suffices to prove preservation for a single atomic constraint $x \bowtie c$.

Fix such an atom. Let p be the unique agent with $x \in X_p$. From $v \sim_{\text{pr}} v'$ we get $v|_{X_p} \sim^p_M v'|_{X_p}$, hence in particular:

- either $\lfloor v(x) \rfloor = \lfloor v'(x) \rfloor$, or both $\lfloor v(x) \rfloor > M$ and $\lfloor v'(x) \rfloor > M$;
- if $v(x) \leq M$ then $\{v(x)\} = 0$ iff $\{v'(x)\} = 0$.

Now consider cases.

Case I: $v(x) > M$. Then $v'(x) > M$ as well. Since $c \leq M$, we have $v(x) \bowtie c$ iff $v'(x) \bowtie c$ for all $\bowtie \in \{>, \geq\}$ (both true) and for all $\bowtie \in \{<, \leq, =\}$ (both false). So the atom is preserved.

Case II: $v(x) \leq M$. Then $\lfloor v(x) \rfloor = \lfloor v'(x) \rfloor =: k \leq M$. Write $v(x) = k + f$ and $v'(x) = k + f'$ with $f, f' \in [0, 1)$ and $f = 0$ iff $f' = 0$.

Because c is an integer, the truth of $x \bowtie c$ depends only on: (i) whether $k < c$, $k > c$, or $k = c$; and (ii) in the boundary case $k = c$, whether the fractional part is 0 or not.

Indeed:

- $v(x) < c$ iff $k < c$;
- $v(x) \leq c$ iff $k < c$ or ($k = c$ and $f = 0$);
- $v(x) = c$ iff $k = c$ and $f = 0$;
- $v(x) \geq c$ iff $k > c$ or ($k = c$ and $f = 0$);
- $v(x) > c$ iff $k > c$.

All these conditions are preserved when replacing (k, f) by (k, f') because k is the same and $f = 0 \Leftrightarrow f' = 0$. Hence $v \models (x \bowtie c)$ iff $v' \models (x \bowtie c)$.

This proves preservation for atoms, hence for any Boolean combination g.

(2) Preservation under resets. Let $Y \subseteq X$ be any set of local clocks to reset to 0. We must show $v[Y] \sim_{\text{pr}} v'[Y]$.

Fix an agent p. We show $(v[Y])|_{X_p} \sim_M^p (v'[Y])|_{X_p}$. Let $Y_p := Y \cap X_p$. Then $(v[Y])|_{X_p}$ is obtained from $v|_{X_p}$ by setting clocks in Y_p to 0 (and leaving others unchanged), and similarly for v'.

Take any $x \in X_p$.

- If $x \in Y_p$, then both valuations assign 0 to x, so the first two bullets of $\sim_M^p$ hold trivially for x.
- If $x \notin Y_p$, then $v[Y](x) = v(x)$ and $v'[Y](x) = v'(x)$, so the first two bullets hold because they held for v and v'.

For the third bullet (ordering of fractional parts among clocks $\leq M$), note that every reset clock has fractional part 0, and every non-reset clock keeps its old fractional part. Since in v and v' the relative order relations $\{v(x)\} \leq \{v(y)\}$ among clocks $\leq M$ were identical, and since we apply the same transformation "set some designated clocks to 0" on both sides, the resulting order relations

among fractional parts remain identical as well. Formally, for any $x, y \in X_p$ with $(v[Y])(x) \leq M$ and $(v[Y])(y) \leq M$:

- if both are not reset, the relation is the same as before;
- if one is reset, its fractional part becomes 0 on both sides;
- if both are reset, both fractional parts are 0 on both sides.

Hence the iff-condition in the third bullet is preserved.

Therefore $(v[Y])|_{X_p} \sim_M^p (v'[Y])|_{X_p}$ for every p, i.e. $v[Y] \sim_{\text{pr}} v'[Y]$. □

4.1.3 The product-region automaton

We now build a finite automaton over locations that captures precisely the feasible untimed location sequences of a synchronization-free LTN.

Definition 4.8 (Product-region automaton)**.** Let $\mathcal{N}$ be synchronization-free. The product-region automaton $\mathcal{A}_{\text{pr}}(\mathcal{N})$ is an NFA with:

- **States**: pairs (C, r) where C is a marking and r is a product-region.
- **Transitions**: $(C, r) \xrightarrow{(n,a)} (C', r')$ iff there exist $v \in r$ and local delay Δ such that:
$$(C, v) \xrightarrow{\Delta} (C, v + \Delta) \xrightarrow{(n,a)} (C', v')$$
in $\mathcal{N}$, with $v' \in r'$.
- **Initial state**: $(C_0, [v_0]_{\text{pr}})$ where $C_0(p) = \{n_{\text{in}}\}$ for all p, and v_0 maps all clocks in X to 0.

4.1.4 Product-Region Calculations on an Example

We illustrate the product-region abstraction (Section 4.1) by working out regions explicitly on a tiny synchronization-free LTN instance. Recall that in our model, each agent's guard mentions only its own local clocks (see Lemma **??**(2)). [1]

The example. Let $P = \{p, q\}$, with local clocks $X_p = \{x\}$ and $X_q = \{y\}$. Consider three nodes $N = \{n_0, n_1, n_2\}$ with

$$dom(n_0) = \{p, q\}, \qquad dom(n_1) = \{p\}, \qquad dom(n_2) = \{p, q\}.$$

We fix outcomes a, b, c and local transition data:

- At (n_0, a): no guards, no resets (only control-flow updates).

[1] This locality assumption is used throughout this chapter.

- At (n_1, b) (agent p only): guard $x \geq 1$, reset $x := 0$.
- At (n_2, c) (agents p, q): for q we require guard $y \geq 1$ and reset $y := 0$ (agent p has trivial guard/reset at this step).

This example is synchronization-free, i.e. $Sync = \emptyset$.

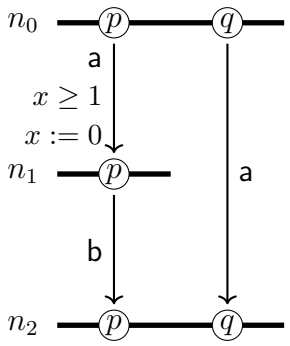


FIGURE 4.2: A simple synchronization-free LTN.

Maximal constant. The only constants appearing in guards are 1, hence

$$M = 1.$$

Per-agent regions for one clock (with $M = 1$)

Since each agent has a single clock, the standard region equivalence partitions $\mathbb{R}_{\geq 0}$ into four regions:

$$\begin{aligned} R^0 &:= \{z \mid z = 0\}, \\ R^{(0,1)} &:= \{z \mid 0 < z < 1\}, \\ R^1 &:= \{z \mid z = 1\}, \\ R^{>1} &:= \{z \mid z > 1\}. \end{aligned}$$

For agent p (clock x) we write these as $R_x^0, R_x^{(0,1)}, R_x^1, R_x^{>1}$, and similarly for agent q (clock y) we write $R_y^0, R_y^{(0,1)}, R_y^1, R_y^{>1}$.

Product regions

A product-region is a pair of per-agent regions:

$$r \;=\; R_x^\alpha \times R_y^\beta \qquad \text{where} \qquad \alpha, \beta \in \{0, (0,1), 1, > 1\}.$$

Hence there are $4 \cdot 4 = 16$ product-regions in total.

Guard satisfaction at the region level. With $M = 1$ and one clock per agent:

$$x \geq 1 \text{ holds exactly on } R_x^1 \cup R_x^{>1}, \qquad y \geq 1 \text{ holds exactly on } R_y^1 \cup R_y^{>1}.$$

This is the concrete instantiation of Lemma **??**(2).

A concrete run and its product-region trace

We now pick explicit local delays (a delay vector $\Delta \in \mathbb{R}_{\geq 0}^P$) and track the induced product-region sequence.

Initial valuation. Let $v_0(x) = 0$ and $v_0(y) = 0$. Then

$$(x, y) = (0, 0) \in R_x^0 \times R_y^0, \quad \text{so} \quad r_0 := [v_0]_{pr} = R_x^0 \times R_y^0.$$

Step 1: delay then (n_1, b). Choose a local delay Δ_0 with

$$\Delta_0(p) = 1.3, \qquad \Delta_0(q) = 0.4.$$

After delay, we have

$$(x, y) = (1.3, 0.4) \in R_x^{>1} \times R_y^{(0,1)}, \quad \text{so} \quad r_1 = R_x^{>1} \times R_y^{(0,1)}.$$

Now (n_1, b) is enabled because $x \geq 1$ holds in $R_x^{>1}$. Executing (n_1, b) resets $x := 0$, yielding

$$(x, y) = (0, 0.4) \in R_x^0 \times R_y^{(0,1)}, \quad \text{so} \quad r_2 = R_x^0 \times R_y^{(0,1)}.$$

Step 2: delay then (n_2, c). Choose a local delay Δ_1 with

$$\Delta_1(p) = 0.2, \qquad \Delta_1(q) = 0.8.$$

After delay, we have

$$(x, y) = (0.2, 1.2) \in R_x^{(0,1)} \times R_y^{>1}, \quad \text{so} \quad r_3 = R_x^{(0,1)} \times R_y^{>1}.$$

Now (n_2, c) is enabled because $y \geq 1$ holds in $R_y^{>1}$. Executing (n_2, c) resets $y := 0$, yielding

$$(x, y) = (0.2, 0) \in R_x^{(0,1)} \times R_y^0, \quad \text{so} \quad r_4 = R_x^{(0,1)} \times R_y^0.$$

Region-trace summary. Altogether, this concrete run induces the product-region trace

$$R_x^0 \times R_y^0 \xrightarrow{\Delta_0 = (1.3,\, 0.4)} R_x^{>1} \times R_y^{(0,1)} \xrightarrow{(n_1, b):\, x \geq 1,\ x := 0} R_x^0 \times R_y^{(0,1)} \xrightarrow{\Delta_1 = (0.2,\, 0.8)} R_x^{(0,1)} \times R_y^{>1} \xrightarrow{(n_2, c):\, y \geq 1,\ y := 0}$$

This makes the product abstraction fully explicit: the global abstract state changes by (i) componentwise time-elapse within each per-agent region partition, followed by (ii) componentwise resets triggered by discrete steps.

Lemma 4.9 (Soundness and completeness)**.** *Assume $\mathcal{N}$ is synchronization-free.*

1. *For every run of $\mathcal{N}$*

$$(C_0, v_0) \xrightarrow{\Delta_0,(n_0,a_0)} (C_1, v_1) \xrightarrow{\Delta_1,(n_1,a_1)} \cdots \xrightarrow{\Delta_{m-1},(n_{m-1},a_{m-1})} (C_m, v_m),$$

there exists a run in $\mathsf{ProdRegAut}(\mathcal{N})$

$$(C_0, [v_0]) \xrightarrow{(n_0,a_0)} (C_1, [v_1]) \xrightarrow{(n_1,a_1)} \cdots \xrightarrow{(n_{m-1},a_{m-1})} (C_m, [v_m]).$$

2. *Conversely, for every run in* $\mathsf{ProdRegAut}(\mathcal{N})$

$$(C_0, r_0) \xrightarrow{(n_0,a_0)} (C_1, r_1) \xrightarrow{(n_1,a_1)} \cdots \xrightarrow{(n_{m-1},a_{m-1})} (C_m, r_m),$$

there exists a run in $\mathcal{N}$

$$(C_0, v_0) \xrightarrow{\Delta_0,(n_0,a_0)} (C_1, v_1) \xrightarrow{\Delta_1,(n_1,a_1)} \cdots \xrightarrow{\Delta_{m-1},(n_{m-1},a_{m-1})} (C_m, v_m)$$

such that $v_i \in r_i$ for all $0 \le i \le m$.

Detailed proof of Lemma 4.9. Assume $\mathcal{N}$ is synchronization-free, so only local clocks X matter and there is no synchronization test involving reference clocks.

(1) Soundness (from $\mathcal{N}$ to $\mathsf{ProdRegAut}(\mathcal{N})$). Let

$$(C_0, v_0) \xrightarrow{\Delta_0,(n_0,a_0)} (C_1, v_1) \xrightarrow{\Delta_1,(n_1,a_1)} \cdots \xrightarrow{\Delta_{m-1},(n_{m-1},a_{m-1})} (C_m, v_m)$$

be a run of $\mathcal{N}$. By the LTN semantics, each macro-step $\xrightarrow{\Delta_i,(n_i,a_i)}$ abbreviates a small-step decomposition

$$(C_i, v_i) \xrightarrow{\Delta_i} (C_i, v_i + \Delta_i) \xrightarrow{(n_i,a_i)} (C_{i+1}, v_{i+1}).$$

Set $r_i := [v_i]$ for each i. By Definition **??** of $\mathsf{ProdRegAut}(\mathcal{N})$, a transition

$$(C_i, r_i) \xrightarrow{(n_i,a_i)} (C_{i+1}, r_{i+1})$$

exists whenever there are witnesses $v \in r_i$, $\Delta \ge 0$ such that $(C_i, v) \xrightarrow{\Delta} (C_i, v + \Delta) \xrightarrow{(n_i,a_i)} (C_{i+1}, v')$ with $v' \in r_{i+1}$. We can take the concrete witnesses $v := v_i$, $\Delta := \Delta_i$, and $v' := v_{i+1}$. Thus every step of the $\mathcal{N}$-run induces a corresponding

step in $\mathsf{ProdRegAut}(\mathcal{N})$, yielding the run

$$(C_0, [v_0]) \xrightarrow{(n_0,a_0)} (C_1, [v_1]) \xrightarrow{(n_1,a_1)} \cdots \xrightarrow{(n_{m-1},a_{m-1})} (C_m, [v_m]).$$

(2) Completeness (from $\mathsf{ProdRegAut}(\mathcal{N})$ to $\mathcal{N}$). Let

$$(C_0, r_0) \xrightarrow{(n_0,a_0)} (C_1, r_1) \xrightarrow{(n_1,a_1)} \cdots \xrightarrow{(n_{m-1},a_{m-1})} (C_m, r_m)$$

be a run of $\mathsf{ProdRegAut}(\mathcal{N})$. We construct, by induction on $i = 0, \ldots, m$, valuations $v_i \in r_i$ and local delays Δ_i (for $i < m$) such that

$$(C_i, v_i) \xrightarrow{\Delta_i} (C_i, v_i + \Delta_i) \xrightarrow{(n_i,a_i)} (C_{i+1}, v_{i+1})$$

is a valid small-step decomposition in $\mathcal{N}$.

Base $i = 0$. By construction of $\mathsf{ProdRegAut}(\mathcal{N})$, $r_0 = [v_0]$ where v_0 is the all-zero valuation, so choose v_0 accordingly.

Induction step. Assume $v_i \in r_i$ has been chosen. Consider the automaton transition

$$(C_i, r_i) \xrightarrow{(n_i,a_i)} (C_{i+1}, r_{i+1}).$$

By Definition **??**, there exist witnesses: a valuation $u_i \in r_i$, a local delay vector $\widehat{\Delta}_i$, and a valuation $u_{i+1} \in r_{i+1}$ such that in $\mathcal{N}$,

$$(C_i, u_i) \xrightarrow{\widehat{\Delta}_i} (C_i, u_i + \widehat{\Delta}_i) \xrightarrow{(n_i,a_i)} (C_{i+1}, u_{i+1})$$

is a valid pair of steps.

Now $v_i \in r_i$ and $u_i \in r_i$ imply $v_i \sim_{\mathrm{pr}} u_i$. Apply Lemma 4.6 to $v_i \sim_{\mathrm{pr}} u_i$ and delay $\widehat{\Delta}_i$ to obtain a (possibly different) local delay vector Δ_i such that

$$v_i + \Delta_i \ \sim_{\mathrm{pr}} \ u_i + \widehat{\Delta}_i. \tag{$*$}$$

We claim that (n_i, a_i) is enabled from $(C_i, v_i + \Delta_i)$ and that firing it produces some $v_{i+1} \in r_{i+1}$.

Let g be the conjunction of all local guards required for executing outcome (n_i, a_i) (one per participating agent, combined as in the LTN semantics). Since $\mathcal{N}$ is synchronization-free, g depends only on local clocks and uses only constants $\leq M$ (by definition of M in the chapter). Because $u_i + \widehat{\Delta}_i$ enables (n_i, a_i), we have $u_i + \widehat{\Delta}_i \models g$. By $(*)$ and Lemma 4.7(1), we also have $v_i + \Delta_i \models g$, hence (n_i, a_i) is enabled from $(C_i, v_i + \Delta_i)$.

Let $Y \subseteq X$ be the set of local clocks reset by firing (n_i, a_i) (across all agents in $dom(n_i)$, exactly as prescribed by δ). In a synchronization-free LTN, the discrete step sets precisely those clocks in Y to 0 and leaves the others unchanged; thus

the successor valuation is

$$v_{i+1} := (v_i + \Delta_i)[Y], \qquad u_{i+1} := (u_i + \widehat{\Delta}_i)[Y].$$

By Lemma 4.7(2) applied to $(*)$, we get

$$(v_i + \Delta_i)[Y] \ \sim_{\mathrm{pr}} \ (u_i + \widehat{\Delta}_i)[Y],$$

i.e. $v_{i+1} \sim_{\mathrm{pr}} u_{i+1}$. Since $u_{i+1} \in r_{i+1}$ and r_{i+1} is a $\sim_{\mathrm{pr}}$-class, it follows that $v_{i+1} \in r_{i+1}$, completing the induction step.

Thus we obtain a concrete $\mathcal{N}$-run

$$(C_0, v_0) \xrightarrow{\Delta_0,(n_0,a_0)} (C_1, v_1) \xrightarrow{\Delta_1,(n_1,a_1)} \cdots \xrightarrow{\Delta_{m-1},(n_{m-1},a_{m-1})} (C_m, v_m)$$

with $v_i \in r_i$ for all i, proving completeness. □

Theorem 4.10. *Location-reachability is PSPACE-complete for synchronization-free LTNs.*

Proof. Let K denote the input size of $\mathcal{N}$, measured as the total encoding size of nodes, outcomes, clocks, and the binary encodings of constants in guards.

Upper bound. By Lemma 4.9, a location $\ell = (n, a)$ is reachable in $\mathcal{N}$ iff it is reachable in the finite graph underlying $\mathsf{ProdRegAut}(\mathcal{N})$. The number of markings is at most $2^{\mathcal{O}(|P|\cdot|N|)}$, hence $2^{\mathcal{O}(K)}$, and the number of product-regions is $2^{\mathcal{O}(K)}$ by Lemma 4.5. Therefore $\mathsf{ProdRegAut}(\mathcal{N})$ has at most $2^{\mathcal{O}(K)}$ states.

Although $\mathsf{ProdRegAut}(\mathcal{N})$ is exponentially large, each state has a polynomial-size encoding and successors can be generated on the fly in polynomial space. Hence we can decide reachability by a nondeterministic polynomial-space procedure: start at the initial state, and for at most 2^m steps (where m is the encoding length) nondeterministically guess a successor and verify it using the on-the-fly successor test; accept if a target state is reached. This uses only polynomial space (current state plus an m-bit step counter), so the problem is in NPSPACE, and therefore in PSPACE by Savitch's theorem [**?**].

Lower bound. PSPACE-hardness follows from reachability in timed automata [**?**]. A timed automaton can be encoded as an LTN with a single agent; with one agent, local time coincides with global time and $Sync = \emptyset$ vacuously. □ □

4.2 Always-synchronizing LTNs

4.2.1 Definition and Intuitive Behavior

Definition 4.11 (Always-synchronizing LTN)**.** An LTN $\mathcal{N}$ is *always-synchronizing* if $Sync = N$.

In this fragment, when location (n, a) is executed, all agents $p \in \mathrm{dom}(n)$ must have equal reference clocks: $t_p = t_q$ for all $p, q \in \mathrm{dom}(n)$. This essentially enforces a global timestamp for each interaction.

In the synchronization-free case, reference clocks were irrelevant. Here, they're central—but not in the way that caused undecidability. In general LTNs, synchronization is *selective*. Some nodes synchronize, others don't. This allows building gadgets where agents drift apart (at unsynchronized nodes), then compare drifts (at synchronized nodes), simulating counter tests. In always-synchronizing LTNs, synchronization is *global*. Since every node synchronizes, reference clocks stay aligned throughout. There's no opportunity to accumulate unbounded drift between synchronized interactions. This is essentially a return to global-time semantics.

Need for different technique. We cannot directly use the product-region construction from Section 4.1, because:

1. Reference clocks now appear in constraints (synchronization $v(t_p) = v(t_q)$),
2. Standard region abstractions for TA rely on clock values beyond M being safely abstractable until reset, but reference clocks are never reset.

Key insight: monotone witnesses. Timestamps are not monotone here. Despite of that, we can always reorder any run into one where interactions occur in increasing timestamp order. This "monotone-by-time" property enables a reduction to global-time timed automata.

4.2.2 Monotone runs

Definition 4.12 (Timestamp of a location and monotonicity)**.** Let $\mathcal{N}$ be always-synchronizing. Consider a run:

$$\rho = (C_0, v_0) \xrightarrow{\Delta_0,(n_0,a_0)} (C_1, v_1) \xrightarrow{\Delta_1,(n_1,a_1)} \cdots \xrightarrow{\Delta_{m-1},(n_{m-1},a_{m-1})} (C_m, v_m).$$

The timestamp of location (n_i, a_i) in ρ is:

$$\theta_i^\rho := (v_i + \Delta_i)(t_p) \quad \text{for any } p \in \mathrm{dom}(n_i).$$

This is well-defined because $n_i \in \text{Sync}$ implies all agents in $\text{dom}(n_i)$ have equal reference clocks at execution.

Run ρ is monotone if $\theta_0^\rho \leq \theta_1^\rho \leq \cdots \leq \theta_{m-1}^\rho$.

Intuition. Since every node synchronizes, each discrete step occurs at a moment when all participating agents agree on "what time it is". The timestamp records this agreed-upon moment. A monotone run executes locations in chronological order.

Example 4.12.1 (A non-monotone run in the running example, and its monotone rearrangement). We illustrate how non-monotone runs arise in always-synchronizing LTNs, using the running two-agent example introduced earlier.

Recall that the running example has agents p, q, a p-only node n_1, and a joint node n_2 with $dom(n_2) = \{p, q\}$. For symmetry, we additionally consider a q-only node n_1' with $dom(n_1') = \{q\}$ (this does not affect any reachability results, and merely exposes the symmetry between the agents).

Assume that all guards are trivial and that no local clocks are reset at n_1, n_1', or n_2; we focus only on the behavior of reference clocks.

A non-monotone run. Consider the following execution fragment:

- *Step 1 (private step of p).* Agent p delays for 8 time units, while q does not delay, and executes (n_1, a_1). Since the system is always-synchronizing, the timestamp of this step is
$$\theta_1 = t_p = 8.$$
- *Step 2 (private step of q).* Next, agent q delays for 7 time units, while p does not delay, and executes (n_1', a_1'). The timestamp of this step is
$$\theta_2 = t_q = 7.$$

Thus $\theta_1 > \theta_2$, and the run is non-monotone. Note that this does not contradict monotonicity of any single agent's reference clock: the inversion occurs between two actions whose domains are disjoint, since

$$dom(n_1) \cap dom(n_1') = \emptyset.$$

Reordering into a monotone run. Because (n_1, a_1) and (n_1', a_1') involve disjoint agents, their delays, guards, resets, and marking updates are independent and commute. We may therefore swap the two steps to obtain the following execution:

- First execute (n_1', a_1') at timestamp $\theta_2 = 7$.
- Then execute (n_1, a_1) at timestamp $\theta_1 = 8$.

This reordered run is monotone, since $7 \leq 8$.

Effect on subsequent synchronization. After either ordering of the two private steps, the reference-clock valuation is the same:

$$t_p = 8, \qquad t_q = 7.$$

Consequently, any subsequent execution of the joint node (n_2, a_2) behaves identically in both runs: the agent with the smaller reference clock is forced to wait until the clocks synchronize before (n_2, a_2) can occur.

This example concretely illustrates the key phenomenon used in the proof of Theorem **??**: inversions can arise only between actions with disjoint domains, and such inversions can always be eliminated by swapping independent steps.

Here $\theta_1 = 8 > \theta_2 = 7$, so the run is non-monotone. But since $\mathrm{dom}(n_1) \cap \mathrm{dom}(n_2) = \{p\} \neq \emptyset$... wait, actually this can't happen! If p participates in both, then t_p must increase, so $\theta_2 \geq \theta_1$. The key observation (proven below) is that inversions only occur between actions involving disjoint agents.

Lemma 4.13 (Inversions involve disjoint domains)**.** *Let ρ be a run of an always-synchronizing LTN. If $\theta_i^\rho > \theta_{i+1}^\rho$ for some i, then $dom(n_i) \cap dom(n_{i+1}) = \emptyset$.*

Proof. Suppose $p \in \mathrm{dom}(n_i) \cap \mathrm{dom}(n_{i+1})$. Reference clocks never reset and only increase via local delays. After step i, agent p has reference clock $(v_i + \Delta_i)(t_p) = \theta_i^\rho$. Before step $i+1$, p undergoes delay $\Delta_{i+1}(p) \geq 0$, reaching reference clock $(v_{i+1} + \Delta_{i+1})(t_p) = \theta_i^\rho + \Delta_{i+1}(p) \geq \theta_i^\rho$. But $(v_{i+1} + \Delta_{i+1})(t_p) = \theta_{i+1}^\rho$ (by definition of timestamp for step $i+1$), so $\theta_{i+1}^\rho \geq \theta_i^\rho$, contradicting $\theta_i^\rho > \theta_{i+1}^\rho$. □ □

Timeline intuition. In always-synchronizing LTNs, every interaction occurs at a moment when all participating agents agree on the current reference time. Thus, each executed location can be assigned a global timestamp.

Although different agents may perform independent actions at different times, any apparent reordering of timestamps can only arise between actions involving disjoint sets of agents. Such actions are causally independent: their guards, resets, and marking updates affect disjoint components of the system.

As a result, any run can be rearranged—by repeatedly swapping adjacent independent steps—into a run where timestamps are non-decreasing. This monotone form reveals an underlying global-time structure, enabling a reduction to timed automata.

Lemma 4.14 (Existence of monotone witnesses)**.** *For every location (n, a) that is reachable in an always-synchronizing LTN $\mathcal{N}$, there exists a* monotone *run of $\mathcal{N}$ that executes (n, a).*

Proof of Lemma 4.14. Fix any run ρ of $\mathcal{N}$ that executes the target location (n, a).

Write the discrete steps of ρ as

$$(C_0, v_0) \xrightarrow{\Delta_0} (C_0, w_0) \xrightarrow{\ell_0} (C_1, v_1) \xrightarrow{\Delta_1} (C_1, w_1) \xrightarrow{\ell_1} \cdots \xrightarrow{\Delta_{m-1}} (C_{m-1}, w_{m-1}) \xrightarrow{\ell_{m-1}} (C_m, v_m),$$

where $w_i := v_i + \Delta_i$ and $\ell_i = (n_i, a_i)$. For each i, let $D_i := dom(n_i)$ and let $\theta_i := \theta_i^\rho$ be the timestamp of ℓ_i (Definition **??**), i.e. $\theta_i = w_i(t_p)$ for any $p \in D_i$.

Step 1: A normal form for delays ("tight" runs). We first transform ρ into a run $\widehat{\rho}$ with the *same* sequence $\ell_0, \ldots, \ell_{m-1}$ of discrete steps, such that for every i,

$$\widehat{\Delta}_i(p) = 0 \quad \text{whenever } p \notin D_i,$$

i.e. the delay before ℓ_i is applied only by agents that actually participate in ℓ_i.

This is done by repeatedly pushing *irrelevant* delay components to the right: suppose for some i and some agent $p \notin D_i$ we have $\Delta_i(p) = \delta > 0$. Since p does not participate in ℓ_i, the enabling of ℓ_i and all its guards/resets are independent of the clocks of p. Moreover, reference clocks are never reset, and local clocks of p are neither tested nor reset by ℓ_i (as $p \notin D_i$). Hence we can replace the pair of consecutive delay vectors (Δ_i, Δ_{i+1}) by

$$\Delta'_i := \Delta_i - \delta \cdot \mathbf{e}_p, \qquad \Delta'_{i+1} := \Delta_{i+1} + \delta \cdot \mathbf{e}_p,$$

(where $\mathbf{e}_p$ is the unit vector for agent p), and obtain another valid run with the same discrete step sequence. Intuitively, we simply postpone a portion of p's waiting time across an action that does not involve p. This preserves all clock values for all agents at every discrete step in which they participate, hence preserves satisfaction of all guards and yields the same marking evolution.

Iterating this operation finitely many times (pushing each irrelevant component to the next delay until it reaches a step where that agent participates, or the end) yields a run $\widehat{\rho}$ in the desired normal form. Importantly, timestamps of steps are unchanged by this normalization because θ_i depends only on agents in D_i and we never move delay components of agents in D_i away from the position immediately preceding ℓ_i.

From now on, replace ρ by this normalized run and keep the same notation.

Step 2: Inversions can only happen between disjoint domains. We claim that if $\theta_i > \theta_{i+1}$, then $D_i \cap D_{i+1} = \emptyset$. Indeed, if $p \in D_i \cap D_{i+1}$, then p participates in both steps. Reference clocks are never reset and only increase by nonnegative local delays. Therefore $w_{i+1}(t_p) \geq w_i(t_p)$. But $w_i(t_p) = \theta_i$ and $w_{i+1}(t_p) = \theta_{i+1}$ by definition of timestamp, contradicting $\theta_i > \theta_{i+1}$.

Step 3: Swapping an adjacent inversion. Assume there is an index i with $\theta_i > \theta_{i+1}$. By Step 2, $D_i \cap D_{i+1} = \emptyset$. Since we are in the normal form, Δ_i has support included in D_i and Δ_{i+1} has support included in D_{i+1}. Consider the local

fragment

$$(C_i, v_i) \xrightarrow{\Delta_i} (C_i, w_i) \xrightarrow{\ell_i} (C_{i+1}, v_{i+1}) \xrightarrow{\Delta_{i+1}} (C_{i+1}, w_{i+1}) \xrightarrow{\ell_{i+1}} (C_{i+2}, v_{i+2}).$$

Because D_i and D_{i+1} are disjoint:

- executing ℓ_i does not read/reset/update clocks of agents in D_{i+1}, and executing ℓ_{i+1} does not read/reset/update clocks of agents in D_i;
- the marking update of ℓ_i changes only the components $C(p)$ for $p \in D_i$, and the marking update of ℓ_{i+1} changes only the components $C(p)$ for $p \in D_{i+1}$, hence the two marking updates commute;
- the delay vector Δ_i affects only agents in D_i, and Δ_{i+1} affects only agents in D_{i+1}, hence the two delays commute and each can be moved across the other's discrete action without affecting its guards.

Therefore we can swap the two *blocks* (Δ_i, ℓ_i) and $(\Delta_{i+1}, \ell_{i+1})$ to obtain another valid run fragment

$$(C_i, v_i) \xrightarrow{\Delta_{i+1}} (C_i, \widetilde{w}_{i+1}) \xrightarrow{\ell_{i+1}} (C'_{i+1}, \widetilde{v}_{i+1}) \xrightarrow{\Delta_i} (C'_{i+1}, \widetilde{w}_i) \xrightarrow{\ell_i} (C_{i+2}, v_{i+2}),$$

that reaches the *same* configuration (C_{i+2}, v_{i+2}) as before. Moreover, the timestamp of ℓ_{i+1} in the swapped fragment remains θ_{i+1} (since the participants in D_{i+1} experience exactly the same total delay before executing ℓ_{i+1}), and similarly the timestamp of ℓ_i remains θ_i. Hence the adjacent inversion at positions $i, i+1$ is eliminated.

Step 4: Bubble-sort to obtain monotonicity. Define the number of *adjacent inversions* of the timestamp sequence $\theta_0, \ldots, \theta_{m-1}$ as

$$Inv(\rho) := \big|\{\, i \mid 0 \le i < m-1,\ \theta_i > \theta_{i+1} \,\}\big|.$$

If $Inv(\rho) = 0$, the run is monotone and we are done. Otherwise, Step 3 produces a new run with the same set of discrete steps (in particular still executing (n, a)) and strictly smaller $Inv(\cdot)$, because we remove at least the chosen adjacent inversion and do not create any new inversion involving steps whose domains overlap (Step 2 prevents that). Since $Inv(\rho)$ is a natural number, iterating this operation finitely many times yields a run with $Inv = 0$, i.e. a monotone run that executes (n, a).

This proves the lemma. □

Why this helps. Monotone runs have a global-time interpretation: locations occur in chronological order. This allows us to simulate the entire LTN with a single timed automaton using global time.

4.2.3 Reduction to timed automata

We now construct a timed automaton that recognizes exactly the monotone runs of $\mathcal{N}$ (projected to locations). Intuitively, the always-synchronizing constraint forces all participating agents to act at the same timestamp, which corresponds to the global-time view of a single timed automaton.

Definition 4.15 (Timed automaton of an always-synchronizing LTN)**.** Let $\mathcal{N}$ be always-synchronizing. Define a timed automaton $\mathsf{TA}(\mathcal{N})$ as follows:

- States are all markings of $\mathcal{N}$; write Q for this set.
- The alphabet is the set of locations: $\Sigma_{\mathsf{TA}} := N \times \Sigma$.
- The clock set is the same local-clock set X (reference clocks are not needed).
- The initial state is the initial marking C_0.
- Transitions are defined as follows. There is a transition

$$(C,\, (n, a),\, g,\, Y,\, C')$$

  iff:

  1. n is enabled in C, i.e. $n \in C(p)$ for all $p \in dom(n)$;
  2. for each $p \in dom(n)$, $\delta_p(n, a) = (g_p, M_p, Y_p)$, and $g := \bigwedge_{p \in dom(n)} g_p$ and $Y := \bigcup_{p \in dom(n)} Y_p$;
  3. $C'(p) = M_p$ for $p \in dom(n)$ and $C'(p) = C(p)$ for $p \notin dom(n)$.

Intuition. The TA simulates the LTN in "global time." Each LTN node becomes a TA transition. Since nodes synchronize, all participating agents act at the same global timestamp—which is exactly what TA semantics provides. Local clocks X_p advance with global time, just as they advanced with (synchronized) reference times in the LTN.

Lemma 4.16 (Correctness of the reduction)**.** *Let $\mathcal{N}$ be always-synchronizing. A location (n, a) is reachable in $\mathcal{N}$ iff $\mathsf{TA}(\mathcal{N})$ has a run that executes a transition labeled by (n, a).*

Proof of Lemma 4.16. We show both directions.

($\Rightarrow$) If (n, a) is reachable in $\mathcal{N}$, then $\mathsf{TA}(\mathcal{N})$ executes (n, a). Assume (n, a) is reachable in $\mathcal{N}$. By Lemma 4.14, there exists a *monotone* run ρ of $\mathcal{N}$ that executes (n, a). Write ρ as in Definition **??**, with discrete steps $\ell_i = (n_i, a_i)$ and timestamps

$$\theta_0 \leq \theta_1 \leq \cdots \leq \theta_{m-1}.$$

Let $\theta_{-1} := 0$ and define global delays

$$d_i := \theta_i - \theta_{i-1} \quad \text{for } i = 0, \dots, m-1.$$

Monotonicity ensures $d_i \geq 0$.

We now construct a run of $\mathsf{TA}(\mathcal{N})$ that takes the same sequence of labels $\ell_0, \dots, \ell_{m-1}$ with those global delays. Let u_0 be the all-zero valuation of the TA clocks X. Inductively, suppose we are at TA state C_i with valuation u_i. We let time elapse by d_i, reaching valuation $u_i + d_i$. We claim that the transition of $\mathsf{TA}(\mathcal{N})$ labeled $\ell_i = (n_i, a_i)$ is enabled at $(C_i, u_i + d_i)$, and taking it yields (C_{i+1}, u_{i+1}).

To see this, first note that C_i in the TA is the same marking as in the LTN run, hence n_i is enabled in C_i (Definition 4.15(2.1)). Next, let the local rule for each $p \in D_i := dom(n_i)$ be $\delta_p(n_i, a_i) = (g_p, M_p, Y_p)$, and let $g = \bigwedge_{p \in D_i} g_p$ and $Y = \bigcup_{p \in D_i} Y_p$ as in Definition 4.15(2.2).

The key observation is that in a monotone run, the value of any local clock $x \in X_p$ at the execution of ℓ_i depends only on the timestamp θ_i and the most recent timestamp at which x was reset (by an earlier discrete step involving p). Indeed, local clocks advance exactly with agent p's local time (reference time) and are reset only when p participates; in particular, between the last reset time θ_j of x and θ_i, the elapsed local time of p is exactly $\theta_i - \theta_j$, so x has value $\theta_i - \theta_j$.

In the timed automaton run we are constructing, clocks in X advance with the global time, and the reset set on label ℓ_j is exactly the union of the LTN resets at that step. Therefore, by induction on i, for every agent p and every clock $x \in X_p$, the value of x in the TA valuation just before taking ℓ_i is *exactly the same* as the value of x in the LTN run at timestamp θ_i (both equal $\theta_i - \theta_j$ where θ_j is the last reset timestamp of x).

Since each guard g_p uses only clocks in X_p, it follows that the LTN satisfaction $w_i \models g_p$ (which holds because ρ executes ℓ_i) implies $(u_i + d_i) \models g_p$. Hence $(u_i + d_i) \models g$ and the TA transition labeled ℓ_i is enabled. Taking it updates the marking to C_{i+1} as in Definition 4.15(2.3) and resets exactly the clocks in Y, matching the LTN reset effect. This defines u_{i+1} and maintains the induction invariant.

Thus $\mathsf{TA}(\mathcal{N})$ has a run that executes $\ell_k = (n, a)$ for the appropriate index k.

($\Leftarrow$) If $\mathsf{TA}(\mathcal{N})$ executes (n, a), then (n, a) is reachable in $\mathcal{N}$. Assume $\mathsf{TA}(\mathcal{N})$ has a run

$$(C_0, u_0) \xrightarrow{d_0} (C_0, u_0 + d_0) \xrightarrow{\ell_0} (C_1, u_1) \xrightarrow{d_1} \cdots \xrightarrow{d_{m-1}} (C_{m-1}, u_{m-1} + d_{m-1}) \xrightarrow{\ell_{m-1}} (C_m, u_m),$$

where some $\ell_k = (n, a)$ occurs. We build a run of $\mathcal{N}$ that executes the same discrete steps in the same order, using *uniform* local delays. For each i, define the local delay vector $\Delta_i \in \mathbb{R}_{\geq 0}^P$ by $\Delta_i(p) := d_i$ for all agents $p \in P$.

Start from the LTN initial configuration (C_0, v_0) where all local clocks in X are 0 and all reference clocks t_p are 0. At step i, apply the local delay Δ_i. This

advances every agent's reference clock by the *same* amount d_i, hence for every node (and in particular for n_i) the synchronization constraint $t_p = t_q$ over $p, q \in dom(n_i)$ is satisfied at the moment of execution. Moreover, since local clocks in X advance at the same rate as reference time for each agent, after the delay Δ_i the valuation of all clocks in X coincides with the TA valuation after global delay d_i.

Now consider the discrete step $\ell_i = (n_i, a_i)$. By construction of $\mathsf{TA}(\mathcal{N})$, the TA guard on ℓ_i is the conjunction of the participating agents' guards. Since the TA transition is enabled, this conjunction holds; hence each individual local guard g_p holds for $p \in dom(n_i)$ in the LTN valuation as well. Therefore ℓ_i is enabled in $\mathcal{N}$ and can be executed, producing exactly the same marking update as in the TA transition definition and resetting exactly the same set of clocks in X.

Iterating this for all i yields an LTN run that executes $\ell_k = (n, a)$. Hence (n, a) is reachable in $\mathcal{N}$.

Combining both directions proves the lemma. □

Theorem 4.17. *Location-reachability is PSPACE-complete for always-synchronizing LTNs.*

Proof. By Lemma 4.16, it suffices to decide reachability of a transition labeled (n, a) in $\mathsf{TA}(\mathcal{N})$.

Let K be the input size of $\mathcal{N}$ (as in Theorem 4.10). The number of markings is at most $2^{\mathcal{O}(K)}$, hence $|Q| = 2^{\mathcal{O}(K)}$. The clock set is X, and maximal constants are bounded by those in $\mathcal{N}$. Therefore the region graph of $\mathsf{TA}(\mathcal{N})$ has at most $|Q| \cdot 2^{\mathcal{O}(K)} = 2^{\mathcal{O}(K)}$ nodes. Reachability in this exponentially large region graph can be checked in polynomial space via on-the-fly exploration, yielding the PSPACE upper bound.

PSPACE-hardness again follows from timed automata reachability: a timed automaton is an LTN with a single agent, and in that case $Sync = N$ holds vacuously. □ □

4.3 Summary and perspective

We have established PSPACE-completeness for both synchronization-free and always-synchronizing LTNs (Theorems 4.10 and 4.17). Both match the complexity of timed automata, but via fundamentally different proof techniques. This section synthesizes the main lessons.

4.3.1 Comparing the Two Fragments

Property	**Sync-Free**	**Always-Sync**
Synchronization	None (Sync = $\emptyset$)	Universal (Sync = N)
Reference clocks	Irrelevant (never tested)	Central (globally aligned)
Key structure	Clocks independent per agent	Global timeline emerges
Abstraction	Product of per-agent regions	Monotone witness runs
Technique	Componentwise quotient	Reduction to global time
Complexity	PSPACE-complete	PSPACE-complete
Expressiveness	Local constraints only	Enforces temporal alignment

4.3.2 Why Both Are PSPACE (Not Lower, Not Higher)

Why not lower than PSPACE? Both fragments inherit the essential difficulty of timed automata: reasoning about continuous time with finite representations. The region/zone abstractions yield exponentially many equivalence classes, and reachability in exponentially large graphs is PSPACE-complete. The concurrency structure of negotiations (multiple agents, complex control flow) doesn't reduce this complexity—it just changes how we build the abstraction.

Why not higher than PSPACE (avoiding undecidability)? Sync-free case: Reference clocks never interact. Each agent's timing is independent, preventing any mechanism to encode or compare unbounded values. The product abstraction succeeds because there's no cross-agent timing dependency.

Always-sync case: Reference clocks stay globally aligned. While agents can "wait" for each other, they can't accumulate unbounded relative drift that persists across interactions. Every interaction resets the playing field—all participating agents synchronize. The monotone-witness property shows this is equivalent to global time, where standard TA techniques apply.

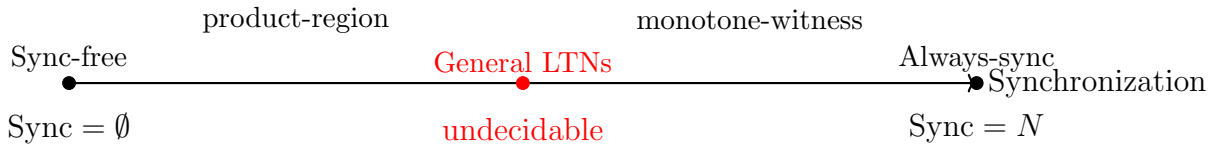
What's missing for undecidability? Both fragments avoid the dangerous combination:

- **Unbounded drift + selective synchronization**

In sync-free LTNs, there's no synchronization to test accumulated drift. In always-sync LTNs, there's no opportunity to accumulate unbounded drift between synchronized interactions. General LTNs allow both, enabling counter-machine simulations.

4.3.3 The Design Space

These extremes bracket a spectrum of possible restrictions:



Intermediate fragments. What about LTNs where some nodes synchronize and others don't ($\emptyset \subset \text{Sync} \subset N$)? Two approaches:

1. Quantitative restrictions (Chapter ??): Bound how much drift can accumulate, even with selective synchronization. If differences $t_p - t_q$ stay bounded, finite abstractions may still exist.

2. Structural restrictions (Chapter ??): Require that control-flow structure prevents "problematic" patterns. For instance, demand that every cycle includes enough synchronization to prevent unbounded drift accumulation.

4.4 Coming next

This chapter established PSPACE-completeness at the two extremes of synchronization: when synchronization is absent, timing constraints remain purely local and admit a product-region abstraction; when every node synchronizes, every reachable location has a monotone witness run and reachability reduces to timed-automata reachability.

In the next chapter we move to a genuinely intermediate regime: the *bounded-drift* fragment, where the deviation between agents' reference times is globally bounded. We show how this quantitative restriction can be exploited to recover a finite abstraction for reachability, and we analyze the resulting complexity.

Chapter 5

Drift-Bounded and Acyclic Local-Timed Negotiations

5.1 Introduction: From Synchronization to Drift

The previous chapters established a sharp dichotomy in the decidability landscape of local-timed negotiations based on synchronization patterns:

- **Chapter 3**: General LTNs have undecidable reachability. The undecidability proof encodes counter values as *differences between reference clocks*, exploiting unbounded drift combined with selective synchronization to test and manipulate these differences.
- **Chapter 4**: Both synchronization extremes—synchronization-free (Sync $= \emptyset$) and always-synchronizing (Sync $= N$)—yield PSPACE-complete reachability. The synchronization-free fragment avoids drift accumulation by preventing clock comparisons across agents, while the always-synchronizing fragment enforces tight coupling that precludes the selective testing needed for counter simulation.

These results point to a natural question: rather than restricting synchronization patterns syntactically, can we instead control the *semantic consequence* that makes undecidability possible—namely, unbounded divergence between reference clocks? If reference clocks can diverge arbitrarily, they can encode unbounded counters. But if their divergence remains bounded by some constant k along all executions, then reference clock differences range over a finite domain $[-k, k]$, potentially enabling finite abstraction techniques analogous to region constructions for timed automata.

This chapter develops this intuition systematically, introducing *drift-bounded LTNs* as a semantic restriction that cuts across synchronization patterns. The key insight is that boundedness of drift—the maximum difference between any

two agents' reference clocks—can restore decidability even in the presence of partial synchronization, provided every discrete behavior admits at least one timed realization with bounded drift.

Chapter Overview and Contributions

This chapter makes the following contributions:

Semantic restriction: k-drift boundedness. We formalize drift as the maximum pairwise difference between reference clocks and introduce an *existential* notion of k-drift boundedness (Definition 5.5): an LTN is k-drift bounded if every discrete run admits at least one timed realization in which drift never exceeds k. This retiming-based notion, standard in the timed automata literature, focuses on time-abstract behaviors and enables reasoning about reachability via finite abstractions.

Finite abstraction and decidability. For k-drift bounded LTNs, we adapt region-equivalence techniques to construct a finite k-region automaton that preserves reachability (Section 5.3.1). This yields our first main result:

Theorem 5.1 (Decidability for drift-bounded LTNs)**.** *For each fixed $k \in \mathbb{N}$, location reachability for k-drift bounded LTNs is PSPACE-complete.*

Structural implication: acyclic LTNs. We then investigate when drift boundedness can be guaranteed structurally. We show that *acyclicity*—the absence of cycles in the marking graph—implicitly enforces bounded drift (Corollary **??**), even though acyclicity imposes no explicit constraints on timing or clock values. For acyclic LTNs, reachability becomes significantly simpler:

Theorem 5.2 (Complexity for acyclic LTNs)**.** *Location reachability for acyclic local-timed negotiations is NP-complete.*

The reduction from NP-completeness (for acyclic) to PSPACE-completeness (for general k-drift bounded) reflects the difference between systems where boundedness is structurally guaranteed versus those where it must be verified semantically.

Meta-complexity. We also investigate the complexity of verifying drift boundedness itself, showing that checking whether an LTN satisfies a universal drift bound is PSPACE-complete, while determining the existence of such a bound is undecidable (Section 5.4).

Significance and Perspective

Drift-bounded LTNs represent a shift from *syntactic* restrictions (forbidding or mandating synchronization) to *semantic* restrictions (bounding divergence in reachable behaviors). This perspective has several advantages:

- **Unification:** Drift boundedness subsumes and explains the decidability of both synchronization extremes studied in Chapter 4. Synchronization-free LTNs are trivially drift-bounded because independent clocks never need to converge; always-synchronizing LTNs enforce bounded drift through constant resynchronization.
- **Generality:** Unlike synchronization-based restrictions, drift boundedness applies to systems with *partial* synchronization—the most common case in practice, where some but not all interactions require coordination.
- **Connection to structure:** The acyclicity result demonstrates that semantic properties (bounded drift) can arise from purely structural constraints (absence of control cycles), providing a bridge between syntax and semantics.

The remainder of this chapter is organized as follows. Section 5.2 formalizes drift and drift-bounded runs, with geometric intuition and concrete examples. Section 5.3 introduces the k-drift bounded fragment and establishes decidability via region abstraction. Section 5.5 studies acyclic LTNs as a structurally drift-bounded class and proves NP-completeness. Section 5.4 investigates the decidability of drift boundedness verification. Section 5.6 summarizes the chapter and previews the structural communication restrictions studied in Chapter **??**.

5.2 Drift: Definition, Intuition, and Examples

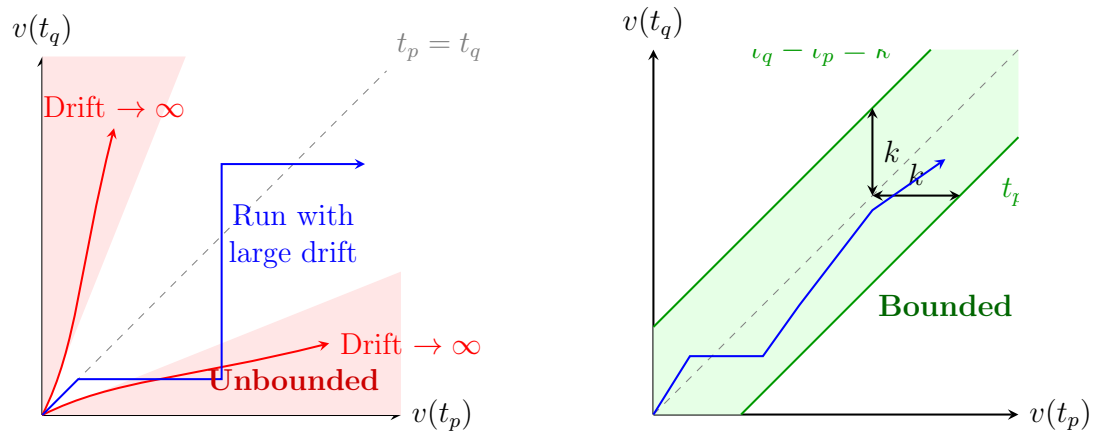
In local-timed negotiations, each agent maintains a local reference clock that tracks its cumulative elapsed time. These clocks evolve independently during local delays and can therefore diverge. The extent of this divergence is captured by the notion of *drift*.

Definition 5.3 (Drift)**.** Given a valuation v over clocks including reference clocks $\{t_p\}_{p \in P}$, the *drift* of v is

$$\mathsf{drift}(v) := \max_{p,q \in P} |v(t_p) - v(t_q)|.$$

A configuration (C, v) is *k-drift bounded*, for $k \in \mathbb{N}$, if $\mathsf{drift}(v) \leq k$.

Intuitively, drift measures how far apart agents' reference times can be at any given configuration. When drift is unbounded, the differences $v(t_p) - v(t_q)$ can encode arbitrary integers, enabling counter simulation and leading to undecidability (as demonstrated in Chapter 3).



(A) **Unbounded Drift.** The difference $|v(t_p) - v(t_q)|$ can grow arbitrarily. Reachable states diverge far from the diagonal $t_p = t_q$, requiring infinite state space to track all possibilities.

(B) **Bounded Drift (k-bounded).** All reachable configurations satisfy $|v(t_p) - v(t_q)| \leq k$. States are confined to a diagonal band of width $2k$, enabling finite region abstraction.

FIGURE 5.1: Geometric interpretation of drift in the space of reference clocks. The axes represent reference-clock values $v(t_p)$ and $v(t_q)$ for two agents. Although both values grow unboundedly over time, k-drift boundedness constrains their relative positions.

5.2.1 Geometric Intuition

Figure 5.1 visualizes drift in a two-agent system as a trajectory in the $(v(t_p), v(t_q))$ plane. The diagonal $v(t_p) = v(t_q)$ represents perfect synchronization.

Unbounded drift (left). Without restrictions, executions may diverge arbitrarily far from the diagonal. The reachable region is potentially the entire quadrant, yielding an unbounded two-dimensional state space. This illustrates why standard region abstractions fail: tracking unbounded clock differences requires infinitely many regions, and these differences can encode counter values.

Bounded drift (right). Under a drift bound $|v(t_p) - v(t_q)| \leq k$, reachable configurations are constrained to an infinite strip of fixed width around the diagonal. Although time itself grows unboundedly, the *relative positions* of reference clocks remain bounded. Crucially, this strip can be collapsed into a finite abstraction by quotienting time elapse, which is precisely what the k-region construction exploits (Section 5.3.1).

5.2.2 Motivating Examples

To make drift boundedness concrete, we present two examples that illustrate the difference between unbounded and bounded drift at the level of simple control structures.

Example 5.3.1 (Unbounded vs. bounded drift). Figure 5.2 shows two LTNs over agents p and q that differ only in their timing constraints and synchronization patterns.

LTN $\mathcal{N}_1$ (left): Unbounded drift. Nodes n_1 and n_2 both have self-loops labeled a and b respectively, each with guard $x = 1$ (or $y = 1$) and reset set $\{x\}$ (or $\{y\}$). For any $k \geq 0$, agent p can execute the a-loop on n_1 exactly $k+1$ times while agent q does not execute its b-loop at all. At the resulting configuration, node n_3 becomes enabled, but the reference-clock difference satisfies

$$|v(t_p) - v(t_q)| \;\geq\; k.$$

Hence, the drift can grow arbitrarily large along reachable configurations.

Moreover, this drift cannot be reduced afterward. If agent q allows more than one unit of local time to elapse while agent p continues executing a-loops, then q will violate the guard of its b-loop and can no longer perform it. Consequently, any run realizing the word $a^{k+1}b^{k+1}$ necessarily maintains drift at least k throughout its execution, demonstrating that unbounded drift is intrinsic to the system's behavior.

LTN $\mathcal{N}_2$ (right): Bounded drift. The structure is similar, but $\mathcal{N}_2$ enforces bounded drift through the time-synchronizing node n_3 (shown with a dotted line, indicating synchronization). Whenever n_3 is executed, the reference clocks of p and q are synchronized to the same value, so $\mathsf{drift}(v) = 0$ at that point. After synchronization, agent p must wait at least 2 time units before executing a (guard $x \geq 2$), while agent q must execute b within 1 time unit (guard $y \leq 1$). This creates a maximum drift of $2 - 1 = 1$.

Repeating this synchronization pattern ensures that all reachable configurations of $\mathcal{N}_2$ satisfy $\mathsf{drift}(v) \leq 1$, regardless of how many times the agents cycle through their behaviors.

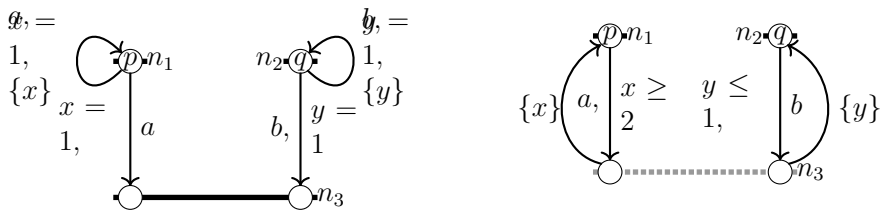


FIGURE 5.2: LTN $\mathcal{N}_1$ with unbounded drift (left) and 1-drift bounded LTN $\mathcal{N}_2$ (right). The dotted line in $\mathcal{N}_2$ indicates a synchronizing interaction.

Example 5.3.2 (ATM transaction with bounded drift). Consider the ATM transaction LTN from Figure 3.1 (Chapter 3) with three agents: customer c, ATM a,

and bank b. The negotiation exhibits partial synchronization—some interactions involve all three agents, while others involve only pairs.

Claim: This LTN is 10-drift bounded (assuming appropriate guard constants).

Intuition:

- The customer c can delay at most 10 units between interactions (guard $x \leq 10$ at outcome e_otp).
- The bank b can delay at most 3 units between interactions (guards $y \leq 3$ at match and fail).
- The ATM a coordinates both, so its reference clock stays between t_c and t_b.
- Maximum drift occurs when c is at $t_c = 10$ and b hasn't started ($t_b = 0$), giving drift $= 10$.
- Subsequent interactions force resynchronization, preventing unbounded divergence.

This example illustrates that drift boundedness is compatible with partial synchronization and does not require the extreme restrictions of synchronization-free or always-synchronizing LTNs.

5.2.3 Drift-Bounded Runs and LTNs

We now formalize drift boundedness for runs and systems.

Definition 5.4 (k-drift-bounded run)**.** A run

$$\rho = (C_1, v_1) \xrightarrow{(\Delta_1, e_1)} (C_2, v_2) \xrightarrow{(\Delta_2, e_2)} \cdots$$

is *k-drift bounded* if every configuration (C_i, v_i) occurring in the run satisfies $\mathsf{drift}(v_i) \leq k$.

The central notion studied in this chapter is an *existential*, *retiming-based* boundedness condition, standard in the timed-automata literature.

Definition 5.5 (k-drift-bounded LTN)**.** An LTN $\mathcal{N}$ is *k-drift bounded* if for every run

$$\rho = (C_1, v_1) \xrightarrow{\Delta_1, e_1} (C_2, v_2) \xrightarrow{\Delta_2, e_2} \cdots$$

of $\mathcal{N}$, there exists a run

$$\rho_k = (C_1, v_1) \xrightarrow{\Delta'_1, e_1} (C_2, v'_2) \xrightarrow{\Delta'_2, e_2} \cdots$$

with the same sequence of discrete outcomes $e_1, e_2, \ldots$, obtained by modifying only the local delays, such that ρ_k is k-drift bounded.

Thus, an LTN is k-drift bounded if every discrete behavior (untimed sequence of outcomes) admits at least one timed realization in which drift never exceeds k.

Existential vs. universal drift boundedness. Definition 5.5 uses an *existential* notion: we require that *some* timed realization of each discrete behavior is drift-bounded, not that *all* realizations are. This is the appropriate notion for reasoning about time-abstract behaviors via retiming and aligns with standard practice in timed automata verification.

In contrast, a *universal* notion would require *every reachable configuration* to satisfy the drift bound. Universal drift boundedness is stronger and less commonly satisfied, but it is also a natural property to verify (see Section 5.4). Throughout this chapter, "drift-bounded" without qualification means existentially drift-bounded.

5.3 The k-Drift Bounded Fragment

We now establish decidability and complexity for the class of k-drift bounded LTNs. The key insight is that bounded drift enables a finite abstraction analogous to region constructions for timed automata.

5.3.1 Region Abstraction Under Bounded Drift

Fix a constant $k \in \mathbb{N}$ and let M be the maximum constant appearing in any clock constraint of $\mathcal{N}$. We adapt the region equivalence from networks of timed automata [**?**] to LTNs.

Definition 5.6 (k-Drift region equivalence $\cong_M^k$)**.** Let M be the maximal constant in guards and k the drift bound. Two k-drift bounded valuations v, v' are equivalent, written $v \cong_M^k v'$, if:

1. **Integral parts of local clocks:** For all $x \in \bigcup_{p \in P} X_p$, either $\lfloor v(x) \rfloor = \lfloor v'(x) \rfloor$ or both exceed M.

2. **Fractional ordering:** For all clocks $x, y \in \bigcup_{p \in P}(X_p \cup \{t_p\})$, we have $\{v(x)\} \leq \{v(y)\}$ if and only if $\{v'(x)\} \leq \{v'(y)\}$, and equality of fractional parts is preserved: $\{v(x)\} = \{v(y)\}$ iff $\{v'(x)\} = \{v'(y)\}$.

3. **Reference clock differences:** For all agents $p, q \in P$, we have $\lfloor v(t_p) - v(t_q) \rfloor = \lfloor v'(t_p) - v'(t_q) \rfloor$.

Although LTNs use explicit local clocks rather than the offset representation common in timed automata, the equivalence $\cong_M^k$ captures the same essential information:

- Condition (1) abstracts local clock values by their integer parts up to M, beyond which guards cease to distinguish values.
- Condition (2) preserves the ordering of fractional parts, which determines the order in which clocks become integral and thus which guards become enabled.
- Condition (3) tracks integer differences between reference clocks, which are bounded by k under drift boundedness and determine synchronization constraints.

The key property of $\cong_M^k$ is that it is a *time-abstract bisimulation* on the space of k-drift-bounded configurations: if $(C, v) \cong_M^k (C, v')$, then

1. For every local delay Δ from (C, v), there exists a delay Δ' from (C, v') such that $(C, v + \Delta) \cong_M^k (C, v' + \Delta')$.
2. v satisfies a guard with constants at most M if and only if v' does.
3. For every synchronizing outcome, the equality constraints $v(t_p) = v(t_q)$ for participants p, q are preserved under equivalent valuations.

Consequently, for every k-drift-bounded run starting from (C, v), there exists a k-drift-bounded run with the same discrete behavior starting from any (C, v') with $(C, v) \cong_M^k (C, v')$.

Lemma 5.7 (Finiteness of $\cong_M^k$)**.** *The equivalence $\cong_M^k$ has finitely many equivalence classes. Specifically, the number of regions is bounded by*

$$O\big(|N| \cdot 2^{|X|} \cdot |X|! \cdot (2M + 1)^{|X|} \cdot (2k + 1)^{|P|^2}\big),$$

where $|N|$ is the number of nodes, $|X| = \sum_{p \in P} |X_p|$ is the total number of local clocks, and $|P|$ is the number of agents.

Proof. Each region is determined by:

- A marking C (at most $|N|$ possibilities).
- Integer parts of local clocks up to M (at most $(M+2)^{|X|}$ possibilities, accounting for values $> M$).
- An ordering of fractional parts among $|X| + |P|$ clocks (at most $(|X| + |P|)!$ orderings, and $2^{|X|+|P|}$ to record which are zero).
- Integer differences $\lfloor t_p - t_q \rfloor$ for all pairs, bounded by k (at most $(2k + 1)^{|P|^2}$ combinations).

The product of these bounds yields an exponential dependence on $|X|$ and $|P|$, but polynomial in M and k when these are treated as parameters. □

5.3.2 The k-Region Automaton

Using equivalence classes of $\cong_M^k$ as states, we construct a finite *k-region automaton* $\mathcal{R}_k(\mathcal{N})$ whose transitions abstract both time elapse and discrete outcomes.

Definition 5.8 (k-Region automaton)**.** The k-region automaton $\mathcal{R}_k(\mathcal{N})$ is a finite automaton with:

- **States:** Equivalence classes $[C, v]_{\cong_M^k}$ of k-drift bounded configurations.
- **Initial state:** $[C_0, v_0]_{\cong_M^k}$ where (C_0, v_0) is the initial configuration.
- **Transitions:** $[C, v]_{\cong_M^k} \xrightarrow{e} [C', v']_{\cong_M^k}$ if there exist representatives (C, v) and delays Δ, Δ' such that $(C, v + \Delta) \xrightarrow{e} (C', v' + \Delta')$ and both configurations are k-drift bounded.

The time-abstract bisimulation property ensures that reachability in $\mathcal{R}_k(\mathcal{N})$ faithfully represents reachability of discrete behaviors in the original LTN.

Theorem 5.9 (Soundness and completeness of $\mathcal{R}_k(\mathcal{N})$)**.** *A marking C is reachable in $\mathcal{N}$ via a k-drift bounded run if and only if some region $[C, v]_{\cong_M^k}$ is reachable in $\mathcal{R}_k(\mathcal{N})$.*

Proof sketch. Soundness: If $[C, v]_{\cong_M^k}$ is reachable in $\mathcal{R}_k(\mathcal{N})$, then by definition of transitions, there exists a corresponding k-drift bounded run in $\mathcal{N}$ reaching (C, v).

Completeness: If C is reachable via a k-drift bounded run ρ in $\mathcal{N}$, then by the bisimulation property, there exists an equivalent run through region representatives that witnesses reachability of $[C, v]_{\cong_M^k}$ in $\mathcal{R}_k(\mathcal{N})$. □

5.3.3 Decidability and Complexity

We now establish the main complexity result for k-drift bounded LTNs.

Theorem 5.10 (Main result for drift-bounded LTNs)**.** *For each fixed $k \in \mathbb{N}$, location reachability is PSPACE-complete for k-drift bounded LTNs.*

Proof. **PSPACE Upper Bound:** By Lemma 5.7, the k-region automaton has exponentially many states (in $|X|$ and $|P|$, polynomial in M and k). Reachability in this finite automaton can be solved by a nondeterministic algorithm that guesses a path, storing only the current region at each step. This requires polynomial space: $O(\log(|N| \cdot |X|! \cdot M^{|X|} \cdot k^{|P|^2}))$, which is polynomial in the size of the LTN encoding. By Savitch's theorem, NPSPACE = PSPACE, yielding the upper bound.

PSPACE Lower Bound: A standard timed automaton $\mathcal{A}$ can be viewed as a single-agent LTN $\mathcal{N}_\mathcal{A}$. With only one agent, drift is trivially 0 (bounded). Location reachability for timed automata is PSPACE-complete [**?**], so reachability in k-drift bounded LTNs is PSPACE-hard. □

This result positions drift-bounded LTNs at the same complexity level as timed automata, despite their greater structural flexibility in modeling multi-agent interactions.

5.4 Deciding Drift Boundedness

A natural meta-question is whether drift boundedness itself can be effectively verified. We consider four variants:

1. **Universal k-boundedness (fixed k):** Given $\mathcal{N}$ and k, is every reachable configuration k-drift bounded?
2. **Universal boundedness (existence of k):** Given $\mathcal{N}$, does there exist k such that all reachable configurations are k-drift bounded?
3. **Existential k-boundedness (fixed k):** Given $\mathcal{N}$ and k, does every discrete behavior admit a k-drift bounded realization? (This is Definition 5.5.)
4. **Existential boundedness (existence of k):** Given $\mathcal{N}$, does there exist k such that $\mathcal{N}$ is existentially k-drift bounded?

We address the first two questions here; the existential variants remain open but are less central to practical verification.

5.4.1 Universal k-Drift Boundedness (Fixed k)

Proposition 5.11 (Deciding universal k-boundedness)**.** *Given an LTN $\mathcal{N}$ and a constant k, deciding whether $\mathcal{N}$ is universally k-drift bounded is PSPACE-complete.*

Proof. **PSPACE Membership:** We reduce this problem to safety reachability over a finite state space. Define an augmented region equivalence $\cong^{\text{univ}}_{M,k}$ that extends $\cong^{k}_{M}$ by tracking a special "Unbounded" state reached whenever $|v(t_p) - v(t_q)| > k$ for any pair p, q.

Construct a region automaton $\mathcal{R}^{\text{univ}}(\mathcal{N}, k)$ over these regions. The LTN violates universal k-drift boundedness if and only if the "Unbounded" state is reachable in $\mathcal{R}^{\text{univ}}(\mathcal{N}, k)$. Reachability in this finite graph can be checked in NPSPACE, hence PSPACE by Savitch's theorem.

PSPACE-hardness: A timed automaton is a single-agent LTN with drift 0. Location reachability for timed automata is PSPACE-complete. We reduce it by adding a self-loop at the target location that enforces large drift (e.g., by having a second dummy agent with an unbounded clock). Reachability of the target in the timed automaton corresponds to reachability of an unbounded-drift state in the constructed LTN. □

5.4.2 Universal Drift Boundedness (Existence of k)

Proposition 5.12 (Undecidability of universal boundedness)**.** *Given an LTN $\mathcal{N}$, it is undecidable whether there exists $k \in \mathbb{N}$ such that $\mathcal{N}$ is universally k-drift bounded.*

Proof. In Chapter 3, it was shown that LTNs can simulate 2-counter machines, where the difference $v(t_p) - v(t_q)$ between reference clocks encodes counter values. Universal drift boundedness is therefore equivalent to asking whether the counters in the simulated machine remain bounded for all possible executions.

The *boundedness problem* for 2-counter machines—determining whether there exists k such that no counter ever exceeds k in any execution—is known to be undecidable [**?**]. Since this problem reduces to checking universal drift boundedness for the corresponding LTN, the latter is also undecidable. □

5.5 Acyclic Local-Timed Negotiations

The results of the previous section establish decidability for k-drift bounded LTNs when k is given. However, drift boundedness is a *semantic* property—verifying it requires reasoning about all possible behaviors. A natural question is: when can drift boundedness be guaranteed *structurally*, based solely on the syntax of the LTN?

We now study *acyclic LTNs*, where the absence of cycles in the marking graph implicitly enforces bounded drift, even though acyclicity imposes no explicit constraints on timing or clock values.

Definition 5.13 (Acyclic LTN)**.** A local-timed negotiation $\mathcal{N}$ is *acyclic* if its marking graph contains no cycle.

In an acyclic LTN, every discrete run visits each marking at most once. This immediately bounds the length of any run by the number of reachable markings, which is finite and computable from the structure of $\mathcal{N}$.

5.5.1 Acyclicity Implies Drift Boundedness

Theorem 5.14 (Acyclic LTNs are drift bounded)**.** *Every acyclic local-timed negotiation is existentially drift bounded. Moreover, for every acyclic LTN $\mathcal{N}$ there exists a constant $k \in \mathbb{N}$, computable from $\mathcal{N}$, such that $\mathcal{N}$ is k-drift bounded.*

Proof. Fix an acyclic LTN $\mathcal{N}$ over agents P and let m be the maximum length of any discrete run in $\mathcal{N}$ (i.e., the maximum number of outcomes in a run). Such an m exists because the marking graph is acyclic, hence every run visits each marking at most once. Clearly, $m \leq |N|$ where $|N|$ is the number of nodes.

Let M be the maximum constant occurring in any guard of $\mathcal{N}$. We claim that $\mathcal{N}$ is k-drift bounded for $k := m \cdot (M + 1)$.

Consider an arbitrary run

$$\rho = (C_0, v_0) \xrightarrow{\Delta_0, e_0} (C_1, v_1) \xrightarrow{\Delta_1, e_1} \cdots \xrightarrow{\Delta_{m-1}, e_{m-1}} (C_m, v_m).$$

We construct a retiming ρ_k of ρ that is k-drift bounded, proceeding step-by-step.

At step i, let $e_i = (n_i, r_i)$ and let $P_i := dom(n_i)$ be the set of participating agents. Choose the local delays $\Delta'_i = [\delta_{p,i}]_{p \in P}$ as follows:

- For each participating agent $p \in P_i$, set $\delta_{p,i}$ to be the *least nonnegative* value that enables e_i with respect to p local clocks (after applying previous resets).
- For each non-participating agent $q \notin P_i$, set $\delta_{q,i} := 0$.

These choices preserve the discrete skeleton because enabling of e_i depends only on the clocks of agents in P_i. Moreover, by the standard region argument for timed automata, when guards have constants bounded by M, the least enabling delay for any agent is strictly less than $M+1$: once a clock exceeds M, guards no longer distinguish its precise value, so there always exists an enabling representative with delay $< M + 1$.

Consequently, between two consecutive discrete outcomes, each participating agent advances by at most $M + 1$ units of reference time, and non-participants do not advance at all. Since the run has at most m outcomes, for every agent p we have

$$v_m(t_p) - v_0(t_p) \leq m \cdot (M + 1) = k.$$

Because all reference clocks start at the same value in the initial configuration ($v_0(t_p) = 0$ for all p), for every pair of agents $p, q \in P$ and every configuration along the retimed run,

$$|v_i(t_p) - v_i(t_q)| \leq m \cdot (M + 1) = k.$$

Thus the retimed run ρ_k is k-drift bounded. Since ρ was arbitrary, $\mathcal{N}$ is existentially k-drift bounded. □

This result is significant because it provides a *syntactic sufficient condition* for a *semantic property*. Acyclicity can be checked efficiently by graph algorithms, yet it guarantees drift boundedness with an explicit, computable bound.

5.5.2 NP-Completeness of Reachability

Having established that acyclic LTNs are drift bounded, we now characterize the complexity of reachability for this fragment.

Theorem 5.15 (Reachability for acyclic LTNs)**.** *Location reachability for acyclic local-timed negotiations is NP-complete.*

Proof. **NP Membership.** Let $\mathcal{N}$ be acyclic and let C_f be a target marking. Because the marking graph is acyclic, every discrete run visits each marking at most once, so any run reaching C_f has a discrete skeleton $w = e_0 \cdots e_{m-1}$ of length m bounded by the number of markings.

A nondeterministic algorithm guesses such a sequence w and checks whether it forms a valid path from C_0 to C_f in the marking graph. It then verifies whether w admits a timed realization by encoding the problem as a system of difference constraints.

For each agent p and step i, introduce variables t_p^i (reference time at step i) and $\tilde{x}^i$ (time of last reset for each local clock $x \in X_p$). Guards $x \bowtie c$ at step i become constraints on $t_p^i - \tilde{x}^i$, resets are encoded by equalities $\tilde{x}^{i+1} = t_p^i$ (or $\tilde{x}^{i+1} = \tilde{x}^i$ if not reset), local time elapse by $t_p^{i+1} \geq t_p^i$, and synchronizing outcomes by $t_p^i = t_q^i$ for all participants.

This yields a system $\Phi(w)$ of difference constraints of size polynomial in m, $|P|$, and $|X|$. Feasibility of difference constraints can be checked in polynomial time using the Bellman-Ford algorithm. Since m is polynomial in the size of $\mathcal{N}$ (due to acyclicity), verification is in P. Thus, the overall problem is in NP.

NP hardness. We reduce SUBSET-SUM to reachability in acyclic LTNs. Given nonnegative integers $S = \{n_1, \ldots, n_k\}$ and a target $a \in \mathbb{N}$, determine whether there exists a subset $I \subseteq \{1, \ldots, k\}$ such that $\sum_{i \in I} n_i = a$.

Construction: From an instance (S, a), construct a single-agent acyclic LTN $\mathcal{N}_{S,a}$ as follows:

- Agent set: $\{p\}$.
- Clock set: $X_p = \{x, y\}$, where x is reset at each stage and y accumulates total elapsed time.
- Nodes: $\{n_0, n_1, \ldots, n_k, n_f\}$.
- Initial marking enables n_0; target is n_f.

Transitions:

- Node n_0 has outcome start with guard $\top$, reset set $\{x\}$, and successor n_1.
- For each $i = 1, \ldots, k$, node n_i has two outcomes:
  - *Include* in: guard $(x = n_i)$, reset $\{x\}$, successor n_{i+1} (or n_f if $i = k$).
  - *Exclude* out: guard $(x = 0)$, reset $\{x\}$, successor n_{i+1} (or n_f if $i = k$).
- Node n_f has outcome done with guard $(y = a)$ and no resets.

FIGURE 5.3: Acyclic LTN encoding SUBSET-SUM. Clock x is reset at every step and enforces a choice of delay 0 or n_i; clock y accumulates total elapsed time and is checked at the final node.

The marking graph forms a linear chain $n_0 \to n_1 \to \cdots \to n_k \to n_f$, hence is acyclic.

Correctness: At node n_i, clock x has just been reset. Outcome out forces zero delay, while outcome in forces exactly n_i units of delay. Thus each stage contributes either 0 or n_i time units. Since y is never reset, upon reaching n_f its value equals $\sum_{i \in I} n_i$ where I is the set of indices for which in was chosen. The final guard $(y = a)$ is enabled iff this sum equals a.

Therefore, n_f is reachable iff the SUBSET-SUM instance has a solution. This reduction is polynomial and preserves acyclicity, proving NP-hardness. □

The drop from PSPACE (for general k-drift bounded LTNs) to NP (for acyclic LTNs) reflects the computational difference between verifying a semantic property versus exploiting a structural guarantee. In acyclic LTNs, the bound on run length provides additional structure that simplifies reachability analysis.

5.6 Summary and Perspective

This chapter investigated *drift*—the maximum divergence between agents' reference clocks—as both a source of undecidability and a unifying lens for understanding decidable fragments of local-timed negotiations.

Main Contributions

Semantic restriction: drift boundedness. We formalized an existential notion of k-drift boundedness (Definition 5.5), requiring that every discrete behavior admit at least one timed realization with bounded clock divergence. This retiming-based notion, standard in timed automata theory, enables finite abstraction techniques.

Decidability via region abstraction. For k-drift bounded LTNs, we adapted region-equivalence constructions from timed automata to obtain a finite k-region automaton (Section 5.3.2). This yielded PSPACE-completeness for location reachability (Theorem 5.10), matching the complexity of timed automata despite the greater expressiveness of multi-agent interactions.

Structural implication: acyclicity. We showed that acyclicity of the marking graph—a purely structural, efficiently checkable property—implicitly enforces bounded drift (Theorem 5.14). For acyclic LTNs, reachability drops to NP-complete (Theorem 5.15), reflecting the added structure that simplifies verification.

Meta-complexity. We established that verifying universal k-drift boundedness is PSPACE-complete (Proposition 5.11), while determining the existence of a universal drift bound is undecidable (Proposition 5.12).

Conceptual Insights

Three key insights emerge from this chapter:

1. **Drift as a unifying principle:** Unbounded drift enables counter encoding and undecidability, while bounded drift—whether enforced semantically or structurally—restores finite abstractions. Both synchronization extremes from Chapter 4 can be understood as special cases where drift is controlled: synchronization-free LTNs trivialize drift by preventing clock comparisons, while always-synchronizing LTNs bound drift through constant resynchronization.

2. **Existential vs. universal:** The existential notion of drift boundedness (retiming-based) is weaker but more useful for verification than universal boundedness. It aligns with time-abstract verification and enables reasoning about discrete behaviors independently of specific timing choices.

3. **Syntax implies semantics:** Acyclicity provides a syntactic sufficient condition for the semantic property of drift boundedness. This connection is non-obvious—acyclicity constrains control flow but says nothing explicitly about timing—yet it guarantees bounded drift with a computable bound. This pattern suggests that other structural restrictions might similarly imply semantic boundedness properties.

Broader Perspective

From a broader perspective, this chapter highlights a fundamental trade-off in real-time concurrent systems: expressiveness arises from the combination of local autonomy (independent clocks), selective coordination (partial synchronization), and unbounded temporal evolution, while decidability is recovered by constraining how divergence accumulates across interactions.

The dichotomy between NP (acyclic) and PSPACE (general k-drift bounded) also reveals a computational spectrum: systems where boundedness is structurally guaranteed admit simpler verification than those where boundedness must be

checked semantically. This suggests that identifying richer classes of structurally drift-bounded LTNs could yield practical decidable fragments.

5.7 Coming Next: Connected Communication

Acyclicity is a strong restriction: many natural systems exhibit cycles in their control structure while still maintaining disciplined patterns of communication and synchronization. The next chapter studies such systems through the notion of *connectedly communicating local-timed negotiations*.

Connectedly communicating LTNs allow cycles in the marking graph but impose a structural constraint on how agents interact across these cycles. Intuitively, agents that diverge in time cannot later compare their clocks without passing through a communication path that enforces synchronization. This condition rules out the counter-encoding patterns responsible for undecidability while retaining significantly more expressive power than acyclic LTNs.

In Chapter **??**, we show that connected communication suffices to bound drift and yields decidable reachability, thereby bridging the gap between purely semantic drift bounds (this chapter) and purely structural acyclicity. This completes our characterization of the decidability landscape for LTNs, delineating the boundary between tractable and intractable fragments based on the interplay of timing, synchronization, and interaction structure.

Chapter 6

Connectedly Communicating LTN

6.1 Introduction

In the previous chapter we considered a *bounded-drift* property for an LTN, similar to the notion considered in [**?**] for networks of timed automata. An LTN is said to have drift bounded by K if in every behaviour, the local-time delays can be *tightened* so that the drift between the local-times of every pair of processes is always bounded by a constant K. As we showed in the previous chapter, this property makes reachability decidable by enabling the construction of a region automaton for the set of untimed behaviours.

In this chapter, we identify a structural fragment of LTNs where bounded drift holds automatically. We call this fragment *connectedly-communicating* negotiations.

6.1.1 The Connectedly-Communicating Fragment

As we have seen in previous chapters, local-timed negotiations (LTNs) sit between two well-behaved extremes: on the one hand, fully synchronizing LTNs behave essentially like a single global clock; on the other hand, synchronization-free LTNs behave like independent local clocks. The difficulty begins exactly when we *mix* local time elapse with occasional time-synchronizing negotiations. Then one process can wait for a long time, later force another process to catch up at a synchronizing node, and thereby transport large time gaps across the system. This is precisely the mechanism behind the undecidability phenomena established earlier.

This chapter isolates a robust *structural* reason why such transport of time gaps can be controlled. The key intuition is:

> If, in every cycle of the control structure, time-synchronizations connect all agents (directly or through a chain), then no agent can accu-

mulate unbounded local time without repeatedly synchronizing it back into the rest of the system.

We formalize this intuition through the notion of *connectedly-communicating* LTNs and prove that this structural condition forces bounded drift.

Main result. Our main theorem shows that connected communication implies a uniform drift bound.

Theorem 6.1 (Informal statement)**.** *Every connectedly-communicating local-timed negotiation is drift-bounded. More precisely, if $\mathcal{N}$ is connectedly-communicating, then every behaviour of $\mathcal{N}$ can be tightened so that the drift between any two processes is bounded by*

$$O(|N| \cdot |P|^3 \cdot (M + 1)),$$

where N is the set of negotiation nodes, P is the set of processes, and M is the largest timing constant appearing in the guards.

Combined with the region-style construction from the previous chapter, this immediately yields the following consequence.

Corollary 6.2 (Informal statement)**.** *Reachability is decidable for connectedly-communicating LTNs.*

Thus, the chapter identifies a natural structural fragment of LTNs that still allows partial local-time evolution, but rules out the unbounded drift patterns responsible for undecidability.

Two proof layers: local tightening, then global boundedness. The proof decomposes into two conceptually separate layers:

1. **Tightening (local normal form).** Every run can be transformed into a run in which no unnecessary waiting above $M+1$ remains, except what is forced by time-synchronizations. This step is purely semantic and does not use connected communication.

2. **Bounded drift (global consequence of connectivity).** For connectedly-communicating LTNs, timestamps in a tightened run cannot diverge arbitrarily: connectivity forces the effect of synchronizations to propagate across all processes within a bounded amount of control progress.

Related work. The idea that connected communication restores decidability is inspired by two classical results:

1. Alur and Yannakakis's work on hierarchical message sequence graphs [**?**], where a related connectedness requirement yields decidability of emptiness, and

2. Madhusudan, Thiagarajan, and Yang's work on distributed controller synthesis [**?**], where a suitable connectedness condition again rescues decidability from an otherwise undecidable setting.

Our setting differs somewhat from [**?**], where the requirement is in fact weaker: roughly speaking, if process p never hears from q (directly or indirectly) along a cycle, then p and q never synchronize again in the future. We discuss this weaker variant in Section 6.5.

The notion of drift has also been studied extensively for networks of timed automata in [**?**, **?**]. In that setting, local-time semantics supports more efficient symbolic verification by avoiding one dimension of exponential blowup caused by interleaving independent actions. Bounded drift then provides simulation arguments ensuring finiteness of zone-based exploration.

Organization of the chapter. Section 6.2 recalls the definition of local-timed negotiations and introduces our trace-based view of behaviours, which will be essential for the technical development. Section **??** recalls the drift-bounded framework from the previous chapter and the reachability consequences that we will use here. Our main technical contribution begins in Section 6.3, where we prove that connectedly-communicating LTNs have bounded drift. The proof proceeds in two main steps: in (Section 6.4.3) we describe tightening, i.e., how to systematically reduce time delays in any run while preserving validity; and in (Section 6.4.4) we prove that tight runs of connectedly-communicating LTNs have uniformly bounded drift. Section 6.5 concludes the chapter and discusses extensions and future directions.

6.2 Understanding Connectedly Communicating LTNs Through Examples

Before turning to the formal development, we discuss a small family of examples that illustrates the structural idea behind connected communication. The point of these examples is not to analyze all timed behaviours in full detail, but to show how the pattern of *time-synchronizing contacts along loops of the marking graph* controls the possibility of unbounded drift.

Figure 6.1 shows three closely related LTNs with the same overall control structure, but with different synchronization patterns. These differences are not yet fully visible at the level of the LTN itself. They become clearer when we pass to loops in the untimed marking graph and then abstract each such loop to a graph on agents that records which pairs of agents synchronize together along that loop. Figures 6.2 and 6.3 make this viewpoint explicit, while Figure 6.4 illustrates that, even in the connectedly-communicating setting, propagation of timing information may require several iterations.

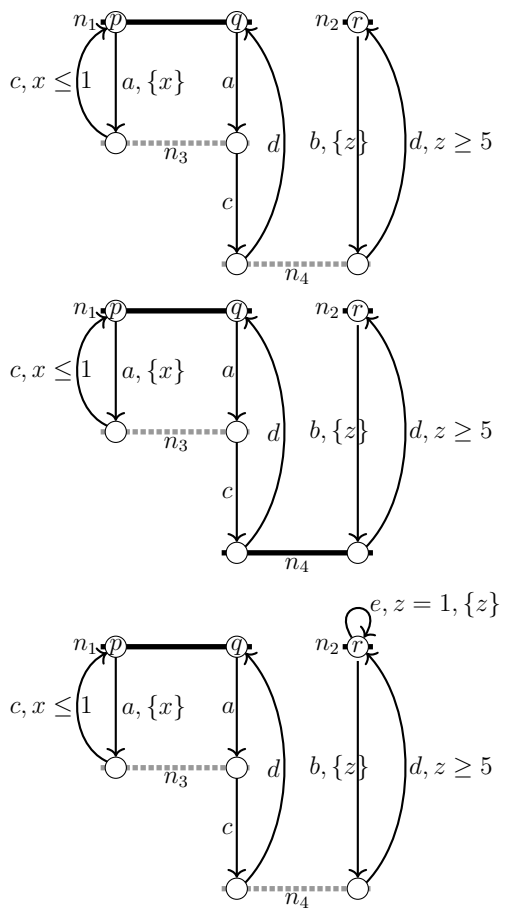


FIGURE 6.1: Three variations of LTNs illustrating different synchronization patterns. (a) Connectedly communicating; (b) not connectedly communicating because one synchronizing link is missing; (c) not connectedly communicating because of an isolated local loop, leading to unbounded drift.

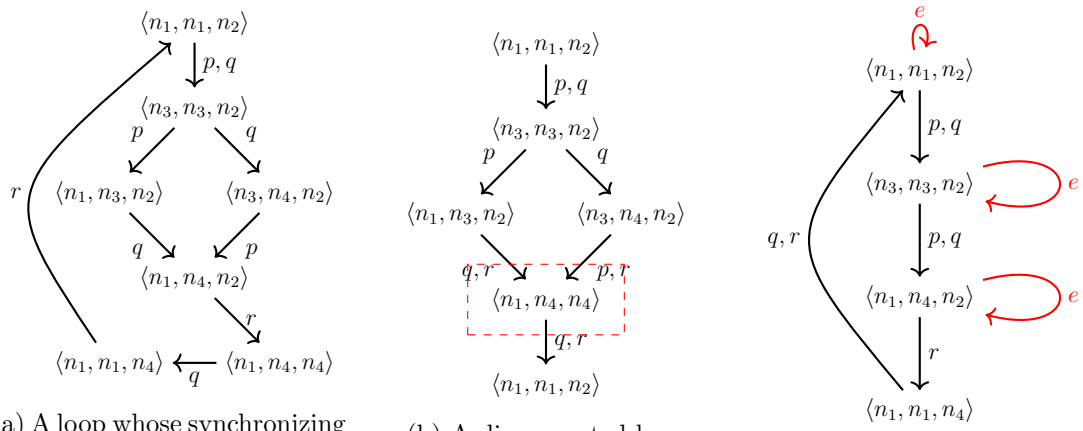
Example 6.2.1. Let

$$C = (\{n_1\}, \{n_1\}, \{n_2\})$$

be the marking in which the three agents p, q, r are ready at n_1, n_1, n_2, respectively. For each of the three LTNs in Figure 6.1, Figure 6.2 shows a representative loop from C back to C in the untimed marking graph $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$, and Figure 6.3 shows the corresponding communication graph.

To analyze whether timing information can propagate through a cyclic behaviour, we proceed in two steps. First, we look at a loop in the untimed marking graph $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$, as illustrated in Figure 6.2. Such a loop records which local moves can occur cyclically from a given marking, but it still contains more information than we need for connected communication.

From this loop, we then extract a simpler graph whose vertices are the agents. We put an edge between two agents p and q whenever some time-synchronizing node appearing on the chosen loop has both p and q in its domain. In other words, the communication graph forgets the intermediate markings and remembers only



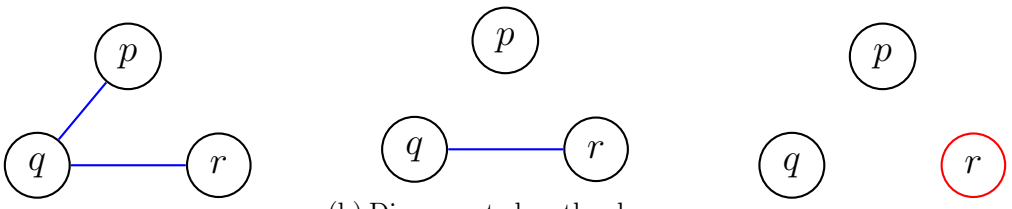
(a) A loop whose synchronizing contacts connect all agents

(b) A disconnected loop

(c) An isolated local loop

FIGURE 6.2: Representative loops in the untimed marking graphs $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$ for the three variants. These loops will be used to extract the communication graphs of Figure 6.3 by recording which agents synchronize together along each loop.

which agents are forced to time-synchronize together somewhere along that loop. Figure 6.3 shows the communication graphs obtained from the loops of Figure 6.2.



(a) Connected

(b) Disconnected on the chosen loop

(c) Totally disconnected

FIGURE 6.3: Communication graphs extracted from the loops shown in Figure 6.2. The vertices are the agents, and two agents are connected whenever they participate together in some time-synchronizing node occurring on the corresponding loop.

Figure 6.3 extracts, from the representative loops of Figure 6.2, the corresponding communication graphs. The vertices are the participating agents, and two agents are joined by an edge whenever they occur together in some time-synchronizing node along the chosen loop. In (a), the graph is connected, which is the structural pattern underlying the connectedly-communicating property. In (b), the graph is disconnected for the chosen loop, showing that loop-level connectedness fails here. In (c), agent r is completely isolated, reflecting the presence of a purely local loop and the resulting possibility of unbounded drift.

Example (a): connectedly communicating. Consider the loop from C back to C obtained by executing the outcomes a, b, d, c. Along this loop, the time-synchronizing nodes are

$$n_3 \text{ with participants } \{p, q\}, \qquad n_4 \text{ with participants } \{q, r\}.$$

Hence the induced communication graph has edges (p, q) and (q, r), and is therefore the connected path

$$p - q - r.$$

This is exactly the pattern required by connected communication.

For intuition, consider one traversal of the cycle from the marking (n_1, n_1, n_2). At n_1, agents p and q execute a. Since n_1 is not time-synchronizing, they may do so at different local times. The reset of x at a, together with the guard $x \leq 1$ at the time-synchronizing node n_3, forces p and q to meet again within one time unit from p's reset.

Meanwhile, agent r traverses the loop through n_2 and n_4, where the reset of z and the guard $z \geq 5$ force each iteration to take at least 5 time units. When q synchronizes with r at n_4, this timing constraint is transferred to q, and later propagated from q back to p through their own synchronizations. Thus timing information flows through the chain $r \leftrightarrow q \leftrightarrow p$, which is the key reason drift remains bounded.

Example (b): a disconnected loop. Now suppose that n_4 is no longer time-synchronizing, as in Figure 6.1(b). Then on the analogous loop the only time-synchronizing node is n_3, with participants $\{p, q\}$. The induced communication graph therefore contains only the edge (p, q), while r is isolated. Thus this example is *not* connectedly communicating.

The reason drift can now blow up is that time accumulated by r is no longer forced back into the rest of the system through synchronization. The pair (p, q) may remain mutually constrained, but r can move arbitrarily far away in local time without creating any contradiction.

Thus Example (b) lies outside the strict connectedly-communicating fragment. A weaker relaxation of this condition will be introduced later in Section 6.4.5.

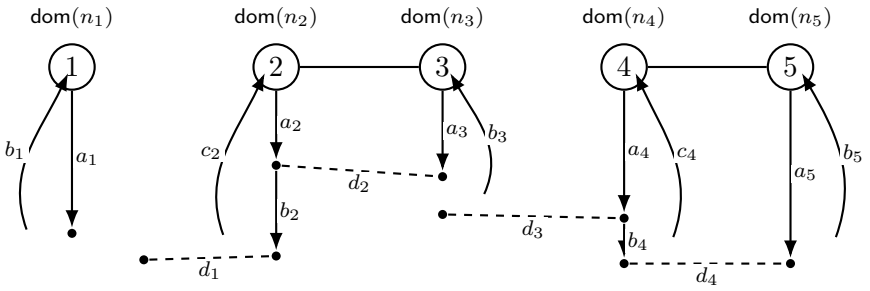
Example (c): an isolated local loop and unbounded drift. Finally, Figure 6.1(c) adds a self-loop e at n_2 for agent r. Consider the one-step loop

$$C \xrightarrow{e} C.$$

No time-synchronizing node appears on this loop. Hence the communication graph has vertex set $\{p, q, r\}$ and no edges at all, so it is totally disconnected.

Here the possibility of unbounded drift is immediate: agent r can iterate the local loop e arbitrarily many times while p and q remain where they are. Thus r can accumulate arbitrarily large local time without any synchronizing contact. This is the kind of behaviour that later makes region-like arguments fail.

The previous examples show when loop-level connectedness holds or fails. Even in the positive case, however, timing information need not propagate across all



It takes at least 2 iterations for the information of 5 to reach 1.

FIGURE 6.4: A schematic example showing that timing information need not propagate to all agents within a single propagation step. In general, the propagation argument may require up to $|P|$ iterations; in this example, timing information originating at node 5 reaches node 1 only after 2 iterations, through a chain of intermediate dependencies.

processes in a single step.

Iterated propagation of timing information. Figure 6.4 illustrates an important feature of the propagation argument even in the ordinary connectedly communicating setting. Connectedness of the communication structure does not mean that timing information becomes globally available in a single step. Rather, one obtains a local propagation effect, and this effect must be iterated.

Starting from one process (here, process 5), the timing information first reaches only those processes that are directly linked to it by the relevant local dependencies. In the next round, it can propagate further to processes that are linked to those, and so on. Thus the proof proceeds by repeated closure under this propagation relation. In general, timing information propagates at most one edge farther in each iteration. Hence the number of iterations needed is bounded by the length of the longest path in the communication graph, and therefore by $|P| - 1$. In the present example, 2 iterations are necessary for the influence of process 5 to reach process 1.

Takeaway. These examples isolate the structural issue behind the chapter. In Example (a), the synchronizing contacts that occur on the loop connect all agents, so timing constraints can propagate throughout the system. In Example (b), this loop-level connectedness fails: one agent is no longer tied back to the others through synchronizing contacts on the loop, and unbounded drift becomes possible. In Example (c), the failure is more drastic, since there is a genuinely isolated local loop, and unbounded drift follows immediately. The bounded-drift theorem proved later in the chapter shows that the positive pattern exemplified by (a) is exactly the one that suffices for the main decidability argument.

6.3 Connectedly communicating LTNs

Now we're ready to formally define connectedly-communicating LTNs and prove our main result: that this fragment has bounded drift. We'll build this up in stages, first defining the necessary graph-theoretic concepts, then explaining the tightening procedure, and finally proving the bound.

6.3.1 Marking Graphs and Communication Graphs

To define connectedness, we need to look at two different graphs derived from an LTN.

The Untimed Negotiation $\mathcal{N}_{\mathsf{untime}}$ For an LTN $\mathcal{N}$, let $\mathcal{N}_{\mathsf{untime}}$ denote the untimed version obtained by structurally removing all clock guards and resets from transitions. This gives us an over-approximation of possible behaviors—if something is impossible in $\mathcal{N}_{\mathsf{untime}}$, it's impossible in $\mathcal{N}$, but some behaviors of $\mathcal{N}_{\mathsf{untime}}$ might not be achievable in $\mathcal{N}$ due to timing constraints.

The Marking Graph $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$ The marking graph captures which nodes agents can be at simultaneously. A *marking* C maps each agent to a set of nodes that agent participates in.

The vertices of $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$ are all possible markings. There is an edge $C \xrightarrow{n} C'$ if node n is enabled at marking C (meaning $n \in C(p)$ for all agents p participating in n), and C' is the marking after executing n. The three variants in our running example (Figure 6.1) provide a useful illustration: For each of the LTNs of our running example (Figure 6.1), Figure 6.2 shows the corresponding marking graphs. The marking graphs make explicit the loops on which our connectedness condition will later be imposed.

Markings and the size of the marking graph. Since we work with $\mathcal{N}_{\mathsf{untime}}$ where outcomes are abstracted away in the edge labels (we write $C \xrightarrow{n} C'$ rather than $C \xrightarrow{(n,a)} C'$), a marking records, for each agent, *a set of possible current nodes.* Formally, a marking is a function $C : P \to 2^N$ such that $C(p) \subseteq \{\, n \in N : p \in \mathit{dom}(n) \,\}$ for each agent p. Hence the number of vertices of $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$ is at most

$$|\mathrm{V}(\mathrm{MG}(\mathcal{N}_{\mathsf{untime}}))| \;\leq\; \prod_{p \in P} 2^{|N|} \;=\; 2^{|N| \cdot |P|}.$$

Moreover, from any marking there are at most $|N|$ outgoing edges (one per node label), so $|\mathrm{E}(\mathrm{MG}(\mathcal{N}_{\mathsf{untime}}))| \leq 2^{|N| \cdot |P|} \cdot |N|$.

For example, in Figure 6.1(a), one possible marking is $C = (\{n_1\}, \{n_1\}, \{n_2\})$, meaning p and q are at n_1 while r is at n_2. From this marking, we can execute n_1 with outcome a, leading to marking $C' = (\{n_3\}, \{n_3\}, \{n_2\})$.

Loops in the Marking Graph A loop in $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$ is a cyclic sequence:

$$C = C_1 \xrightarrow{n_1} C_2 \xrightarrow{n_2} \cdots \xrightarrow{n_k} C_{k+1} = C$$

This corresponds to a repeating pattern of behavior where the system returns to the same marking.

The Communication Graph of a Loop For each loop in $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$, we define its communication graph:

- **Vertices**: The set of all agents P
- **Edges**: There is an edge (p, q) if processes p and q participate together in some time-synchronizing node during the loop. Formally, there exists some n_i in the loop such that $p, q \in dom(n_i)$ and $n_i \in Sync$ (the set of time-synchronizing nodes).

Intuitively, the communication graph shows which agents synchronize their clocks (directly) during one iteration of the loop.

Intuition: communication prevents hidden time. In a loop of the untimed behaviour, an agent can locally "wait" an arbitrary amount of time unless that waiting is eventually revealed to others through a time-synchronizing event. If the time-synchronizing nodes occurring in the loop connect all agents (directly or via a chain), then repeatedly executing the loop forces local-times to repeatedly "catch up" along that connected structure. The connectedly-communicating condition is precisely the requirement that no agent can remain permanently isolated from such catch-up constraints inside any cyclic behaviour.

Why cycles in the marking graph. The marking graph $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$ captures the *control backbone* of the negotiation: it forgets timing constraints but remembers which nodes can follow which, under concurrency. A loop in $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$ represents the possibility of returning to the *same global control situation* while executing some nontrivial sequence of negotiations.

This is precisely where unbounded drift can arise: if an agent can traverse a loop while remaining "time-disconnected" from others, it may accumulate local time repeatedly and later export it through a synchronization. The connectedly-communicating restriction prevents such time-disconnected looping: every control loop must contain enough time-synchronizations to connect all agents into one communication component.

Definition 6.3 (Connectedly Communicating LTN)**.** An LTN $\mathcal{N}$ is *connectedly communicating* if for every loop in the marking graph $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$, the corresponding communication graph is connected.

Lemma 6.4 (Cut property of loop communication graphs)**.** *Assume* $|P| \geq 2$*. Let*

$$C_1 \xrightarrow{n_1} C_2 \xrightarrow{n_2} \cdots \xrightarrow{n_k} C_{k+1} = C_1$$

be a loop in $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$ *and let* G *be its communication graph (vertices* P*, edges induced by time-synchronizing nodes occurring along the loop). If* G *is connected, then:*

1. ***No isolated vertex.*** *Every agent participates in at least one time-synchronizing node along the loop: for every* $p \in P$ *there exists* i *such that* $n_i \in Sync$ *and* $p \in dom(n_i)$*.*

2. ***Edge crosses every nontrivial cut.*** *For every nonempty proper subset* $S \subsetneq P$*, there exists a time-synchronizing node* $n_i \in Sync$ *on the loop and agents* $p \in S$*,* $q \in P \setminus S$ *such that* $p, q \in dom(n_i)$*. Equivalently, along* any execution segment *realizing this loop, there is a time-synchronizing event whose participant set intersects both* S *and* $P \setminus S$*.*

Proof. (1) If some agent p did not appear in any time-synchronizing node on the loop, then p would have degree 0 in G, contradicting connectedness.

(2) In any connected undirected graph, every nontrivial cut $(S, P \setminus S)$ is crossed by some edge. Such an edge (p, q) in G exists iff there is a time-synchronizing node on the loop in which both p and q participate (by definition of G). The statement about execution segments follows because the loop label sequence contains that node, hence any realizing segment executes a corresponding time-synchronizing event. □ □

Lemma 6.5 (Every long enough segment contains a synchronizing contact)**.** *Let* $\mathcal{N}$ *be connectedly communicating and let* ρ *be any run. Fix* $N^* := |\mathrm{V}(\mathrm{MG}(\mathcal{N}_{\mathsf{untime}}))| + 1$*. In any segment of* ρ *containing* N^* *consecutive events, some marking repeats in* $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$*, hence the segment contains a loop of* $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$*. Consequently, within that segment, every agent that appears in the repeated marking participates in time-synchronizing events that connect the agents according to the loop's communication graph.*

6.4 Main Theorem

We can now state our main result:

Theorem 6.6. *Every connectedly communicating LTN* $\mathcal{N}$ *is* K*-drift bounded with*

$$K \in \mathcal{O}\big(|\mathrm{V}(\mathrm{MG}(\mathcal{N}_{\mathsf{untime}}))| \cdot |P|^3 \cdot (M+1)\big) \subseteq \mathcal{O}\big(2^{|N| \cdot |P|} \cdot |P|^3 \cdot (M+1)\big).$$

where N *is the set of nodes,* P *the set of agents, and* M *the maximum constant appearing in guards of* $\mathcal{N}$*.*

6.4.1 Proof Roadmap

The bounded-drift argument proceeds in two layers. First, we move from concrete runs to their trace representation, where for each agent the local order of events and the positions of time-synchronizations are explicit. On traces we define the *slack graph*, which records both where excess delay above $M + 1$ can be removed and how synchronization constraints may prevent further reduction. This yields the notion of a *tight trace*, namely a trace in which every time-synchronizing event already has a zero-weight witness path from an initial event.

The proof then has two main parts. In the first part, we show that every run can be transformed into a tight trace by eliminating unnecessary delay while preserving validity. In the second part, we show that in a connectedly-communicating LTN, every tight trace has uniformly bounded drift. The key point is that tightness gives a bound on individual synchronization timestamps via witness paths, while the finiteness of the marking graph and connected communication on loops allow these local bounds to propagate across agents and become uniform over the whole run.

More precisely, the argument unfolds as follows:

1. **Trace representation (Section ??).** We encode a run as a trace, making explicit for each agent its local event order together with the synchronization points shared with other agents. This gives a finite combinatorial object on which the tightening argument can be formulated cleanly.

2. **Slack graph (Section ??).** For a trace, we build the slack graph. Its forward edges measure delay that may potentially be removed beyond the threshold $M + 1$, while its backward edges capture the fact that future synchronizations can constrain present reductions.

3. **Tight traces (Section ??).** Using the slack graph, we define what it means for a trace to be tight: every synchronizing event admits a zero-weight witness path from an initial event. Intuitively, such a trace contains no remaining removable delay at synchronization points.

4. **Bounding one synchronization timestamp (Section ??).** In a tight trace, the timestamp of a synchronizing event can be bounded by a witness path leading to it. Along such a path, only forward time edges contribute positively, and each contributes at most $M + 1$. This gives a bound for a single synchronization point, but one that still depends on the length of the witness path, and therefore on the particular run.

5. **Extracting loops from long runs (Section ??).** To remove this run-dependence, we use the finiteness of the marking graph. Any sufficiently long segment of a run must repeat a marking, and hence contains a loop in the untimed marking graph.

6. **Using connected communication on loops (Section ??).** For connectedly-communicating LTNs, the communication graph induced by the synchronizing nodes on such a loop is connected. As a result, synchronization constraints can be transferred from one agent to another along a bounded chain of synchronizing nodes.

7. **Uniform drift bound (Section ??).** Combining tightness with this loop-chaining argument yields a bound on the spread of synchronization timestamps that depends only on the system, not on the particular run. This gives a uniform bound on drift for all tight traces, and hence for all runs after tightening.

Thus, the proof splits naturally into two sections: Section 6.4.3, which shows that every run can be tightened, and Section 6.4.4, which proves that every tight run of a connectedly-communicating LTN has uniformly bounded drift.

The rest of this section develops these two steps in detail.

The basic saturation principle. Let M be the largest constant appearing in any guard of $\mathcal{N}$. Guards can only distinguish whether a clock value is $\leq M$ and, if so, its precise value within that range. Once a clock exceeds M, its exact magnitude becomes irrelevant for future guard satisfaction, until it is reset.

This yields the following meta-principle: if we shorten a local delay by shaving off time that was spent while all relevant clocks were already $> M$, then the truth of all guards is preserved.

Lemma 6.7 (Saturation lemma)**.** *Fix an event e in a run, and consider an agent p with a local delay segment of length θ between two consecutive events of p. Let $\theta' \geq 0$ be obtained from θ by reducing it to at least $M{+}1$, i.e. $\theta' = \min(\theta, M{+}1)$, and keep all other delays unchanged. Then for every clock valuation just before the second event, every clock x satisfies:*

$$v(x) \leq M \iff v'(x) \leq M,$$

where v' is the valuation under the reduced delay θ'. In particular, every guard with constants bounded by M is enabled at exactly the same events under v and under v'.

Proof sketch. Only clocks not reset in the segment can change their values. If a clock is $\leq M$ after the original delay, then the original delay could not have exceeded $M{+}1$ *since the last reset of that clock along this local segment*; hence it remains unchanged under truncation. If a clock is $> M$ originally, then reducing the delay to at least $M{+}1$ keeps it $> M$. □ □

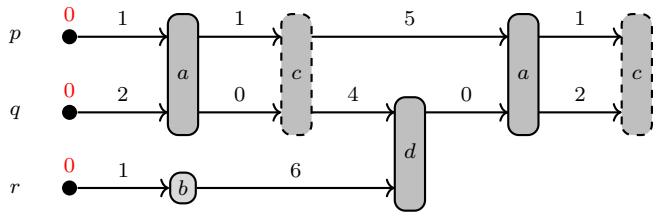


FIGURE 6.5: Trace representation of a run from Figure 6.1(a). Each agent has its own timeline (horizontal line), events are shown as boxes, and edge labels indicate time delays.

6.4.2 Trace Representation of a Run

To reason formally about LTN behaviors, we introduce a trace-based representation. This view makes the temporal relationships between events explicit and will be essential for our tightening algorithm.

Why traces. A run of an LTN is a *linear* sequence of steps, but much of this linearity is accidental: independent actions can commute, and time elapse is accounted for locally. What we need for the drift argument is the *causal structure* of a run: for each agent, which event comes next for that agent, and how much time that agent accumulates in between, which was hidden by the interleaving view of a run we have seen so far.

Traces provide exactly this: they forget irrelevant interleavings and retain only per-agent successor relations (the "local timelines") together with the synchronization constraints at time-synchronizing nodes. In spirit, this is analogous to Mazurkiewicz traces in concurrency theory, except that edges carry quantitative information (elapsed local time).

Interpreting the Trace Diagram Figure 6.5 shows the trace of a run from Example 6.2.1. Let's understand each component:

- **Agent timelines**: Each horizontal line represents one agent's local timeline. Process p has a timeline at the top, q in the middle, r at the bottom.
- **Initial events**: The small filled circles (dots) on the left represent initial events, denoted $\bot_p, \bot_q, \bot_r$. The red numbers above them show initial local times (all 0 in this case).
- **Events**: Boxes represent negotiation outcomes. Light gray boxes (like a, b, d) are non-synchronizing events. Darker boxes with dashed borders (like c) are time-synchronizing events.
- **Time delays**: The numbers on edges show how much local time elapses for that agent between consecutive events. For example:

  - p waits 1 unit before participating in a
  - q waits 2 units before participating in a
  - After a, p waits 1 more unit before c, while q waits 0 units

- **Computing timestamps**: We can calculate the local time at any event by summing delays along the path from the initial event. For instance:
  - $\tau_p(a) = 0 + 1 = 1$ (initial time plus delay to a)
  - $\tau_q(a) = 0 + 2 = 2$
  - $\tau_p(c) = 0 + 1 + 1 = 2$
  - $\tau_q(c) = 0 + 2 + 0 = 2$ (same as p since c is time-synchronizing)

Key Properties of Traces The trace representation makes several important aspects explicit:

1. **Independence**: Each agent's timeline is separate, showing that agents advance time independently except at time-synchronizing events.
2. **Synchronization points**: Time-synchronizing events (dashed boxes) span multiple agent timelines, visually showing where times must match.
3. **Causality**: The left-to-right ordering along each agent's timeline respects causality—events to the right happen after events to the left for that agent.
4. **Partial ordering**: Events on different agent timelines are only ordered if there's a synchronization relationship. For example, b (for r) and the first a (for p, q) are concurrent—neither causally depends on the other.

Formal Definition Fix an LTN $\mathcal{N}$ and a run:

$$\rho := (C_1, v_1) \xrightarrow{\Delta_1, e_1} (C_2, v_2) \xrightarrow{\Delta_2, e_2} \cdots (C_k, v_k)$$

starting from configuration (C_1, v_1).

The trace $\mathsf{Tr}(\rho)$ is a directed graph $(E_\rho, \{\rightarrow_p\}_{p \in P})$ with an initial timestamp assignment $\{\tau_p^0\}_{p \in P}$, where:

- **Vertex set** E_ρ contains:
  - The negotiation events $\{e_1, e_2, \ldots, e_{k-1}\}$ from the run
  - Special initial events $\{\perp_p\}_{p \in P}$, one per agent
- **Edge set** partitioned as $\{\rightarrow_p\}_{p \in P}$: There is an edge $e_i \xrightarrow{\theta}_p e_j$ labeled θ if:
  - e_j is the next event p participates in after e_i

- Formally: $i < j$, $p \in dom(e_i) \cap dom(e_j)$, and no $i < i' < j$ has $p \in dom(e_{i'})$
- The label $\theta = \Delta_i(p) + \Delta_{i+1}(p) + \cdots + \Delta_{j-1}(p)$ is the total time p elapses between e_i and e_j

- **Initial timestamps** $\tau_p^0 = v_1(t_p)$ for each agent p

Given the trace and its initial timestamps, we can compute the local time at any event e for any agent $p \in dom(e)$ by summing along the path:

$$\tau_p(e) = \tau_p^0 + \sum_{\text{edges on path from } \bot_p \text{ to } e} \theta$$

For time-synchronizing events s, all participating agents have the same timestamp, so we write $\tau(s)$ for this common value.

Bidirectional Conversion Intuitively, a trace is a labelled partial-order view of a run, similar in spirit to Mazurkiewicz traces or message sequence charts: vertices are event occurrences, and edges record *the next event seen by each agent* together with the *total time spent in between.* Thus, forgetting the numeric labels yields the causal skeleton of the run, while keeping them allows us to reconstruct the exact local timestamps. Time-synchronizing events are special because they force the participating agents' timestamps to coincide, turning the trace into a system of timing constraints.

Run to trace. Fix an LTN $\mathcal{N}$ with set of agents P. Let

$$\rho \;:=\; (C_0, v_0) \xrightarrow{(\Delta_1, e_1)} (C_1, v_1) \xrightarrow{(\Delta_2, e_2)} \cdots \xrightarrow{(\Delta_k, e_k)} (C_k, v_k)$$

be a (finite) run of $\mathcal{N}$, where each e_i is an executed outcome (an event) and each $\Delta_i : P \to \mathbb{R}_{\geq 0}$ is the local-delay vector of the i-th step.

We define the trace $\mathsf{Tr}(\rho)$ as a structure

$$\mathsf{Tr}(\rho) \;=\; (E_\rho,\ (\to_p)_{p \in P},\ (\tau_p^0)_{p \in P})$$

where:

1. **Events.** The event set is

$$E_\rho \;:=\; \{\bot_p : p \in P\} \;\cup\; \{e_1, \ldots, e_k\}.$$

   The elements $\bot_p$ are fresh symbols (not occurring among the e_i) and represent the initial "dummy event" of agent p.

2. **Initial timestamps.** For each agent $p \in P$ we set

$$\tau_p^0 \;:=\; v_0(t_p).$$

3. **Per-agent successor edges.** Fix an agent $p \in P$ and consider the indices at which p participates:

$$I_p \;:=\; \{\, i \in \{1, \dots, k\} : p \in dom(e_i) \,\}.$$

If $I_p = \emptyset$, then $\bot_p$ has no outgoing $\to_p$-edge.

Otherwise write $I_p = \{i_1 < i_2 < \cdots < i_m\}$. We put:

- an edge $\bot_p \xrightarrow{\theta_{p,0}}_p e_{i_1}$, where

$$\theta_{p,0} \;:=\; \sum_{j=1}^{i_1} \Delta_j(p),$$

  i.e. the total time elapsed locally for p before its first participating event;
- for every $r \in \{1, \dots, m-1\}$ an edge

$$e_{i_r} \xrightarrow{\theta_{p,r}}_p e_{i_{r+1}}, \qquad \text{where} \qquad \theta_{p,r} \;:=\; \sum_{j=i_r+1}^{i_{r+1}} \Delta_j(p).$$

We write $u \xrightarrow{\theta}_p u'$ when (u, u') is an edge in the p-component with label θ.

Induced timestamps. Given $\mathsf{Tr}(\rho)$, for any event $e \in E_\rho$ and any agent $p \in P$ such that e lies on the p-chain (i.e. $e \in \{e_i : i \in I_p\}$), define $\tau_p(e)$ by

$$\tau_p(e) \;:=\; \tau_p^0 \;+\; \sum (\text{labels along the unique } \to_p\text{-path from } \bot_p \text{ to } e).$$

Equivalently, if $e = e_{i_r}$ then $\tau_p(e) = v_0(t_p) + \sum_{j=1}^{i_r} \Delta_j(p)$. In particular, for any time-synchronizing event s and any $p, q \in dom(s)$ we have $\tau_p(s) = \tau_q(s)$ (because ρ satisfies the time-synchronization constraint at s).

Trace to run. Let $\mathsf{Tr} = (E, (\to_p)_{p \in P}, (\tau_p^0)_{p \in P})$ be a trace. We say that an initial configuration (C_0, v_0) *matches* the initial timestamps of Tr if

$$v_0(t_p) = \tau_p^0 \quad \text{for all } p \in P.$$

Assume (C_0, v_0) matches Tr. We now define a canonical run

$$\rho[(C_0, v_0), \mathsf{Tr}]$$

obtained by choosing any linearization of the trace and executing the corresponding timed steps.

1. **The trace order and linearizations.** For each agent p, the relation $\rightarrow_p$ is a (finite) directed path starting at $\perp_p$ and visiting exactly the events in which p participates, in the order they appear for p. Let $\prec$ be the strict partial order on $E \setminus \{\perp_p : p \in P\}$ generated by these edges:

$$e \prec e' \quad \text{if} \quad \exists p \in P \text{ with } e(\rightarrow_p)^+ e'.$$

   A *linearization* of Tr is any sequence $(e_1, \ldots, e_k)$ listing all non-initial events exactly once such that $e \prec e'$ implies e appears before e' in the sequence.

2. **Delays induced by a linearization.** Fix a linearization $(e_1, \ldots, e_k)$. For each position $i \in \{1, \ldots, k\}$ and each agent $p \in P$, define the local delay

$$\Delta_i(p) \;:=\; \begin{cases} \theta & \text{if } pred_p(e_i) \xrightarrow{\theta}_p e_i \text{ is the incoming } p\text{-edge into } e_i, \\ 0 & \text{if } p \notin dom(e_i), \end{cases}$$

   where $pred_p(e_i)$ denotes the unique predecessor of e_i on the p-chain (it is either $\perp_p$ or a previous event of p). Thus, Δ_i is a function $P \rightarrow \mathbb{R}_{\geq 0}$.

3. **Executing the steps.** Starting from (C_0, v_0), execute the events in the chosen order. Concretely, for $i = 1, \ldots, k$ let

$$(C_{i-1}, v_{i-1}) \xrightarrow{(\Delta_i, e_i)} (C_i, v_i)$$

   be the unique successor configuration obtained by:

   (a) letting each agent p elapse $\Delta_i(p)$ time units in its local time (and updating all clocks of p accordingly under the LTN semantics), while agents not in $dom(e_i)$ elapse 0;

   (b) firing the outcome/event e_i from marking C_{i-1} (which updates the marking component to C_i in the standard way);

   (c) applying the guards/resets of e_i to update the valuation from the post-delay valuation to v_i (again as per the LTN operational semantics).

   The resulting sequence is $\rho[(C_0, v_0), \mathsf{Tr}]$.

Well-definedness and validity. For a fixed linearization, the above construction is *well-defined* because each event e_i has exactly one incoming $\rightarrow_p$-edge for each $p \in dom(e_i)$, hence Δ_i is uniquely determined.

Whether $\rho[(C_0, v_0), \mathsf{Tr}]$ is a *valid run* of $\mathcal{N}$ depends on feasibility: it is a valid run iff every step is enabled when executed, i.e. the marking enables e_i and the valuation after applying Δ_i satisfies the guard of e_i (and time-synchronization constraints, if e_i is time-synchronizing). When this holds for one (equivalently, for every) linearization consistent with $\prec$, we say that Tr is *realizable from* (C_0, v_0).

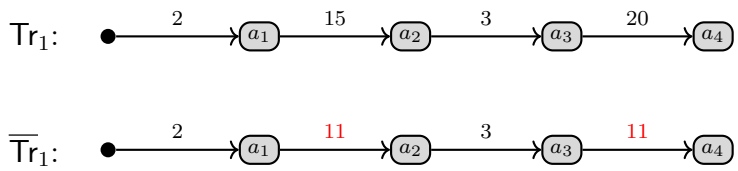


FIGURE 6.6: Tightening a single-agent trace with $M = 10$. Delays larger than $M + 1 = 11$ are reduced to exactly 11.

A useful viewpoint. Once a trace Tr is fixed, the only freedom left in reconstructing a run is the choice of a linearization (a topological order compatible with all $\rightarrow_p$ edges). Importantly, *timestamps of events do not depend on the chosen linearization*: $\tau_p(e)$ is determined purely by the unique $\rightarrow_p$-path from $\perp_p$ to e. This makes traces an ideal object for reasoning about drift, because drift compares timestamps, not interleavings.

The sequence $\rho[(C_0, v_0), \mathsf{Tr}]$ is a valid run if and only if all guards are satisfied and synchronization constraints are met.

This trace representation will be the foundation for our tightening algorithm in Section 6.4.3.

6.4.3 Tightening a Run

The key idea is that delays larger than $M + 1$ contain "slack" that can often be reduced. We'll formalize this through a sequence of increasingly complex examples, then define the general tightening procedure.

6.4.3.1 Intuition Through Examples

Single Agent: Simple Slack Removal Figure 6.6 shows the simplest case—a single agent with some large delays.

With maximum constant $M = 10$, any delay larger than $M + 1 = 11$ contains slack. Here's why we can reduce it:

- **Clock values**: All guards compare clocks against constants $\leq M = 10$.
- **Clock regions**: For verification purposes, we only need to distinguish:
  1. Clock values in $\{0, 1, 2, \ldots, M\}$ (individually)
  2. Clock values $> M$ (all equivalent from the guard's perspective)
- **Key insight**: After time $M + 1$, any clock that hasn't been reset will have value $> M$. Whether its value is 15, 20, or 100 doesn't matter—it will satisfy the same set of guards (those with $x > c$ for $c \leq M$, and fail those with $x \leq c$ or $x = c$).
- **Reduction**: In trace Tr_1:

- The delay from a_1 to a_2 is $15 = 11 + 4$. We can reduce it to 11.
- The delay from a_3 to a_4 is $20 = 11 + 9$. We can reduce it to 11.
- The delay from $\perp$ to a_1 is $2 \leq 11$, so we keep it.
- The delay from a_2 to a_3 is $3 \leq 11$, so we keep it.

- **Preservation**: The tightened trace $\overline{\mathsf{Tr}}_1$ has the same enabling of outcomes as Tr_1, because at each event:
  - Clocks with value $\leq M$ have the same exact value
  - Clocks with value $> M$ still have value $> M$ (just a smaller value $> M$)

  This gives us the tightened trace $\overline{\mathsf{Tr}}_1$ shown at the bottom of Figure 6.6.

Multiple Agents: Synchronization Constraints Now consider two agents with a time-synchronizing event. Figure 6.7 shows why this is more subtle.

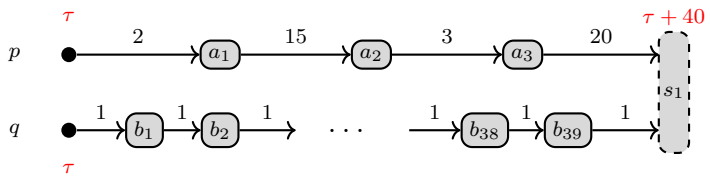


FIGURE 6.7: Two-agent trace with $M = 10$. The synchronization at s_1 prevents tightening, even though p has large delays.

Both agents start at local time τ. Process p has slack (delays of 15 and 20, both > 11), but we *cannot* reduce them! Here's why:

- Agent p reaches s_1 at time $\tau + 40 = \tau + 2 + 15 + 3 + 20$
- Agent q reaches s_1 at time $\tau + 40 = \tau + 1 + 1 + \cdots + 1$ (40 events, each 1 unit apart, but only a few shown)
- Since s_1 is time-synchronizing, we must have $\tau_p(s_1) = \tau_q(s_1)$
- If we reduce p's delays (say, to 11 each), p reaches s_1 at time $\tau+2+11+3+11 = \tau + 27$
- But q still reaches s_1 at time $\tau + 40$
- Now $\tau_p(s_1) \neq \tau_q(s_1)$, violating the synchronization constraint!

The lesson: we can only reduce delays by an amount that *all* agents participating in future synchronizations can accommodate.

This trace Tr_2 is already tight—there's no slack we can safely remove.

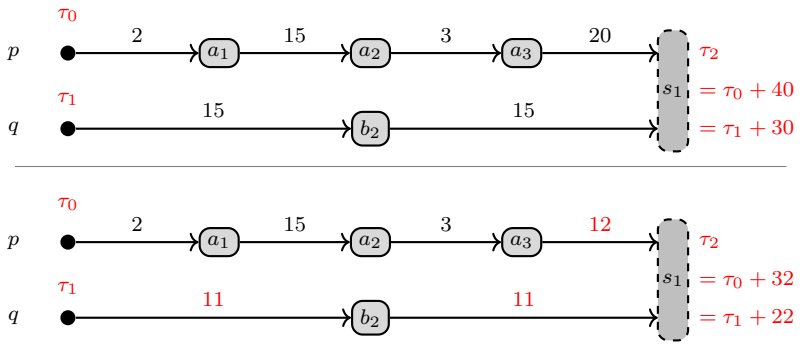


FIGURE 6.8: Above: Trace Tr_3 with $M = 10$. Agent q has less slack than p. Below: Tightened trace $\overline{\mathsf{Tr}}_3$ reduces both by the minimum slack (8 units).

Multiple Agents with Different Slacks Figure 6.8 shows a more interesting case where both agents have slack, but different amounts.

Let's compute the slack for each agent independently:

- **Agent p's slack**:
  - Edge $a_1 \to a_2$: delay 15, slack $= 15 - 11 = 4$
  - Edge $a_3 \to s_1$: delay 20, slack $= 20 - 11 = 9$
  - Total slack: $4 + 9 = 13$
- **Agent q's slack**:
  - Edge $\bot_q \to b_2$: delay 15, slack $= 15 - 11 = 4$
  - Edge $b_2 \to s_1$: delay 15, slack $= 15 - 11 = 4$
  - Total slack: $4 + 4 = 8$

Now here's the crucial point: if we reduce by more than 8 units, q won't be able to reach the synchronization time. So we reduce by exactly 8:

- In the original trace:
  - $\tau_p(s_1) = \tau_0 + 40$
  - $\tau_q(s_1) = \tau_1 + 30$
  - Synchronization requires: $\tau_0 + 40 = \tau_1 + 30$, so $\tau_1 = \tau_0 + 10$
- After reducing by 8:
  - $\tau_p(s_1) = \tau_0 + 32$ (reduced from 40)
  - $\tau_q(s_1) = \tau_1 + 22$ (reduced from 30)
  - Still synchronized: $\tau_0 + 32 = \tau_1 + 22$ when $\tau_1 = \tau_0 + 10$ ✓

The tightened trace $\overline{\mathsf{Tr}}_3$ shown in the bottom of Figure 6.8 achieves this by:

- Reducing p's delay from a_3 to s_1 from 20 to 12 (saves 8)
- Reducing q's delays from 15 to 11 each (saves $4 + 4 = 8$)

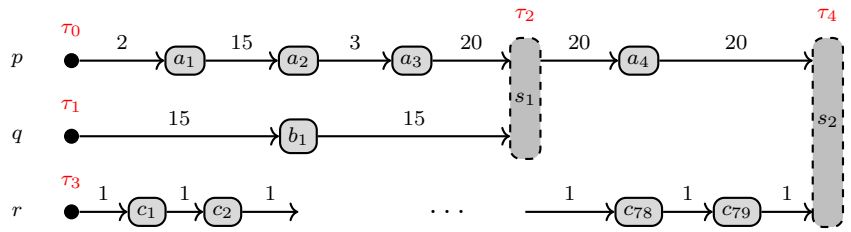


FIGURE 6.9: Trace Tr_4 with $M = 10$. Despite slack in p and q before s_1, the trace is tight because r has no slack before s_2.

Future Synchronizations Constrain Present Slacks Our final example (Figure 6.9) reveals perhaps the most subtle aspect of tightening: slack can be limited by *future* synchronizations, not just past ones.

This trace extends Tr_3 with a second synchronization s_2 between p and r. Let's analyze it:

- **Before** s_1: Just like in Tr_3:
  - p has slack 13 (from delays 15 and 20)
  - q has slack 8 (from two delays of 15)
- **Between** s_1 **and** s_2:
  - p has large delays (20 twice), slack $= (20 - 11) + (20 - 11) = 18$
  - r has only small delays (many 1's), no slack at all!
- **The critical constraint**: At s_2, we need $\tau_p(s_2) = \tau_r(s_2)$.
  - If we reduce any of p's delays, $\tau_p(s_2)$ decreases
  - But r has no slack—we cannot reduce $\tau_r(s_2)$
  - Therefore, we cannot reduce p's delays at all!
- **Backward propagation**: This constraint propagates backward through s_1:
  - We can't reduce p's delays to s_2
  - At s_1, p and q must synchronize
  - So we also can't reduce q's delays to s_1 (otherwise s_1 wouldn't synchronize)

Even though p and q have apparent slack before s_1, the trace Tr_4 is already tight! The future synchronization s_2 constrains what we can do now.

6.4.3.2 The Slack Graph

The examples above motivate our formal definition of slack. The key idea is to build a graph that captures both:

1. How much each edge can be reduced (*forward* information about available slack)
2. How future synchronizations constrain present reductions (*backward* information via synchronization points)

What "slack" measures. For each agent, every local edge $e \xrightarrow{\theta}_p e'$ can be written as

$$\theta = \underbrace{\min(\theta, M{+}1)}_{\text{“relevant” part}} + \underbrace{\max(0, \theta - (M{+}1))}_{\text{slack}},$$

where the relevant part is the only part that guards can ever observe. Tightening is the process of removing slack wherever possible, while still respecting future time-synchronizations.

Definition 6.8 (Slack Graph)**.** Given a trace Tr with maximum constant M, its *slack graph* $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$ is constructed as follows:

1. **Start with the trace**: Begin with all vertices and edges from Tr
2. **Compute slack on each edge**: For every edge $e \xrightarrow{\theta}_p e'$ in Tr:

$$\text{slack} = \begin{cases} 0 & \text{if } \theta \leq M+1 \\ \theta - (M+1) & \text{if } \theta > M+1 \end{cases}$$

   Replace the label θ with this slack value.

3. **Add back edges**: For every agent p and every pair of time-synchronizing events e_i, e_j with $p \in dom(e_i) \cap dom(e_j)$:
   - If $e_i(\rightarrow_p)^+ e_j$ (meaning e_i occurs before e_j in p's timeline)
   - Add a back edge $e_j \xrightarrow{0}_p e_i$ with weight 0

What problem is $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$ solving? (difference constraints / potentials). The slack graph is not a heuristic device: it encodes a system of *difference constraints* whose solutions correspond exactly to the globally consistent ways of shaving slack.

Fix a trace Tr and write $\tau(e)$ for the timestamp of an event occurrence e. A *tightening* chooses, for each event e, a reduction amount $\delta(e) \geq 0$ and sets

$$\overline{\tau}(e) \;:=\; \tau(e) - \delta(e).$$

Intuitively, $\delta(e)$ measures how far we shift e to the left. For time-synchronizing events s, the value $\delta(s)$ must be the *same* for all participating agents, because $\overline{\tau}_p(s) = \overline{\tau}_q(s)$ must still hold after tightening.

Now consider a forward edge $e \xrightarrow{\theta}_p e'$ of the trace. Only the part of θ above $M{+}1$ is "removable": the available slack on this edge is $\mathrm{slack}(e \xrightarrow{\theta}_p e') = \max(0, \theta - (M{+}1))$. If we reduce the timestamp of e' by $\delta(e')$ and of e by $\delta(e)$, then the local delay on this edge is reduced by exactly $\delta(e') - \delta(e)$. Hence feasibility imposes the inequality

$$\delta(e') - \delta(e) \;\leq\; \mathrm{slack}(e \xrightarrow{\theta}_p e').$$

Rearranging gives a standard difference constraint:

$$\delta(e') \;\leq\; \delta(e) \;+\; \mathrm{slack}(e \xrightarrow{\theta}_p e').$$

This is precisely what the *weighted forward edges* of $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$ encode.

Back edges encode the second kind of feasibility constraint: if $s \prec_p s'$ are consecutive time-synchronizing events on agent p, then tightening cannot move the earlier anchor more than the later one (otherwise some intermediate local delay would become negative). Formally this gives

$$\delta(s) \;\leq\; \delta(s'),$$

which is exactly the constraint encoded by a back edge $s' \xrightarrow{0}_p s$ in $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$.

Intended meaning of $\sigma(s)$. For a time-synchronizing event s, define

$$\sigma(s) \;:=\; \min_{p \in dom(s)} \mathrm{dist}_{\mathsf{Slack\text{-}Graph}(\mathsf{Tr})}(\bot_p, s),$$

i.e. the minimum (over sources $\bot_p$) of the shortest-path distance to s in $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$. The intended meaning is:

> $\sigma(s)$ is the *maximum* amount by which the timestamp of s can be reduced while keeping all guards and all time-synchronizations satisfiable.

Why shortest paths compute the maximum reduction. Every directed path π from some $\bot_p$ to s in $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$ yields (by summing the edge inequalities along π) an upper bound of the form $\delta(s) \leq w(\pi)$ for *every feasible tightening* δ. Therefore $\delta(s)$ is bounded by the *minimum* of these upper bounds, i.e. by the shortest-path distance. Conversely, setting $\delta(e)$ to be the shortest-path distance from the set of sources $\{\bot_p\}$ to e satisfies all edge inequalities of $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$ (the usual potential property of shortest-path distances), hence gives a feasible

tightening that achieves $\delta(s) = \sigma(s)$. This is the sense in which $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$ is a potentials construction: it computes the componentwise-maximal feasible reduction.

Why back edges are needed. Slack is not only constrained by the past. If an agent p participates in two time-synchronizing events $s \prec s'$, then shifting s' earlier may force s earlier as well, because the intermediate local timeline of p must remain nonnegative. Back edges encode exactly this "future constraint propagates backwards" phenomenon, and allow us to compute globally consistent time reductions by a shortest-path style computation.

Understanding the Construction Let's see why each component is necessary:

- **Slack values**: An edge with slack 0 means "this delay cannot be reduced" (it's already at the minimum safe value $\leq M + 1$). An edge with slack 5 means "we could reduce this delay by up to 5 units while maintaining clock values $> M$ where needed."
- **Back edges**: These propagate constraints from future synchronizations back to earlier events. In trace Tr_4, the back edge from s_2 to s_1 ensures that we recognize: "if we can't reduce delays to s_2, we also can't reduce delays to s_1."

Example: Slack Graph of Tr_4 Let's construct $\mathsf{Slack\text{-}Graph}(\mathsf{Tr}_4)$ step by step:

1. **Forward edges with slack**:

$$\begin{aligned}
\bot_p \xrightarrow{0}_p a_1 &\quad \text{(delay 2, slack 0)}\\
a_1 \xrightarrow{4}_p a_2 &\quad \text{(delay 15, slack 4)}\\
a_2 \xrightarrow{0}_p a_3 &\quad \text{(delay 3, slack 0)}\\
a_3 \xrightarrow{9}_p s_1 &\quad \text{(delay 20, slack 9)}\\
s_1 \xrightarrow{9}_p a_4 &\quad \text{(delay 20, slack 9)}\\
a_4 \xrightarrow{9}_p s_2 &\quad \text{(delay 20, slack 9)}\\
\bot_q \xrightarrow{4}_q b_1 &\quad \text{(delay 15, slack 4)}\\
b_1 \xrightarrow{4}_q s_1 &\quad \text{(delay 15, slack 4)}\\
\bot_r \xrightarrow{0}_r c_1, c_1 \xrightarrow{0}_r c_2, \ldots, c_{79} \xrightarrow{0}_r s_2 &\quad \text{(all delays 1, all slack 0)}
\end{aligned}$$

2. **Back edges**:
   - $s_2 \xrightarrow{0}_p s_1$ (since $s_1 \rightarrow_p^+ s_2$ and both are time-synchronizing for p)

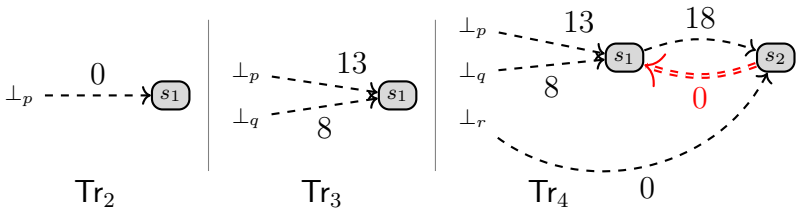


FIGURE 6.10: Shortest weighted paths to synchronizing events in slack graphs. The red double arrow in Tr_4 is a back edge that makes both $\sigma(s_1)$ and $\sigma(s_2)$ equal to 0.

This slack graph explicitly shows that there's a zero-weight path from $\perp_r$ through all the c events to s_2, and then via the back edge to s_1. This is why the trace is tight!

Definition 6.9 (Tight Traces)**.** A trace Tr is *tight* if for every time-synchronizing event s, there exists an agent $p \in dom(s)$ and a path of total weight 0 in $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$ from the initial event $\perp_p$ to s.

This definition formalizes our intuition: a trace is tight when we cannot reduce the timestamp of any synchronization point without violating some constraint.

Intuitively, a trace is tight if every time-synchronizing event already has an "explanation" path from some start that uses only relevant delays (each $\leq M{+}1$), i.e. there is no remaining slack that could pull it further to the left.

Computing Shortest Paths For a time-synchronizing event s and agent $p \in dom(s)$, let $\sigma_p(s)$ denote the weight of the shortest path from $\perp_p$ to s in $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$. Define:

$$\sigma(s) := \min_{p \in dom(s)} \{\sigma_p(s)\}$$

This value $\sigma(s)$ is the (componentwise) maximal feasible reduction of the timestamp of s.

Figure 6.10 shows these shortest paths for our example traces:

- **Trace Tr_2**: The path from $\perp_p$ to s_1 through the 39 b events (each with delay 1, slack 0) has weight 0. So $\sigma(s_1) = 0$ and the trace is already tight.
- **Trace Tr_3**:
  - From $\perp_p$: $\perp_p \xrightarrow{0} a_1 \xrightarrow{4} a_2 \xrightarrow{0} a_3 \xrightarrow{9} s_1$, total weight 13
  - From $\perp_q$: $\perp_q \xrightarrow{4} b_2 \xrightarrow{4} s_1$, total weight 8
  - So $\sigma(s_1) = \min(13, 8) = 8$

  We can reduce timestamps by 8 units, as shown in Figure 6.8.
- **Trace Tr_4**:

- The shortest path to s_2 from $\perp_r$ has weight 0 (through all the small delays)
- The back edge makes the shortest path to s_1 also 0
- So $\sigma(s_1) = \sigma(s_2) = 0$, and the trace is tight

6.4.3.3 Properties of the σ Function

Before presenting the tightening algorithm, we need to understand how σ values relate across consecutive synchronizations.

Lemma 6.10 (Monotonicity of σ)**.** *Let $p \in P$ and let s, s' be time-synchronizing events for p such that $s(\rightarrow_p)^+ s'$ (meaning s occurs before s' in p's timeline) and there is no other time-synchronizing event for p between s and s'. Then:*

1. *$\sigma(s) \leq \sigma(s')$ (non-decreasing along agent timelines)*
2. *$\sigma(s') \leq \sigma(s) + \theta_p$ where θ_p is the total slack for p between s and s'*

Proof. **Part 1**: By construction of $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$, there is a back edge $s' \xrightarrow{0}_p s$ with weight 0. Therefore:

$$\sigma(s) \leq \sigma(s') + 0 = \sigma(s')$$

Since $\sigma(\cdot)$ is defined as the minimum shortest path distance, and we have a path from any $\perp_q$ to s' that can be extended via the back edge to s, we get $\sigma(s) \leq \sigma(s')$.

Part 2: Consider a shortest path from some $\perp_q$ to s witnessing $\sigma(s)$. We can extend this path:

- From s to s' along p's timeline, accumulating slack θ_p
- This gives a path from $\perp_q$ to s' of weight $\sigma(s) + \theta_p$
- By definition of $\sigma(s')$ as the minimum, we have $\sigma(s') \leq \sigma(s) + \theta_p$

□ □

Why This Matters This lemma tells us that we can always "pay for" a reduction at a later synchronization s' using slack accumulated between s and s', as long as $\sigma(s) = 0$ (the earlier synchronization is already tight). This is the key to our tightening algorithm.

6.4.3.4 The Tightening Algorithm

We can now state the main result about tightening:

Lemma 6.11 (Tightening Lemma)**.** *From every trace Tr, we can construct a trace $\overline{\mathsf{Tr}}$ such that:*

1. ***Reduced timestamps***: *For every time-synchronizing event s:*

$$\overline{\tau}(s) = \tau(s) - \sigma(s)$$

where $\tau(s)$ and $\overline{\tau}(s)$ are the timestamps of s in Tr and $\overline{\mathsf{Tr}}$ respectively.

2. ***Preserved small delays***: *For every edge $e \xrightarrow{t}_p e'$ in Tr with $t \leq M + 1$:*

$$e \xrightarrow{t}_p e' \text{ in } \overline{\mathsf{Tr}} \text{ as well}$$

(Delays at most $M + 1$ are unchanged)

3. ***Preserved validity***: *For any configuration (C, v) matching the initial timestamps of Tr:*

$$\rho[(C, v), \mathsf{Tr}] \text{ is a valid run} \quad \Leftrightarrow \quad \rho[(C, v), \overline{\mathsf{Tr}}] \text{ is a valid run}$$

Proof sketch. The algorithm proceeds by distributing the reduction $\sigma(s)$ across the edges leading to each synchronization s:

1. **Identify reductions**: For each time-synchronizing event s, compute $\sigma(s)$ using shortest paths in $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$.

2. **Distribute reductions**: For each agent $p \in dom(s)$, reduce the delays on edges with non-zero slack along the path from $\perp_p$ (or from the previous synchronization) to s, distributing the total reduction of $\sigma(s)$.

3. **Update timestamps**: Recompute all timestamps based on the new delays.

Why it preserves validity: At each event, we ensure:

- Clocks with value $\leq M$ retain exactly the same value (since we only reduce delays $> M + 1$, and by at most their slack)

- Clocks with value $> M$ still have value $> M$ (possibly a smaller value, but still $> M$)

Since all guards compare against constants $\leq M$, the set of enabled transitions remains the same.

Why synchronizations still work: By Lemma 6.10, reducing by $\sigma(s)$ preserves the property that all agents in $dom(s)$ reach the (reduced) timestamp simultaneously. □ □

Proof. We construct $\overline{\mathsf{Tr}}$ by assigning new timestamps to time-synchronizing events and then deriving induced local delays.

For each time-synchronizing event s, define the reduction amount $\delta(s) := \sigma(s)$, where $\sigma(s)$ is the minimum slack-weight of a path from any $\bot_p$ to s in Slack-Graph(Tr). Set

$$\overline{\tau}(s) := \tau(s) - \delta(s).$$

By the back-edge monotonicity discussed above, these reductions are consistent along each agent timeline: if $s \prec_p s'$ are consecutive synchronizing events of p, then $\delta(s) \leq \delta(s')$, so no local delay becomes negative.

Now traverse each agent linearly from left to right and define each new local delay by shaving slack exactly by the change in $\delta(\cdot)$ between the surrounding synchronizing anchors (and leaving delays $\leq M{+}1$ untouched). This ensures the second bullet: edges with delay $\leq M{+}1$ are preserved verbatim.

Finally, guard preservation follows from Lemma 6.7: at every event, each clock either keeps its value (if it is $\leq M$) or remains $> M$ (if it was already $> M$), and all time-synchronizing equalities are preserved by construction. Hence $\rho[(C, v), \mathsf{Tr}]$ exists iff $\rho[(C, v), \overline{\mathsf{Tr}}]$ exists. □ □

6.4.3.5 Bounding Tight Traces

Now we can bound the timestamps in tight traces:

Lemma 6.12 (Tight Trace Bound)**.** *Let Tr be a tight trace and let* $\tau_m := \max_{p \in P} \tau_p(\bot_p)$. *Then for every time-synchronizing event s there exists an agent $p \in dom(s)$ and a path π from $\bot_p$ to s in Slack-Graph(Tr) of total weight 0. Let $k_{\text{fwd}}(\pi)$ be the number of* forward *edges of π (i.e. edges coming from the original trace; excluding the added back edges). Then*

$$\tau(s) \;\leq\; \tau_m \;+\; k_{\text{fwd}}(\pi) \cdot (M + 1).$$

Proof. Because Tr is tight, there exist $p \in dom(s)$ and a path π in Slack-Graph(Tr) from $\bot_p$ to s of total weight 0. Every forward edge of Slack-Graph(Tr) carries weight equal to the slack of some trace delay segment. Hence, along π, every forward edge must have slack 0, which means the corresponding original delay is at most $M{+}1$.

Crucially, back edges represent *constraints* (monotonicity between synchronizing anchors) and do not correspond to any elapsed time. Therefore the time elapsing between $\bot_p$ and s is bounded by counting only forward edges on π: each such edge contributes at most $M{+}1$ time units. Thus

$$\tau(s) \;=\; \tau_p(\bot_p) \;+\; (\text{time elapse from } \bot_p \text{ to } s) \;\leq\; \tau_m \;+\; k_{\text{fwd}}(\pi) \cdot (M + 1),$$

as required.

Remark. In what follows, when we bound time using witness paths in Slack-Graph(Tr), we always count only forward (time) edges; back edges are purely constraint edges

of zero time.

Since the path has total weight 0, every edge has slack 0. This means:

- Each edge $e_i \to_p e_{i+1}$ in π corresponds to a delay $\leq M + 1$ in Tr
- The total time from $\bot_p$ to s is at most $k \cdot (M + 1)$

Therefore:

$$\tau(s) = \tau_p(\bot_p) + (\text{time from } \bot_p \text{ to } s) \leq \tau_m + k \cdot (M + 1)$$

as required. □ □

Why This Matters This lemma says that in a tight trace, the timestamp at any synchronization s is bounded by:

- The maximum initial time τ_m
- Plus $(M + 1)$ times the *number of events* in the shortest zero-weight path to s

However, this bound depends on the specific trace—different traces might have different path lengths k. To prove bounded drift for an entire LTN, we need a *uniform* bound across all possible traces. This is where connectedness enters the picture.

6.4.4 Tight Runs of Connectedly-Communicating LTNs are Drift-Bounded

We now prove that for connectedly-communicating LTNs, there is a uniform bound k (depending only on the LTN structure, not on individual runs) such that all tight runs have drift bounded by $k \cdot (M + 1)$.

Proof map. We prove a *uniform* drift bound for tight runs of a connectedly-communicating LTN. The difficulty is that Lemma 6.12 bounds timestamps by the *length of the path* in the slack graph, which a priori can grow with the run length.

To remove this dependence, we combine two finiteness/structure arguments:

1. **Finiteness of control:** by chopping a run into blocks of length N^* (the number of nodes in the marking graph plus one), each block contains a control loop.
2. **Connectivity inside loops:** in each such loop, time-synchronizations connect all agents; therefore, within $|P|$ consecutive blocks, information about "how large time has become" propagates to *all* agents.

This yields two monotonicity phenomena: (i) the minimum timestamp in a later block eventually overtakes the maximum timestamp in an earlier block, and (ii) tightness prevents the maximum timestamp inside a block from being much larger than its block-minimum.

6.4.4.1 Proof Strategy

The proof has three main ideas:

1. **Chunking**: Break any run into chunks of size N^* (the size of the marking graph), ensuring each chunk contains at least one complete loop of the marking graph.

2. **Local bounds**: Show that within each chunk, the difference between maximum and minimum timestamps is bounded.

3. **Connectedness propagation**: Use the connectedly-communicating property to show that constraints propagate from one agent to all others within $|P|$ iterations of any loop.

Let's work through this carefully.

6.4.4.2 Setting Up: Chunking the Run

Fix a connectedly-communicating LTN $\mathcal{N}$ for this entire section. Consider a tight run:

$$\rho := (C_0, v_0) \xrightarrow{(\Delta_1, e_1)} (C_1, v_1) \cdots (C_\ell, v_\ell)$$

Technical assumption: For convenience, assume the final event $e_{\ell-1}$ is a global synchronization (all agents participate and it's time-synchronizing). If not, we can add a dummy global synchronization at the end—this doesn't affect the drift bound.

Let $N^* = |\operatorname{MG}(\mathcal{N}_{\mathsf{untime}})| + 1$ be one more than the number of markings in the marking graph. We partition ρ into chunks of N^* events each:

$$\begin{aligned}
\rho_1 &:= (C_0, v_0) \xrightarrow{(\Delta_1, e_1)} \cdots \xrightarrow{(\Delta_{N^*}, e_{N^*})} (C_{N^*}, v_{N^*}) \\
\rho_2 &:= (C_{N^*}, v_{N^*}) \xrightarrow{(\Delta_{N^*+1}, e_{N^*+1})} \cdots \xrightarrow{(\Delta_{2N^*}, e_{2N^*})} (C_{2N^*}, v_{2N^*}) \\
&\vdots \\
\rho_L &:= (\text{remaining events, at most } N^*)
\end{aligned}$$

Key property: Since ρ_i contains N^* events (except possibly the last chunk), and there are only $N^* - 1$ distinct markings, by the pigeonhole principle, some marking must repeat. Therefore, ρ_i contains at least one complete cycle of the marking graph.

Fixing a witness loop inside each chunk. For each chunk ρ_i with $i \geq 2$, fix once and for all indices

$$\alpha_i < \beta_i$$

such that the marking at step α_i equals the marking at step β_i (existence by pigeonhole). Let λ_i be the corresponding subrun (the *loop segment*) consisting of the events executed from α_i to $\beta_i - 1$. This segment realizes a loop of $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$; denote by G_i its communication graph. By connectedly-communicating, G_i is connected.

6.4.4.3 Trace Notation

Let Tr be the trace of the entire run ρ. For each chunk ρ_i, let Tr_i be the corresponding trace segment:

- Initial events of Tr_i are denoted $\bot_p^i$ for each agent p
- The timestamp $\tau_p(\bot_p^i) = v_{(i-1)\cdot N^*}(t_p)$ (the local time at the start of chunk i)
- For $i > 1$, the maximal event of p in Tr_{i-1} has the same timestamp as $\bot_p^i$ in Tr_i

For each chunk i, define:

$$\tau_i^{max} = \max\{\tau_p(e) : e \in \mathsf{Tr}_i, p \in dom(e)\} \quad \text{(maximum timestamp in chunk } i\text{)}$$
$$\tau_i^{min} = \min\{\ \tau_p(\bot_p^i) : p \in P\} \quad \text{(minimum initial timestamp in chunk } i\text{)}$$

Our goal is to show $\tau_i^{max} - \tau_i^{min}$ is uniformly bounded.

6.4.4.4 Key Technical Claims

We'll prove the bound through a series of claims that exploit connectedness.

A connectivity hopping lemma (informal but guiding). Fix a loop in $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$ and its communication graph G. Because G is connected, for any agent q and any set of agents $S \neq \emptyset$, there exists a path in G from q to some $p \in S$ of length at most $|P| - 1$. Concretely, along an execution of the loop, this means: by following at most $|P| - 1$ time-synchronizing events, we can move from "being in contact with q" to "being in contact with some agent in S". We will use this repeatedly with $S = dom(f)$ and $q \in dom(f')$.

By Lemma 6.4(2), every chunk whose loop communication graph is connected contains a time-synchronizing event crossing any nontrivial cut of agents. Combined with monotonicity of local time, this yields a clean induction showing that lower bounds on time propagate to all agents within at most $|P| - 1$ chunks.

Claim 6.13 (Forward Connectivity)**.** *Let f be an event in Tr_i and f' be an event in $\mathsf{Tr}_{i+|P|+1}$. Then there is a path of forward edges (not using back edges) from f to f' in $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$.*

Proof. The proof uses the connectedly-communicating property repeatedly. Let $j = i + |P| + 1$ and suppose $p_1 \in dom(f')$.

Case 1: If $p_1 \in dom(f)$, there's a path of $\rightarrow_{p_1}$ edges from f to f' (since both events are on p_1's timeline and f comes before f').

Case 2: Otherwise, we build a path through intermediate synchronizations:

- Each chunk $\rho_{i+1}, \rho_{i+2}, \ldots, \rho_j$ contains a loop of the marking graph
- By the connectedly-communicating property, each loop's communication graph is connected
- Starting from chunk ρ_{j-1}: Since p_1 participates in f' and chunk $j-1$ contains a loop, p_1 must synchronize with at least one other agent p_2 at some event $s_{j-1} \in \mathsf{Tr}_{j-1}$
- If either p_1 or p_2 is in $dom(f)$, we have a path $f \rightarrow s_{j-1} \rightarrow f'$ ✓
- Otherwise, continue to chunk $j-2$: By connectedness, there's an agent p_3 that synchronizes with p_1 or p_2 in chunk $j-2$ at event s_{j-2}
- We keep working backward through chunks until we find an agent in $dom(f)$ (this must happen within $|P|$ steps since there are only $|P|$ agents)

The key insight: within $|P|$ chunks, the connected communication graphs ensure every agent connects to every other agent through a chain of synchronizations. □ □

Claim 6.14 (Backward Connectivity)**.** *Let s be a time-synchronizing event in Tr_i and s' be a time-synchronizing event in $\mathsf{Tr}_{i+|P|+1}$. Then there is a path of back edges from s' to s in $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$.*

Proof. From Claim 6.13, there's a forward path from s to s'. Let $s_1, s_2, \ldots, s_\ell$ be the sequence of time-synchronizing events along this path (with $s_1 = s$ and $s_\ell = s'$).

By construction of $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$:

- There's a back edge from s_2 to s_1
- There's a back edge from s_3 to s_2
- $\vdots$
- There's a back edge from $s_\ell = s'$ to $s_{\ell-1}$

Composing these back edges gives a path (of back edges) from s' to s. □ □

Claim 6.15 (Minimum Time Propagation)**.** *For any $i > |P|$, let $i' = i - |P|$. Then*

$$\tau_{i'}^{max} \leq \tau_i^{min}.$$

Proof. Let $\theta := \tau_{i'}^{max}$. Choose an event e in $\mathsf{Tr}_{i'}$ and an agent $p_0 \in dom(e)$ such that $\tau_{p_0}(e) = \theta$. Since local time is nondecreasing for every agent, at the *start* of chunk $i' + 1$ we have

$$\tau_{p_0}(\bot_{p_0}^{i'+1}) \;\geq\; \theta.$$

For every chunk index $j \geq i' + 1$, define the set of agents that have already "caught up to θ" at the start of chunk j:

$$S_j \;:=\; \{\, p \in P \;:\; \tau_p(\bot_p^j) \geq \theta \,\}.$$

Then $S_{i'+1} \neq \emptyset$ because $p_0 \in S_{i'+1}$.

We claim that as long as $S_j \subsetneq PP$, the next chunk strictly grows this set:

$$S_{j+1} \supsetneq S_j.$$

Indeed, consider the fixed loop segment λ_j inside chunk j (as fixed above) and its communication graph G_j. Since G_j is connected and S_j is a nonempty proper subset, Lemma 6.4(2) yields a time-synchronizing event occurrence s inside λ_j with two agents $p \in S_j$ and $q \notin S_j$ participating. At the moment s fires in the run, time-synchronization gives $\tau_p(s) = \tau_q(s)$. But $\tau_p(s) \geq \tau_p(\bot_p^j) \geq \theta$ by monotonicity, hence $\tau_q(s) \geq \theta$ as well. Since time is nondecreasing, at the start of the next chunk we still have $\tau_q(\bot_q^{j+1}) \geq \theta$, so $q \in S_{j+1}$ and thus $S_{j+1} \supsetneq S_j$.

Therefore, starting from the nonempty set $S_{i'+1}$, after at most $|P|-1$ iterations we obtain $S_{i'+|P|} = P$. In particular, at the start of chunk $i = i' + |P|$ every agent has local time at least θ, so

$$\tau_i^{min} \;=\; \min_{p \in P} \tau_p(\bot_p^i) \;\geq\; \theta \;=\; \tau_{i'}^{max}.$$

□ □

Claim 6.16 (Within-Chunk Bound)**.** *Let f^i be any event in Tr_i with $q \in dom(f^i)$. There exists an initial event $\bot_{p_i}^i$ of Tr_i such that:*

$$\tau_q(f^i) \leq \tau_{p_i}(\bot_{p_i}^i) + \mathcal{O}(N^* \cdot |P|^2 \cdot (M+1))$$

Proof sketch. 1. Since Tr is tight, there's a zero-weight path in $\mathsf{Slack\text{-}Graph}(\mathsf{Tr})$ from some $\bot_p$ to any time-synchronizing event.

2. Pick a time-synchronizing event s^{i+1} in Tr_{i+1} that involves an agent from $dom(f^i)$ (exists by connectedness since chunk $i+1$ has a loop).

3. We have $\tau_q(f^i) \leq \tau(s^{i+1})$ (since f^i precedes s^{i+1}).

4. The zero-weight path to s^{i+1} might zig-zag through earlier chunks, but by Claim 6.14, any excursion beyond chunk $i + |P| + 2$ can be "short-circuited" using back edges.

5. Therefore, the path from the last initial event $\perp^i_{p_i}$ in Tr_i to s^{i+1} has at most $\mathcal{O}(N^* \cdot |P|^2)$ edges.

6. By Lemma 6.12, this gives the bound:

$$\tau(s^{i+1}) \leq \tau_{p_i}(\perp^i_{p_i}) + \mathcal{O}(N^* \cdot |P|^2 \cdot (M+1))$$

□ □

Bounding how far the witness path can wander. The tightness of Tr gives us a zero-slack path π from some initial node to s^{i+1}. A priori, π could run far to the right (into much later blocks) and then return using back edges. The role of Claim 2 is exactly to show that such far wandering is unnecessary: once π reaches a time-synchronizing event in a sufficiently later block, we can shortcut the suffix of π by jumping back through a chain of back edges (all of weight 0) to reach s^{i+1} without increasing slack weight. Therefore we may assume that a shortest zero-slack path to s^{i+1} only visits blocks up to a bounded horizon (here $i+|P|+2$).

6.4.4.5 Putting It All Together

Now we can prove the main theorem:

Proof of Main Theorem. Fix any chunk index $i > |P|+1$. Let $i' = i - |P| - 1$.

Step 1: By Claim 6.16, for the event e^i witnessing τ_i^{max}:

$$\tau_i^{max} \leq \tau_{p'}(\perp^{i'}_{p'}) + (|P|+1) \cdot \mathcal{O}(N^* \cdot |P|^2 \cdot (M+1))$$

for some agent p'. Here we've applied the claim iteratively $(|P|+1)$ times to bridge from chunk i back to chunk i'.

Step 2: By definition, $\tau_{p'}(\perp^{i'}_{p'}) \leq \tau_{i'}^{max}$.

Step 3: By Claim 6.15, $\tau_{i'}^{max} \leq \tau_i^{min}$.

Step 4: Combining:

$$\begin{aligned}
\tau_i^{max} - \tau_i^{min} &\leq \tau_{i'}^{max} + \mathcal{O}(N^* \cdot |P|^3 \cdot (M+1)) - \tau_i^{min} \\
&\leq \tau_i^{min} + \mathcal{O}(N^* \cdot |P|^3 \cdot (M+1)) - \tau_i^{min} \\
&= \mathcal{O}(N^* \cdot |P|^3 \cdot (M+1))
\end{aligned}$$

For chunks $i \leq |P|+1$, we can apply Claim 6.16 iteratively back to the initial chunk ρ_1 to get the same bound.

Step 5: In any configuration (C, v) reached during chunk i, the drift between any two agents is:

$$|v(t_p) - v(t_q)| \leq \tau_i^{max} - \tau_i^{min} = \mathcal{O}(N^* \cdot |P|^3 \cdot (M+1))$$

Since $N^* = |\mathrm{V}(\mathrm{MG}(\mathcal{N}_{\mathsf{untime}}))| + 1$, we obtain

$$K \;=\; \mathcal{O}\big(N^* \cdot |P|^3 \cdot (M+1)\big) \;=\; \mathcal{O}\big(|\mathrm{V}(\mathrm{MG}(\mathcal{N}_{\mathsf{untime}}))| \cdot |P|^3 \cdot (M+1)\big).$$

Using the bound $|\mathrm{V}(\mathrm{MG}(\mathcal{N}_{\mathsf{untime}}))| \leq 2^{|N|\cdot|P|}$ from the discussion above, this yields

$$K \;=\; \mathcal{O}\big(2^{|N|\cdot|P|} \cdot |P|^3 \cdot (M+1)\big).$$

□ □

From block bounds to drift bounds. Fix any configuration (C, v) occurring during the execution of block ρ_i. Every agent timestamp in this configuration lies between the minimum timestamp of block ρ_i (realized at some initial node $\perp_p^i$) and the maximum timestamp of block ρ_i (realized at some event of Tr_i). Hence for all agents p, q,

$$|v(t_p) - v(t_q)| \;\leq\; \tau_i^{max} - \tau_i^{min} \;\leq\; K,$$

where $K = \mathcal{O}(N^* \cdot |P|^3 \cdot (M+1))$ is independent of the run. This is exactly the drift-boundedness requirement.

6.4.5 Extensions and Variations

Weaker Connectedness Requirement The definition of connectedly communicating in [**?**] is actually weaker than ours. Their version would translate to our setting as follows:

> If agents p and q do not participate in any time-synchronization together in a cycle of $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$, then they will never time-synchronize in the future.

This naturally partitions agents at each marking into groups $\pi_1, \pi_2, \ldots, \pi_\ell$ where:

- Agents in different groups never synchronize
- Agents within the same group must form a connected communication graph

Our techniques extend to this weaker setting with some book-keeping:

- The partition can only refine as execution proceeds (groups can split but not merge)
- In any loop, the partition is stable
- We require bounded drift only *within each group* at each marking

- Claims 6.13–6.16 hold within each partition element

This would allow the LTN in Figure 6.1(b) to be included, since r would be in its own partition element (no drift requirement with respect to p and q).

Section **??** defined *connectedly-communicating* LTNs by requiring that in *every* loop of the marking graph of $\mathcal{N}_{\mathsf{untime}}$, the communication graph induced by time-synchronizing nodes is connected. This is a strong condition: it forces *all* agents to remain (directly or indirectly) time-coupled along *every* cyclic behaviour, even in situations where some agents may permanently stop synchronizing with others after a certain point.

A weaker and natural variant, already implicit in the concurrency literature, is to allow the system to *irreversibly split* into groups of agents that never time-synchronize with each other again. Intuitively, once the run enters a region of the marking graph where two groups have "lost synchronizing contact", they remain forever independent w.r.t. time-synchronization. This captures examples where an agent r may still interact with another agent q (perhaps via non-synchronizing nodes) but exerts no future time-synchronizing influence on q and hence on the rest of the system.

Synchronization classes as a monotone partition. Fix an LTN $\mathcal{N}$. Consider its untimed marking graph $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$. We associate with each marking C a partition $\Pi(C)$ of the agent set P into blocks

$$\Pi(C) \;=\; \{\pi_1, \ldots, \pi_\ell\},$$

intended to represent *synchronization classes* from C onwards: agents in different blocks will never participate together in a time-synchronizing node in any continuation.

Formally, one convenient way to define $\Pi(C)$ is as the coarsest partition induced by the following equivalence relation: for agents $p, q \in P$, we set $p \sim_C q$ if there exists a continuation from C that reaches a time-synchronizing node $n \in \mathit{Sync}$ with $\{p, q\} \subseteq \mathit{dom}(n)$. Let $\Pi(C)$ be the set of $\sim_C$-equivalence classes.

This construction enforces a monotonicity property: whenever $C \xrightarrow{n} C'$ in $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$, the partition can only *refine*:

$$\Pi(C') \text{ is a refinement of } \Pi(C).$$

Intuitively, progressing in the system may "break" future synchronizing connections (splitting a class), but cannot create a new synchronizing connection between previously separated classes without contradicting the definition of $\sim_C$.

Weak connected communication. We now relax connected communication *within* each synchronization class.

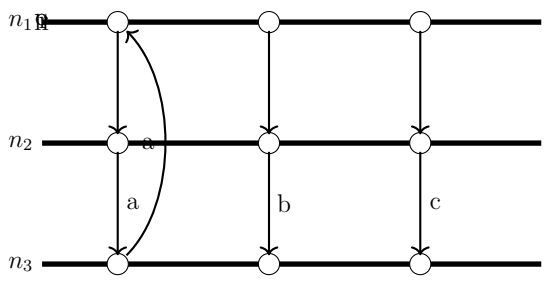


FIGURE 6.11: Three synchronized chains illustrating independent actions a, b, c on timelines p, q, r. The action 'a' features a time-disconnected recursive loop, leading to unbounded local time accumulation for agent r.

Definition 6.17 (Weakly connectedly-communicating LTN)**.** An LTN $\mathcal{N}$ is *weakly connectedly-communicating* if for every loop $C = C_1 \xrightarrow{n_1} C_2 \xrightarrow{n_2} \cdots \xrightarrow{n_k} C_{k+1} = C$ in $\mathrm{MG}(\mathcal{N}_{\mathsf{untime}})$, letting $\Pi(C) = \{\pi_1, \ldots, \pi_\ell\}$, the following holds: for every block π_i, the communication graph of the loop restricted to vertices π_i (i.e. edges induced by time-synchronizing nodes whose domains intersect π_i) is connected.

This condition is strictly weaker than Definition **??**: it permits loops in which the full-agent communication graph is disconnected, as long as every *permanent synchronization class* remains internally connected.

Blockwise drift-boundedness. Since different blocks are, by construction, forever non-synchronizing with each other, it is not meaningful to demand a uniform drift bound between agents in different blocks. Instead, we require bounded drift *within* each block.

Definition 6.18 (Blockwise drift bound)**.** Let (C, v) be a configuration and $\Pi(C) = \{\pi_1, \ldots, \pi_\ell\}$. We say (C, v) is *k-blockwise drift-bounded* if for every block π_i and all $p, q \in \pi_i$ we have $|v(t_p) - v(t_q)| \leq k$. An LTN is *k-blockwise drift-bounded* if every run can be tightened to a run whose every configuration is k-blockwise drift-bounded.

Why the proof technique still applies. The tightening construction (via slack graphs and back-edges between successive time-synchronizing events of the same agent) is agnostic to the global connectivity requirement: it operates on a given trace and preserves feasibility of guards and synchronizations.

The only place where connected communication was used essentially is in propagating bounds between arbitrary agents by chaining synchronizations across a loop. In the weak setting, this propagation is performed *within each block π_i separately*: over any sufficiently long segment that contains a loop, agents in a fixed block π_i still form a connected communication graph, hence the same "synchronization-chaining" argument bounds the spread between the minimum

and maximum timestamps *inside* π_i. Agents in different blocks are simply ignored, because no future synchronization constraints will ever relate their reference clocks.

Theorem 6.19 (Weak connected communication implies blockwise bounded drift)**.** *Every weakly connectedly-communicating LTN is k-blockwise drift-bounded for*

$$k \in O\big(|N| \cdot |A|^3 \cdot (M+1)\big)\,,$$

where $|N|$ is the number of nodes, $|A|$ the number of agents, and M the maximum constant appearing in guards.

Proof sketch (adaptation). The proof follows the same template as for Theorem **??**. We tighten an arbitrary run to a tight run using the slack-graph construction. Then we chunk the run into segments long enough to guarantee a repeated marking.

Fix a chunk starting at marking C and consider the partition $\Pi(C)$. For a block $\pi \in \Pi(C)$, restrict attention to the sub-trace induced by agents in π and to time-synchronizing events whose domains intersect π. By Definition 6.17, every loop encountered inside the chunk induces a connected communication graph on π, so the same chaining argument bounds the difference between the maximum and minimum timestamps attained by agents in π within the chunk by a uniform function of $|N|$, $|A|$, and $(M+1)$. Since $\Pi(\cdot)$ only refines along the run, later chunks can only split blocks further, which can only weaken the requirement (boundedness is required on smaller sets). Stitching the bounds across chunks yields a uniform k kfor all blocks throughout the run. □

Algorithmic and modelling consequences. This fragment strictly enlarges the applicability of the decidability result: it admits systems where some agents permanently stop synchronizing with others (e.g. an agent that continues to interact only through non-synchronizing nodes), while still guaranteeing that within each permanent synchronization class, clock drift can be bounded after tightening. For the reachability decision procedure, the only additional bookkeeping is to track (or conservatively approximate) the current partition $\Pi(C)$ in the abstract state space and to enforce region-equivalence constraints *per block* rather than globally.

6.5 Conclusion

We have developed a comprehensive theory of connectedly-communicating local-timed negotiations, proving that this fragment has decidable reachability via bounded drift.

6.5.1 Summary of Contributions

1. **Tightening algorithm** (Section 6.4.3): We formalized the notion of slack in distributed time-synchronizing systems through:
   - Slack graphs that capture both available time reductions and synchronization constraints
   - A tightening procedure that systematically reduces unnecessary delays
   - The notion of tight traces as a normal form for behaviors
2. **Bounded drift theorem** (Section 6.4.4): We proved that connectedly-communicating LTNs have drift bounded by $\mathcal{O}(|N| \cdot |P|^3 \cdot (M+1))$ through:
   - Chunking runs into segments containing complete marking-graph cycles
   - Exploiting the connected communication graphs to propagate timing constraints
   - Showing that tight runs have uniformly bounded timestamp differences
3. **Decidability**: As a corollary, reachability is decidable for connectedly-communicating LTNs via region automaton construction on the set of tight untimed behaviors.

6.5.2 Future Directions

Efficient Verification While we have established decidability, the region automaton construction has well-known efficiency limitations. LTNs are attractive because they allow concurrency while restricting time-synchronizations to specific user-designated nodes. This suggests several research directions:

1. **Zone-based algorithms**: Can we develop partial-order reduction techniques that exploit the independence structure of LTNs, similar to what's been done for networks of timed automata?
2. **Compositional methods**: The modular structure of LTNs (multiple agents, multiple nodes) suggests compositional verification approaches. Can we verify properties of individual agents or sub-networks and compose the results?
3. **Symbolic methods**: BDD-based or SMT-based approaches might handle the bounded drift region space more efficiently than explicit enumeration.

Practical Applications LTNs model distributed systems where agents coordinate at specific points but otherwise act independently. Promising application domains include:

- **Cyber-physical systems**: Sensors that periodically synchronize with a central controller

- **Distributed protocols**: Processes that occasionally synchronize via barriers or rendezvous
- **Workflow systems**: Business processes with multiple parties that coordinate at specific milestones

Developing case studies in these domains would help validate the utility of the connectedly-communicating fragment and guide further theoretical development.

Relaxing Connectedness Can we identify other syntactic restrictions beyond connectedness that ensure bounded drift? Some possibilities:

- Acyclic communication structures (no cyclic dependencies between synchronizations)
- Bounded-depth hierarchies (limits on the "distance" between synchronizing agents)
- Probabilistic or timed guarantees (synchronization occurs with sufficient frequency)

Generalizations Our results assume discrete time and integer-valued constants. Extensions to dense time and multi-clock systems remain open. Additionally, combining LTNs with other formalisms (e.g., pushdown systems, probabilistic choice) could broaden applicability.

6.5.3 Closing Remarks

This work shows that local-timed negotiations admit a non-trivial decidable fragment through the lens of connectedness. The key insight—that connected communication graphs prevent unbounded drift—bridges ideas from distributed systems, timed automata, and controller synthesis.

The tightening framework we developed (Section 6.4.3) has independent interest as a general technique for analyzing distributed time-synchronizing systems. The slack graph construction and backward edge propagation might apply to other formalisms beyond LTNs.

More broadly, we hope this work motivates further study of LTNs as a sweet spot between expressiveness and tractability for modeling distributed real-time systems. The ability to selectively designate time-synchronizing points, combined with decidable verification, makes LTNs an attractive target for both theoretical investigation and practical application.

Chapter 7

Conclusion and Future Directions

7.1 Conclusion

Concurrent systems with quantitative timing constraints are pervasive in practice—from distributed protocols and embedded systems to cyber-physical infrastructures—yet modeling them presents fundamental tensions between expressiveness, compositionality, and decidability. This thesis introduced *local-timed negotiations (LTNs)* as a unifying formalism that addresses these tensions by combining two complementary perspectives: the interaction-centric view of negotiations, which makes concurrency and causal structure explicit, and the quantitative timing framework of timed automata, which enables precise reasoning about real-time behavior.

The key innovation of LTNs lies in assigning clocks and timing constraints *locally* to agents and interactions rather than globally. This locality principle captures timing phenomena—such as bounded desynchronization, partial observation of delays, and interaction-dependent progress—that are difficult to express cleanly in either negotiations or timed automata alone. At the same time, unrestricted locality leads quickly to undecidability, subsuming the power of networks of timed automata. The central question motivating this thesis is therefore: *which natural restrictions on timing, synchronization, and structure suffice to recover decidability while preserving expressive power?*

Contributions and Main Results

The thesis answers this question through a systematic exploration of the decidability landscape for LTNs, organized along three axes: synchronization patterns, clock divergence, and structural constraints.

Foundations. After establishing the formal semantics of LTNs in Chapter **??**, with careful attention to local clocks, joint occurrences, and monotonicity, the thesis positions LTNs as a strict generalization of untimed negotiations and a struc-

tured alternative to product constructions over timed automata. This foundation clarifies how timing information propagates through interactions and provides the technical basis for subsequent decidability results.

Extreme synchronization regimes. Chapter **??** analyzes two opposite ends of the synchronization spectrum. In the *synchronization-free* fragment, where timing constraints decouple across agents, a finite abstraction via product-region construction yields PSPACE-completeness. At the other extreme, the *always-synchronizing* fragment tightly couples agents through every interaction, yet every reachable configuration admits a monotone witness run, enabling a reduction to timed automata reachability—also PSPACE-complete. These results establish baseline complexity bounds and demonstrate that both maximal independence and maximal dependence can be decidable when properly formalized.

Bounded desynchronization. Between these extremes lies the conceptually rich territory of *partial synchronization*. Chapter 5 introduces *drift-bounded LTNs*, where local clocks may diverge but only within a bounded range. By adopting an existential notion of drift boundedness—requiring only that *some* execution witnesses the target configuration with bounded drift—the thesis shows that decidability can be restored without requiring global synchronization. Crucially, acyclic LTNs are proven drift-bounded, with reachability shown to be NP-complete (Theorem 5.15). This result highlights *bounded desynchronization* as a key semantic principle underlying decidability in timed interaction models.

Structural coherence. Finally, Chapter **??** explores structural restrictions that enforce global coherence without full synchronization. The notion of *connectedly communicating LTNs* demonstrates how interaction topology can be exploited to refine decidability boundaries, complementing the semantic restrictions on drift and synchronization.

Conceptual Framework

Beyond individual complexity results, this thesis contributes a conceptual framework for understanding the interplay between timing, synchronization, and structure in concurrent systems. Three recurring themes emerge:

1. **Locality vs. globality:** Decidability often hinges on whether timing information remains local or must be globally coordinated. The synchronization-free and drift-bounded fragments exploit locality, while the always-synchronizing fragment manages global coordination through tight coupling.

2. **Semantic vs. syntactic restrictions:** Existential drift boundedness is a semantic property (depending on reachable behaviors), yet it can be verified syn-

tactically for acyclic structures. This interplay between semantic guarantees and syntactic sufficient conditions recurs throughout the thesis.

3. **Monotonicity and normal forms:** Many decidability proofs rely on constructing canonical witnesses—monotone runs, truncated configurations, or region-normalized executions—that preserve reachability while admitting finite representation. These proof techniques suggest deeper connections to well-quasi-orderings and well-structured systems.

Taken together, the results delineate a rich landscape of timed interaction models and demonstrate that undecidability can be avoided through principled restrictions on how agents coordinate their timing behavior.

7.2 Broader Directions and Outlook

The framework of local-timed negotiations opens a broader research program centered on *quantitative verification under structural constraints*. The decidable fragments and proof techniques developed in this thesis suggest multiple directions for extending both the theoretical foundations and practical applicability of LTNs. Below, we outline core continuations of this work, organized by increasing scope and ambition.

7.2.1 Counting and Enumeration Problems

The finite abstractions and normal-form constructions developed for decidable LTN fragments suggest natural extensions to *counting problems*. Rather than asking whether a configuration is reachable, one can ask: *how many* distinct executions reach it? Or: how many reachable configurations satisfy a given property?

For synchronization-free LTNs, the product-region construction of Chapter **??** provides a finite graph whose paths correspond to executions. This immediately yields decidability of counting reachable configurations. However, the complexity of *exact counting* or *approximate counting* remains open. Similarly, for always-synchronizing LTNs, the monotone witness runs established in Theorem **??** admit a finite representation, but whether this enables efficient counting is unclear.

A particularly compelling question concerns acyclic and drift-bounded LTNs (Chapter 5). Since reachability is NP-complete (Theorem 5.15), natural follow-up questions include:

- Can reachable configurations be enumerated with polynomial delay?
- Does counting reachable configurations remain in #P, or does drift boundedness enable more efficient algorithms?

- Can standard approximation techniques (e.g., FPRAS) be adapted to count executions up to clock-region equivalence?

Answering these questions would connect LTNs to the growing literature on counting and enumeration in verification and would clarify whether the semantic restrictions ensuring decidability also yield computational tractability for quantitative queries.

7.2.2 Quantitative Extensions: Costs, Weights, and Counters

Chapter **??** deliberately restricts LTNs to clock constraints without accumulated quantitative effects such as costs or counters. A natural next step is to incorporate such features, enabling the study of resource-aware verification and optimization problems.

The drift-boundedness results of Chapter 5 are particularly suggestive here. Since decidability relies on bounding clock *divergence* rather than absolute time, it is plausible that additive cost models or bounded counters could be incorporated without destroying decidability. Concretely:

- **Cost-annotated LTNs:** Associate integer costs with interactions and ask whether a target configuration is reachable with total cost below a threshold. Does existential drift boundedness suffice for decidability of cost-bounded reachability?

- **Optimal reachability:** For always-synchronizing LTNs, can cost-optimal executions be found via reduction to priced timed automata?

- **Counter extensions:** Allow agents to maintain integer counters (e.g., modeling resource counts) that are incremented or decremented during interactions. Do acyclic counter-LTNs preserve NP-completeness, or does interaction with drift boundedness lead to undecidability?

These extensions would position LTNs as a framework for studying quantitative objectives in concurrent systems, bridging the gap between qualitative reachability analysis and optimization.

7.2.3 Structural Parameters and Well-Structuredness

Several results in this thesis point toward the importance of structure: acyclicity (Definition 5.13) and connected communication (Chapter **??**) both impose global coherence without requiring full synchronization. However, these restrictions were introduced somewhat *ad hoc*, motivated by specific decidability proofs rather than a systematic theory.

A more principled approach would be to study *parametric structural restrictions* on LTNs and characterize their impact on decidability and complexity. Promising parameters include:

- **Interaction hypergraph structure:** Does bounding the treewidth, pathwidth, or treedepth of the negotiation hypergraph yield new decidable fragments or improved complexity bounds?
- **Negotiation arity:** All results in this thesis apply to arbitrarily large interactions. Does restricting to binary or bounded-arity interactions simplify analysis or admit efficient algorithms?
- **Hierarchical patterns:** Can layered or modular interaction structures—common in protocol design—be exploited for compositional verification?

More ambitiously, the monotonicity and truncation techniques used in Lemmas **??** and **??** suggest connections to *well-quasi-orderings* (WQOs) and well-structured transition systems (WSTS). An open problem is whether abstract LTN configurations can be equipped with a WQO that respects reachability, thereby importing the powerful termination and coverability results from the WSTS literature. Such a connection would situate LTNs within a broader theoretical framework and could yield generic decidability results for rich classes of timed interaction systems.

7.2.4 Beyond Reachability: Broader Verification and Synthesis

While reachability captures the core decidability challenges in LTNs, many verification problems remain unexplored:

- **Safety and liveness:** Can invariants or bounded-response properties be verified using the region-based abstractions from Chapter **??**?
- **Quantitative specifications:** Can metric temporal logic or timed logics be model-checked over decidable LTN fragments?
- **Parametric verification:** Given a family of LTNs parameterized by timing constants, can one synthesize constraints ensuring reachability or drift boundedness?

The explicit interaction structure of LTNs also suggests new *synthesis* questions. For instance: given a partial specification of interactions and timing constraints, can one automatically complete it to ensure drift boundedness or enforce timely synchronization? Such problems extend the reachability analysis toward controller synthesis, where the goal is to construct strategies that guarantee desired timing properties in the presence of environmental uncertainty or adversarial agents.

7.2.5 Connections to Other Infinite-State Models

The expressiveness results implicit throughout this thesis—particularly the undecidability proofs that motivate restrictions—call for a deeper comparison between LTNs and other infinite-state concurrency models. While Chapter **??** positions LTNs as a synthesis of negotiations and timed automata, precise translations to related models remain open:

- **Timed Petri nets:** How do LTNs compare to time Petri nets or timed-arc Petri nets in expressiveness? Can drift boundedness be related to boundedness in Petri nets?
- **Communicating timed automata:** Networks of timed automata with message passing are known to be undecidable. Can the restrictions ensuring decidability in LTNs be translated to communication protocols, and vice versa?
- **Negotiation-VASS:** Untimed negotiations with counters have been studied recently. Can the techniques from LTNs inform decidability boundaries when both timing and counting are present?

Understanding these connections would clarify the relative power of LTNs and reveal whether principles like drift boundedness and structural coherence apply broadly across infinite-state models.

7.2.6 Toward Practical Modeling and Tool Support

Finally, LTNs suggest a modeling paradigm closely aligned with how distributed protocols are described in practice: via explicit interactions and local timing constraints rather than monolithic global states. This raises the question of whether LTNs can serve as a practical specification language.

A concrete direction is to develop *prototype tool support* for LTNs, targeting the decidable fragments identified in this thesis:

- Implement verification algorithms for acyclic and drift-bounded LTNs (Chapter 5).
- Build abstraction-based model checkers for synchronization-free and always-synchronizing fragments (Chapter **??**).
- Explore heuristic or approximate methods for more general LTNs, using drift boundedness as a runtime check.

Such tools would enable empirical evaluation on case studies drawn from distributed protocols, embedded systems, or cyber-physical applications. They could also guide the discovery of additional restrictions motivated by real-world modeling needs, creating a feedback loop between theory and practice.

Closing Remark

Local-timed negotiations were introduced in this thesis not as an isolated formalism, but as a lens through which concurrency and time can be studied together at the level of *interaction*. The decidability landscape charted here reveals that this lens is productive: it exposes new decidable classes, clarifies fundamental boundaries, and illuminates the subtle interplay between local autonomy and global coordination in real-time concurrent systems.

Yet this thesis represents only a beginning. The future directions outlined above—spanning counting, costs, structure, synthesis, and tool support—collectively point toward a broader research program on *quantitative and structural verification of interaction systems*. Such a program naturally aligns with current trends in formal methods, from parametric verification and well-structured systems to quantitative synthesis and resource-aware logics. It offers multiple entry points for both theoretical advances—connecting LTNs to established frameworks like WSTS or extending them with richer quantitative features—and practical applications in protocol analysis and embedded system design.

It is my hope that local-timed negotiations will serve not only as a theoretical contribution to the foundations of timed concurrency, but also as a generative framework—one that inspires new questions, enables new techniques, and ultimately deepens our understanding of how time, interaction, and structure shape the behavior of concurrent systems.